



Universitat
de les Illes Balears

LABORATORI DE PROJECTES SOFTWARE

DEMOCHAIN

Jordi Vanyó Mestre
Dylan Luigi Canning Garcia
Antonio Contestí Coll
Marc Link Cladera

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Any acadèmic 2025-26

SUMARI

Sumari	i
Acrònims	v
Resum	vii
1 Introducció	1
1.1 Presentació de l'equip	1
1.2 Context del problema	2
1.3 Motivació	2
1.3.1 Motivació tècnica	2
1.3.2 Motivació acadèmica	3
1.3.3 Motivació personal	3
1.4 Objectius del projecte	3
1.5 Contribucions del projecte	4
1.6 Abast i no objectius	5
1.7 Enfocament metodològic	6
1.8 Estructura del document	6
2 Estat de l'art	9
2.1 Experiències nacionals de votació electrònica	9
2.1.1 Estònia: l'e-voting com a cas paradigmàtic	9
2.1.2 Suïssa: el cas SwissPost i la suspensió del 2019	10
2.1.3 Estats Units: l'aplicació Voatz a West Virginia	10
2.1.4 Lliçó general	10
2.2 Sistemes acadèmics de votació verificable	11
2.2.1 Helios	11
2.2.2 Belenios	11
2.2.3 Microsoft ElectionGuard	11
2.3 Projectes contemporanis sobre blockchain	12
2.3.1 DAVINCI Protocol (Vocdoni)	12
2.3.2 Lliçons recollides	12
3 Planificació	15
3.1 Cicle de vida del projecte	15
3.1.1 Influència de la recerca sobre la implementació	16
3.2 Organització del backlog	16

3.2.1	Jerarquia de <i>issues</i>	16
3.2.2	Definition of Ready (DoR)	17
3.2.3	Definition of Done (DoD)	17
3.3	Inventari d'èpiques	18
3.4	Sprints executats	18
3.5	Camins crítics	19
3.5.1	Camí crític 1: Recerca → contracte Algorand	19
3.5.2	Camí crític 2: Contracte Algorand → client web	19
3.5.3	Camí crític 3: Contracte Algorand → ancoratge	20
3.5.4	Camí crític 4: Infraestructura local de desenvolupament	20
3.6	Paquets de treball	20
3.7	Política de versionat del codi	21
3.7.1	Estructura de branques	21
3.7.2	Convencions de <i>commit</i>	21
3.7.3	Procés de <i>Pull Request</i>	22
3.8	Reducció d'abast: de l'MVP criptogràfic a l'MVP en clar	22
4	Anàlisi del Sistema	25
4.1	Identificació de Parts Interessades (<i>Stakeholders</i>)	25
4.2	Recerca Criptogràfica i de Consens per a un Disseny Robust	27
4.2.1	Desviació del Disseny i Limitacions del Prototip MVP	28
4.3	Especificació Funcional	29
4.3.1	Cicle de Vida del Sufragi i Casos d'Ús	29
4.3.2	Metodologia del Backlog i Històries d'Usuari	30
4.3.3	Requisits de Producte	31
4.4	Especificació No Funcional i Paràmetres de Qualitat	31
4.4.1	Requisits No Funcionals	31
4.4.2	Paràmetres de Qualitat	32
4.5	Restriccions	32
4.5.1	Restriccions Tecnològiques	33
4.5.2	Restriccions Legals i de Privacitat	33
4.5.3	Restriccions d'Aprenentatge i Terminis del Projecte	34
4.6	Decisions de Disseny Global	34
4.7	Mètode de Recompte	35
4.7.1	El Teorema d'Impossibilitat d'Arrow	35
4.7.2	Comparativa dels Mètodes de Votació i el Criteri de Condorcet	36
4.7.3	Inconvenients del Mètode Schulze	37
4.8	Llenguatge Ubic (DDD)	37
4.9	Model de Dades	38
4.9.1	Estructures d'Algorand (ARC-4 Structs en algopy)	38
4.9.2	Estructures de Dades d'Ethereum (NotaryContract.sol)	39
4.9.3	Procés de Derivació del <i>hash</i> Canònic	40
5	Disseny	41
5.1	Arquitectura: visió C4	41
5.1.1	Nivell 1 - Context	41
5.1.2	Nivell 2 - Contenedors	42

5.1.3	Nivell 3 - Components	44
5.2	Patró de desplegament	49
5.2.1	Stack Docker Compose	49
5.2.2	Inicialització i flux APP_ID	49
5.2.3	Aixecar un node d'Algorand	50
5.3	Disseny de la interfície d'usuari	50
5.3.1	Llengües, tema i navegació global	50
5.3.2	Gestió d'organitzacions	50
5.3.3	Propostes i cicle de vida	51
5.3.4	Creació d'una proposta	51
5.3.5	Emissió de vot	51
5.3.6	Resultats i verificació pública	51
5.4	Flux de comunicació amb l'usuari	51
5.4.1	Gestió d'organitzacions	52
5.4.2	Crear una proposta	54
5.4.3	Votar l'aprovació d'una proposta	54
5.4.4	Emetre el vot d'elecció (ranking)	55
5.5	Flux autònom i automàtic	56
5.5.1	Ancoratge a Ethereum	56
6	Desenvolupament	59
6.1	Organització de l'equip	59
6.1.1	Rols i responsabilitats	59
6.1.2	Dinàmica de treball	60
6.2	Organització del backlog	60
6.2.1	Materialització de la jerarquia	60
6.2.2	Volum i estat final del backlog	61
6.3	Resum de les iteracions de desenvolupament	61
6.3.1	Sprint 1 – Recerca preliminar (febrer-març)	61
6.3.2	Sprint 2 – Decisions arquitectòniques (març)	62
6.3.3	Sprint 3 – Primer prototip de votació (març – 12 abril)	62
6.3.4	Sprint 4 – Schulze, resultats i integració (17–30 abril)	62
6.3.5	Sprint 5 – Testing, refinament i robustesa (1–14 maig)	62
6.3.6	Sprint 6 – Documentació, demo i entrega (15–25 maig)	63
6.4	Versions generades i entregades	63
6.4.1	Recerca obtinguda setmanalment	63
6.4.2	Versions intermèdies del producte	64
6.4.3	Versió final entregada	64
6.5	Problemes trobats i la seva resolució	64
6.5.1	Criptografia avançada fora de l'abast de l'MVP	64
6.5.2	Falta de coneixement inicial	65
6.6	Demostracions funcionals	66
6.6.1	Demostracions intermèdies	66
6.6.2	Demostració de la versió final	66
7	Conclusions	69
7.1	Producte final entregat	69

7.1.1	Què s'ha fet	69
7.1.2	Què s'ha decidit no fer (i no es farà)	70
7.1.3	Què no s'ha fet, (però es farà més endavant)	71
7.1.4	Posada en producció	71
7.1.5	Manteniment	72
7.2	Lliçons apreses	72
7.2.1	Què ha funcionat	73
7.2.2	Què no ha funcionat	73
7.2.3	El funcionament com a equip	74
7.2.4	El producte: orgull i autocrítica	74
7.3	Aportació a les parts interessades	74
7.4	Tancament i línies de treball futures	75
8	Ús de la intel·ligència artificial	77
8.1	Eines d'intel·ligència artificial utilitzades	77
8.1.1	Models de llenguatge (LLM)	78
8.1.2	Eines de prototipatge i generació d'interfícies	78
8.2	Limitacions dels models de llenguatge	78
8.3	IA com a eina de suport a l'aprenentatge	79
A	Annexos	81
A.1	Captures de pantalla de la interfície	81
A.1.1	Pantalla d'inici	81
A.1.2	Gestió d'organitzacions	82
A.1.3	Propostes	83
A.1.4	Creació d'una proposta	87
A.1.5	Emissió de vot	91
A.1.6	Resultats i verificació pública	93
	Bibliografia	97

ACRÒNIMS

ABI Application Binary Interface

ADR Architecture Decision Record

API Application Programming Interface

ARC Algorand Request for Comments

AVM Algorand Virtual Machine

CI Continuous Integration

CSV Comma-Separated Values

DoD Definition of Done

DoR Definition of Ready

EIP Ethereum Improvement Proposal

EVM Ethereum Virtual Machine

IdP Identity Provider

JSON JavaScript Object Notation

KMD Key Management Daemon

MVP Minimum Viable Product

PR Pull Request

SHA Secure Hash Algorithm

TFG Treball de Fi de Grau

UIB Universitat de les Illes Balears

US User Story

ZK Zero-Knowledge

ZK-SNARK Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

RESUM

Aquest projecte presenta *Demochain*, un sistema de votació descentralitzat de propòsit general construït sobre la xarxa Algorand i ancorat periòdicament a la xarxa pública Ethereum. El sistema permet crear organitzacions amb un cens d'adreces autoritzades, definir propostes amb una fase prèvia d'aprovació per quòrum de dos terços i celebrar eleccions amb vot preferencial. El recompte final es calcula mitjançant el mètode de Schulze, que garanteix propietats formals com la monotonicitat, la independència de clons i la consistència amb Condorcet. Per garantir la integritat del resultat sense dependre d'una única xarxa, un servei d'ancoratge llegeix l'estat final de la cadena Algorand, calcula un *hash* determinista i el publica conjuntament amb la resta de nodes participants en el contracte notari *NotaryContract* desplegat a la *testnet* Sepolia d'Ethereum, on un mecanisme de consens *K-of-N* fixa el *hash* de manera immutable. El client web, construït amb React i TypeScript, ofereix verificació pública del vot individual per a qualsevol observador. El projecte s'ha desenvolupat amb pràctiques modernes d'enginyeria del *software*: *backlog* jerarquitzat a GitHub Projects amb èpiques, històries d'usuari i tasques; control de versions amb *feature branches* i revisió creuada obligatòria; arquitectura documentada amb el model C4; i decisions de disseny recollides en *Architecture Decision Records*. L'abast actual constitueix un Minimum Viable Product (MVP) funcional que demostra la viabilitat arquitectònica del plantejament, deixant la incorporació de privacitat criptogràfica avançada (xifratge homomòrfic ElGamal, proves zk-SNARK i cerimònia de desxifrat amb *trustees*) com a treball futur.

Paraules clau: *blockchain*, Algorand, Ethereum, votació electrònica, vot preferencial, mètode de Schulze, ancoratge, *smart contract*, verificabilitat pública.

Resumen

Este proyecto presenta *Demochain*, un sistema de votación descentralizado de propósito general construido sobre la red Algorand y anclado periódicamente a la red pública Ethereum. El sistema permite crear organizaciones con un censo de direcciones autorizadas, definir propuestas con una fase previa de aprobación por quórum de dos tercios y celebrar elecciones con voto preferencial. El recuento final se calcula mediante el método de Schulze, que garantiza propiedades formales como la monotonía, la independencia de clones y la consistencia con Condorcet. Para garantizar la integridad del resultado sin depender de una única red, un servicio de anclaje lee el estado final de la cadena Algorand, calcula un *hash* determinista y lo publica junto con el resto de nodos participantes en el contrato notario *NotaryContract* desplegado en la *testnet*

Sepolia de Ethereum, donde un mecanismo de consenso *K-of-N* fija el *hash* de forma inmutable. El cliente web, construido con React y TypeScript, ofrece verificación pública del voto individual para cualquier observador. El proyecto se ha desarrollado aplicando prácticas modernas de ingeniería del software: *backlog* jerarquizado en GitHub Projects con épicas, historias de usuario y tareas; control de versiones *confeature branches* y revisión cruzada obligatoria; arquitectura documentada con el modelo C4; y decisiones de diseño recogidas en *Architecture Decision Records*. El alcance actual constituye un **MVP** funcional que demuestra la viabilidad arquitectónica del planteamiento, dejando la incorporación de privacidad criptográfica avanzada (cifrado homomórfico ElGamal, pruebas zk-SNARK y ceremonia de descifrado con *trustees*) como trabajo futuro.

Palabras clave: *blockchain*, Algorand, Ethereum, voto electrónico, voto preferencial, método de Schulze, anclaje, *smart contract*, verificabilidad pública.

Abstract

This project presents *Demochain*, a general-purpose decentralised voting system built on the Algorand blockchain and periodically anchored to the public Ethereum network. The system allows users to create organisations with a census of authorised addresses, define proposals that go through a two-thirds quorum approval phase, and run elections with preferential voting. The final tally is computed using the Schulze method, which guarantees formal properties such as monotonicity, clone independence, and Condorcet consistency. To ensure the integrity of the result without relying on a single network, an anchoring service reads the final state of the Algorand chain, computes a deterministic *hash*, and submits it together with the other participating nodes to the *NotaryContract* smart contract deployed on Ethereum's Sepolia testnet, where a K-of-N consensus mechanism fixes it immutably. The web client, built with React and TypeScript, offers public verification of individual votes for any observer. The project has been developed applying modern software engineering practices: a hierarchical backlog on GitHub Projects with epics, user stories, and tasks; version control with feature branches and mandatory cross-review; architecture documented with the C4 model; and design decisions recorded in *Architecture Decision Records*. The current scope constitutes a functional **MVP** that demonstrates the architectural viability of the approach, leaving the integration of advanced cryptographic privacy (ElGamal homomorphic encryption, zk-SNARK proofs, and threshold decryption ceremony with *trustees*) as future work.

Keywords: blockchain, Algorand, Ethereum, electronic voting, preferential voting, Schulze method, anchoring, smart contract, public verifiability.

INTRODUCCIÓ

Aquest capítol presenta el projecte **Demochain**, un sistema de votació descentralitzada construït sobre la xarxa *blockchain* d'Algorand i ancorat periòdicament a Ethereum. L'objectiu d'aquest capítol és situar el projecte en el seu doble context –acadèmic i tècnic–, exposar la problemàtica que pretén abordar, repassar els antecedents més rellevants en l'àmbit de la votació electrònica verificable, justificar la motivació de l'equip i fixar els objectius i l'abast del qual s'entrega. La discussió detallada de cada aspecte arquitectònic i d'enginyeria es deixa per als capítols posteriors.

1.1 Presentació de l'equip

El projecte l'hem desenvolupat un equip de quatre sense experiència prèvia en desenvolupament de *blockchain* ni en criptografia aplicada en començar el projecte; tots compartim formació en Python i, en menor grau, en *front-end* amb tecnologies basades en JavaScript. Aquest punt de partida ha condicionat fortament tant la planificació –amb una fase inicial de recerca explícita– com l'abast final del producte entregat.

Jordi Vanyó Mestre – Responsable del client web i de la integració amb Algorand. Lidera el desenvolupament del *front-end* en React i TypeScript, l'arquitectura del repositori i la interacció directa amb el contracte d'Algorand (codificació i descodificació Algorand Request for Comments (**ARC**)-4, gestió de *boxes* i signatura de transaccions des del navegador).

Dylan Luigi Canning Garcia – Responsable del sistema d'ancoratge i del contracte notari a Ethereum. Encarregat del servei en Python que llegeix l'estat de la cadena Algorand, en calcula un *hash* determinista i el publica al contracte NotaryContract desplegat a la *testnet* Sepolia.

Antonio Contestí Coll – Responsable de la coordinació del projecte i de la documentació. Encarregat de la gestió del *backlog* a GitHub Projects, del seguiment dels

sprints, de la coherència de la documentació tècnica i de l'enllaç entre els diferents paquets de treball.

Marc Link Cladera – Responsable del *smart contract* d'Algorand i de l'arquitectura general. Lidera el disseny del contracte Demochain en *algopy*, els diagrames del model C4 [1] i les decisions arquitectòniques recollides als Architecture Decision Records (ADRs).

La distribució de responsabilitats no és estricta. Totes les decisions arquitectòniques rellevants s'han pres en reunions setmanals conjuntes, i tot el codi ha entrat a la branca principal a través de *pull requests* amb revisió creuada (vegeu secció 3.7 del capítol 3).

1.2 Context del problema

La transformació digital de processos col·lectius –consultes, primàries, eleccions internes d'entitats, referèndums– ha posat de manifest una tensió de fons: els sistemes de votació electrònica desplegats fins ara tendeixen a substituir un punt únic de confiança (el responsable del recompte físic) per un altre (el responsable del sistema informàtic). Un servidor central que rep, emmagatzema i recompta vots és, per definició, una entitat en qui s'han de dipositar fiançament tant la integritat del recompte com la privacitat del vot individual. La promesa de la tecnologia *blockchain* aplicada a la votació és precisament l'eliminació d'aquest punt únic de confiança.

Els antecedents internacionals d'aquesta problemàtica –casos desplegats a escala nacional com els d'Estònia, Suïssa o els Estats Units–, els sistemes acadèmics que defineixen el referent criptogràfic actual (Helios, Belenios, ElectionGuard) i els projectes contemporanis sobre *blockchain* (DAVINCI Protocol) es revisen en detall al capítol 2 (*Estat de l'art*). En aquesta introducció ens limitem a constatar que cap d'aquests sistemes no s'ajusta plenament al cas d'ús d'una organització de mida moderada –una universitat, una entitat cívica o un col·lectiu professional– que vulgui un sistema obert, descentralitzat, de baix cost operatiu i amb verificabilitat pública. El nostre projecte explora precisament aquesta franja.

1.3 Motivació

La motivació del projecte s'articula en tres plànols complementaris:

1.3.1 Motivació tècnica

L'ecosistema *blockchain* ha viscut una maduració significativa durant els darrers anys. Plataformes com Algorand [2] ofereixen ara propietats que fan tècnicament viable un sistema de votació seriós: finalitat de bloc en pocs segons, comissions menyspreables (de l'ordre de la mil·lèsima de cèntim per transacció), suport nadiu per a *smart contracts* expressius mitjançant *algopy* [3] i operacions criptogràfiques avançades a nivell de màquina virtual (incloent-hi opcodes BN254 [4] per a la verificació de proves zk-SNARK). El conjunt d'aquestes propietats no estava disponible fa cinc anys.

Paral·lelament, Ethereum continua sent la xarxa pública més consolidada i auditada [5]. Tot i el seu cost elevat per a operacions intensives, la seva fiabilitat com a registre

immutable de fets és superior a la de qualsevol alternativa. Això la fa idònia com a *capa de notaria* per a fets ocorreguts en altres xarxes.

La combinació d'aquestes dues plataformes –una xarxa permissionada ràpida i barata per a la lògica electoral, una xarxa pública robusta per al registre immutable del resultat– representa una direcció arquitectònica que mereix exploració empírica.

1.3.2 Motivació acadèmica

El projecte ofereix un escenari pràcticament ideal per posar a prova de manera coherent un conjunt ampli de pràctiques d'enginyeria de *software*:

- Gestió iterativa del *backlog* amb GitHub Projects, *sprints* [6] i criteris d'acceptació en format *Donat/Quan/Aleshores*.
- Documentació arquitectònica mitjançant el model C4 [1] i ADRs per a totes les decisions significatives.
- Control de versions amb *feature branches*, *pull requests* amb revisió creuada obligatòria i *Conventional Commits* [7].
- Integració contínua amb GitHub Actions, *git hooks* locals i comprovacions automàtiques de format i convenció.

La complexitat tècnica del producte (multilingüe quant a *stack*: Python, TypeScript, Solidity; multinivell: *front-end*, *smart contracts* en dues cadenes, servei d'ancoratge) força que aquestes pràctiques no siguin un afegit decoratiu sinó una necessitat operativa.

1.3.3 Motivació personal

A nivell personal, el projecte combina dos ingredients que el fan atractiu per a un equip d'estudiants en fase final de grau. D'una banda, l'aprenentatge significatiu d'una tecnologia emergent –*blockchain*, *smart contracts*– sobre la qual cap dels membres tenia coneixement previ. De l'altra, l'oportunitat de treballar en un problema d'impacte social potencial: la votació és un mecanisme fonamental de qualsevol col·lectiu organitzat, i contribuir a millorar-ne la transparència té un valor que va més enllà del purament tècnic.

1.4 Objectius del projecte

L'objectiu general del projecte és **dissenyar, implementar i documentar un sistema de votació descentralitzat de propòsit general**, basat en una xarxa permissionada d'Algorand i ancorat a Ethereum, que serveixi com a prova tècnicament sòlida de concepte sobre la qual es puguin construir extensions futures –notablement la capa de privacitat criptogràfica.

Concretament, el sistema entregat ha de satisfer els objectius específics següents:

1. **Gestió d'organitzacions i cens.** Qualsevol usuari ha de poder crear una organització amb nom únic, esdevenir-ne administrador automàticament i gestionar un cens d'adreces autoritzades. La càrrega massiva d'adreces s'ha de poder fer mitjançant un fitxer Comma-Separated Values (CSV) de manera transparent per a l'administrador.
2. **Propostes amb fase d'aprovació.** Un membre del cens d'una organització ha de poder crear propostes amb un conjunt d'opcions i un rang temporal de votació. Abans de la data d'inici, la comunitat decideix per quòrum de dos terços si la proposta arribarà a celebrar-se com a elecció.
3. **Votació preferencial.** Quan una proposta es converteix en elecció, els membres del cens han de poder emetre un *rànquing* complet de les opcions disponibles. El recompte final s'ha de calcular pel mètode de Schulze [8], que garanteix propietats formals desitjables com la monotonicitat, la independència de clons i la consistència amb Condorcet.
4. **Verificació pública del vot.** Qualsevol persona, sigui o no membre del cens, ha de poder consultar el contingut del vot emès per una adreça arbitrària directament des de la cadena d'Algorand.
5. **Ancoratge multinode a Ethereum.** En finalitzar una elecció, diversos nodes han de llegir independentment l'estat de la cadena Algorand, calcular el mateix *hash* determinista i enviar-lo al contracte notari d'Ethereum. Quan un nombre suficient de nodes coincideix (consens *K-of-N*), el resultat queda ancorat de manera immutable a Sepolia.
6. **Reproductibilitat del desplegament.** Tot el sistema –*LocalNet* d'Algorand, contracte, *indexer*, *client* web i serveis auxiliars– s'ha de poder aixecar localment amb una sola comanda mitjançant Docker Compose.
7. **Pràctiques d'enginyeria del *software* aplicades amb rigor.** *Backlog* estructurat en èpiques, històries d'usuari i tasques amb criteris d'acceptació explícits; control de versions amb *feature branches* i Pull Request (PR) obligatòria; *Conventional Commits*; ADRs documentats; arquitectura descrita amb el model C4.

1.5 Contribucions del projecte

En relació amb els antecedents discutits al capítol 2, el projecte *Demochain* aporta una combinació de decisions de disseny que el diferencien dels sistemes existents:

- **Algorand com a capa primària.** La gran majoria de sistemes basats en *blockchain* es construeixen sobre Ethereum o variants d'aquesta. La tria d'Algorand com a cadena d'execució del contracte de votació és poc habitual i prové d'una anàlisi documentada (ADR 001) que pondera el cost per transacció, la finalitat ràpida del bloc i la disponibilitat de contractes intel·ligents expressius programables en Python.

- **Ancoratge *K-of-N* a Ethereum com a notari independent.** En lloc de confiar la integritat del resultat exclusivament a la cadena d'execució, el projecte hi afegeix una capa explícita de notaria: un conjunt de nodes independents calculen el *hash* del resultat de manera determinista i el publiquen conjuntament al contracte *NotaryContract* d'Ethereum. La propietat aconseguida és que cap entitat individual –ni Algorand, ni cap node concret– pot alterar el resultat unilateralment sense que això sigui públicament detectable.
- **Recompte amb el mètode de Schulze.** Els sistemes de votació basats en *blockchain* acostumen a oferir recompte per pluralitat o, en el millor cas, vot en doble volta. *Demochain* implementa el mètode de Schulze [8], que permet capturar matisos de preferència que un sistema de pluralitat descarta i ofereix propietats formals demostrables.
- **Codi obert pensat per a desplegament real.** El repositori inclou el *stack* complet (contracte, client, ancoratge, infraestructura amb Docker Compose) llest per ser desplegat per una entitat universitària o associativa amb recursos modestos. No és un experiment de laboratori sinó una base productiva.

És útil situar el projecte respecte a DAVINCI [9], que és el sistema contemporani més proper. DAVINCI opta per construir una capa pròpia de tipus *Layer 2* sobre Ethereum amb una xarxa de *sequencers* dedicats i un *zkVM* propi. Aquest enfocament és més ambiciós i tènicament més pesat, però també exigeix una infraestructura especialitzada per ser desplegat. *Demochain* adopta una estratègia complementària: emprar una cadena pública de propòsit general (Algorand) com a capa de votació, recolzar-se en una altra cadena pública (Ethereum) com a capa de notaria, i mantenir el component d'ancoratge com un servei petit i auditable en Python que qualsevol universitat o entitat pot operar amb recursos modestos. Es tracta de dues respostes diferents d'una mateixa pregunta de fons: com aconseguir verificabilitat pública i resistència a manipulació sense centralitzar la confiança.

Cal subratllar, finalment, que aquestes contribucions són de disseny i d'integració. El projecte no aporta cap primitiva criptogràfica original ni cap protocol nou: tots els components criptogràfics emprats estan ben establerts a la literatura.

1.6 Abast i no objectius

És important deixar constància, ja a la introducció, dels límits del que s'entrega. L'MVP actual **no inclou** privacitat criptogràfica del vot. Concretament, queden fora de l'abast d'aquesta versió:

- Xifratge homomòrfic dels vots (esquema ElGamal exponencial [10]), tal com el descriu Adida [11].
- Proves de coneixement zero (**ZK-SNARKs** *Groth16*) [12, 13] per acreditar l'elegibilitat del votant sense revelar-ne la identitat.
- Generació distribuïda de claus [14] i cerimònia de desxifrat amb *trustees* i proves Chaum-Pedersen [15], segons l'esquema de Belenios [16].

- Integració amb un proveïdor d'identitat (Identity Provider (**IdP**)) extern per a l'autenticació del votant.

Aquests components estaven contemplats al *backlog* inicial (èpiques E3, E4, E5 i E6 al repositori [17]) i es van descartar de l'abast de l'**MVP** al final del primer mes de desenvolupament, en constatar que la seva complexitat depassava l'horitzó temporal del projecte. La justificació detallada d'aquesta decisió i la seva planificació com a treball futur es recullen al capítol 3 (secció 3.8) i al capítol de conclusions.

Cal subratllar que el sistema entregat **sí que ofereix verificabilitat pública** del resultat: qualsevol persona pot consultar els vots individuals a la cadena d'Algorand i comprovar que el *hash* ancorat a Ethereum coincideix amb l'estat real de l'elecció. Aquesta propietat ja és, per ella mateixa, una millora substancial respecte als sistemes de votació centralitzats convencionals; el que queda fora és la capa de **privacitat** criptogràfica, que protegiria el vot individual de la inspecció pública.

1.7 Enfocament metodològic

L'execució del projecte ha adoptat un enfocament **iteratiu i incremental** inspirat en Scrum [6], amb les adaptacions necessàries per a un equip acadèmic petit. La metodologia es resumeix en quatre pràctiques principals que es desenvolupen amb detall al capítol 3:

1. **Recerca prèvia documentada.** Abans de prendre cap decisió arquitectònica significativa, s'ha investigat l'espai de solucions i s'ha documentat la decisió en un Architecture Decision Record (**ADR**). Els dos **ADRs** actuals (Architecture Decision Record 000 sobre llenguatge de *smart contracts* i Architecture Decision Record 001 sobre plataforma *blockchain*) consten al repositori.
2. **Planificació amb *backlog* jerarquitzat.** Èpiques, històries d'usuari, *spikes* i tasques estructurades a GitHub Projects amb identificadors traçables (E1, E2-User Story (**US**)1, E2-**US**1-T3, etc.) i criteris d'acceptació formals.
3. **Arquitectura documentada amb C4.** Diagrames de context, contenidors i components mantinguts al directori docs/diagrams/ del repositori, ampliat a mesura que el sistema creix [1].
4. **Control de versions amb revisió creuada obligatòria.** Tot el codi entra a la branca principal via *pull request*, amb *check* automàtic de format i convenció i revisió manual d'almenys un altre membre de l'equip.

1.8 Estructura del document

La resta del document s'organitza en sis capítols més, amb una estructura que combina les fases conceptuals d'un projecte d'enginyeria del *software* amb una secció final de tancament.

- El **capítol 2 (Estat de l'art)** revisa les experiències nacionals de votació electrònica desplegadas a Estònia, Suïssa i els Estats Units, els sistemes acadèmics consolidats

(Helios, Belenios, ElectionGuard) i els projectes contemporanis sobre *blockchain* (DAVINCI Protocol), i situa el nostre projecte en aquest panorama.

- El **capítol 4 (Anàlisi)** identifica les parts interessades del sistema, exposa l'especificació funcional –casos d'ús, històries d'usuari i requisits de producte– i l'especificació no funcional –requisits de qualitat i restriccions tecnològiques, legals i de calendari. Inclou també el llenguatge ubic del domini, el model de dades de les estructures emmagatzemades al contracte i una síntesi de la recerca prèvia sobre *blockchain* i criptografia.
- El **capítol 5 (Disseny)** presenta l'arquitectura del sistema mitjançant el model C4 (diagrames de context, contenidors i components per a cada peça del sistema), descriu el patró de desplegament basat en Docker Compose i exposa el disseny de la interfície d'usuari amb les pantalles del client i els diagrames de seqüència de les interaccions principals.
- El **capítol 3 (Planificació)** detalla la gestió del cicle de vida del projecte: l'estructura del *backlog*, les Definition of Ready (**DoR**) i Definition of Done (**DoD**), els *sprints* executats, els camins crítics identificats i la política de versionat del codi.
- El **capítol 6 (Desenvolupament)** descriu l'execució real del projecte: l'organització de l'equip, la traça de les iteracions, les versions generades, els problemes trobats i les seves resolucions, i les demostracions funcionals del producte final.
- El **capítol 7 (Conclusions)** valora el producte entregat respecte als objectius inicials, exposa el que s'ha fet, el que no s'ha fet i el que queda per fer, reflexiona sobre el funcionament de l'equip i les lliçons apreses, i identifica les línies de treball futures més rellevants.

El codi font del projecte, els **ADRs**, els diagrames C4 i la traça completa del *backlog* estan disponibles al repositori públic de GitHub [17].

ESTAT DE L'ART

La votació electrònica és un camp amb una història llarga i un nombre significatiu d'experiències reals desplegades a diferents països i contextos institucionals. Aquest capítol revisa, en primer lloc, els casos d'ús desplegats a escala nacional que han marcat el debat internacional sobre la viabilitat del vot electrònic; en segon lloc, els sistemes acadèmics que defineixen el referent criptogràfic actual de la votació verificable; i, en tercer lloc, els projectes contemporanis que estan combinant criptografia avançada amb infraestructura *blockchain*. L'objectiu és situar el projecte *Demochain* en un panorama tècnic concret abans d'entrar en l'anàlisi i el disseny del sistema.

2.1 Experiències nacionals de votació electrònica

La transformació digital de processos col·lectius –consultes, primàries, eleccions internes d'entitats, referèndums– ha posat de manifest una tensió de fons: els sistemes de votació electrònica desplegats fins ara tendeixen a substituir un punt únic de confiança (el responsable del recompte físic) per un altre (el responsable del sistema informàtic). El problema no és intrínsec a la digitalització, sinó al model arquitectònic. Un servidor central que rep, emmagatzema i recompta vots és, per definició, una entitat en qui s'han de dipositar fiançament tant la integritat del recompte com la privacitat del vot individual.

L'experiència internacional dels darrers vint anys mostra el ventall de resultats que ha donat aquest model.

2.1.1 Estònia: l'e-voting com a cas paradigmàtic

Estònia és el cas paradigmàtic d'adopció a gran escala. El sistema *i-Voting* permet el vot a Internet per a eleccions nacionals vinculants des de 2007 [18] i la proporció de votants per Internet ha crescut de manera sostinguda fins a superar el 50% en eleccions recents. El sistema es recolza en el document d'identitat digital nacional (eID) i en

un protocol de doble sobre electrònic: el vot intern es xifra amb la clau pública de la comissió electoral i s'envolta d'una signatura amb la clau privada del votant.

Tot i ser un cas d'èxit pel que fa al desplegament massiu, l'e-voting estonià ha estat objecte d'anàlisi de seguretat externa. Springall et al. [19] van publicar el 2014 un estudi detallat que identifica vulnerabilitats serioses en aspectes operacionals i procedimentals –la gestió del *software* del client, la construcció de les imatges del sistema operatiu d'arrencada, els controls físics al centre d'operacions– que il·lustren la dificultat de garantir totes les propietats desitjades en un sistema centralitzat. Les autoritats estonianes van respondre amb millores incrementals, però el model arquitectònic centralitzat es manté.

2.1.2 Suïssa: el cas SwissPost i la suspensió del 2019

Suïssa va desplegar entre 2018 i 2019 el sistema de *e-voting* de SwissPost, basat en una implementació de Scytl. El sistema declarava oferir verificabilitat universal mitjançant proves de coneixement zero sobre les decisions de mescla (*mixnet*) que reordenen els vots.

El març de 2019 un equip de criptògrafs independents [20] va publicar una anàlisi que demostrava que les proves de Bayer-Groth implementades empraven *commitments* amb *trapdoor*, una propietat que en principi hauria de ser absent. Aquesta debilitat permetria, en escenaris específics, alterar el resultat sense que la verificació detectés cap incidència. La publicació va portar a la suspensió del sistema. Només va tornar a desplegar-se anys més tard amb una reescriptura completa i un règim d'auditoria pública del codi.

2.1.3 Estats Units: l'aplicació Voatz a West Virginia

Als Estats Units, l'aplicació *Voatz* es va emprar en eleccions federals primàries de l'estat de West Virginia el 2018, dirigida inicialment a votants residents a l'estranger. *Voatz* promocionava l'ús de tecnologia *blockchain* per garantir la integritat dels vots, però l'arquitectura interna era essencialment centralitzada: la *blockchain* s'emprava com a capa de registre d'auditoria secundària, no com a capa primària d'execució.

L'anàlisi de seguretat publicada per Specter, Koppel i Weitzner del MIT el 2020 [21] va revelar vulnerabilitats que permetien alterar, observar i invalidar el vot dels usuaris, així com problemes estructurals en el model de seguretat declarat per l'aplicació. La conclusió dels autors –reflectida al títol de l'article, *The Ballot is Busted Before the Blockchain*– és que la mera incorporació de tecnologia *blockchain* no garanteix la integritat del sistema si l'arquitectura de l'aplicació presenta defectes en altres capes.

2.1.4 Lliçó general

Aquests tres casos no qüestionen la viabilitat de la votació electrònica en si: el problema no és la digitalització, sinó com es distribueix la confiança. La pregunta de fons és si és possible construir un sistema en què cap entitat individual –ni l'administrador, ni el desenvolupador, ni el proveïdor d'infraestructura– pugui alterar o manipular el resultat sense que això sigui detectable. És precisament aquesta la promesa de la tecnologia *blockchain* aplicada a la votació, sempre que s'apliqui amb honestat i no com a element de *marketing*.

2.2 Sistemes acadèmics de votació verificable

L'aplicació de la criptografia a la votació electrònica verificable té una tradició acadèmica madura. Els projectes següents han estat referents directes o indirectes durant la fase de recerca del nostre projecte.

2.2.1 Helios

Helios [11], presentat per Ben Adida al USENIX Security Symposium de 2008, és un sistema obert de votació web amb verificabilitat universal. La seva contribució fonamental és la combinació de *mixnets* per garantir l'anonimat dels vots i xifratge ElGamal [10] amb proves de coneixement zero [12] per permetre que qualsevol persona pugui verificar que els vots s'han inclòs al recompte sense conèixer-ne el contingut.

Helios s'ha emprat per a eleccions internes d'institucions acadèmiques: la *International Association for Cryptologic Research* (IACR), la Universitat Catòlica de Lovaina, la Universitat de Princeton, entre d'altres. El seu model d'amenaques, però, es basa en un servidor central que executa el *tally* (el càlcul del recompte): una sola entitat pot bloquejar el procés si es nega a publicar el resultat. Aquest punt feble va motivar el desenvolupament de Belenios.

2.2.2 Belenios

Belenios [16], desenvolupat per l'equip de Véronique Cortier a INRIA Nancy, és una evolució conceptual d'Helios. Aporta dues novetats principals: xifratge *threshold* ElGamal [10] i una cerimònia de desxifrat distribuïda entre múltiples *trustees*. Amb aquest esquema, cap *trustee* per separat pot desxifrar un vot individual; només una majoria qualificada (*p.ex.*, 3 de 5) pot reconstruir la clau privada per descomprimir el recompte agregat.

Aquesta arquitectura elimina el punt únic de fallada del *tally authority* d'Helios, però introdueix una complexitat operativa addicional: la cerimònia de desxifrat requereix coordinació entre els *trustees*, cadascun amb la seva clau, i si massa *trustees* es perden o es neguen a participar, el resultat no es pot calcular. Belenios s'empra activament per a eleccions universitàries franceses.

2.2.3 Microsoft ElectionGuard

ElectionGuard [22] és un SDK obert publicat per Microsoft Research el 2019 per a votació verificable *end-to-end*. La seva proposta principal és aportar una capa criptogràfica reutilitzable que es pot integrar amb sistemes existents (urnes electròniques, sistemes de votació per correu) sense substituir-los. Empra, com Belenios, ElGamal [10] *threshold* amb proves de coneixement zero [12].

ElectionGuard s'ha desplegat a un nombre creixent d'eleccions reals als Estats Units en mode pilot, sempre com a capa de verificabilitat adjunta a un sistema de votació primari. La seva filosofia és deliberadament conservadora: no pretén substituir els sistemes existents, sinó dotar-los d'una capa de verificabilitat criptogràfica.

2.3 Projectes contemporanis sobre blockchain

Helios, Belenios i ElectionGuard han assolit un grau alt de maduresa criptogràfica, però mantenen un component centralitzat clar: l'autoritat electoral és sempre una entitat coneguda i en última instància responsable de l'execució del sistema. La novetat conceptual que aporten les xarxes *blockchain* és la possibilitat de substituir aquesta autoritat per un protocol distribuït de consens, on un conjunt de nodes independents executen i verifiquen la lògica electoral sense que cap d'ells pugui modificar-la unilateralment.

2.3.1 DAVINCI Protocol (Vocdoni)

El projecte més rellevant en aquesta línia, en el moment de redactar aquesta memòria, és *DAVINCI Protocol* [9], desenvolupat per Vocdoni. Vocdoni és un equip amb experiència contrastada des de 2018 en governança descentralitzada, que ha treballat amb partits polítics, governs locals, entitats cíviques i DAO.

DAVINCI combina una capa *Layer 2* sobre Ethereum amb una xarxa descentralitzada de *sequencers*, una màquina d'estat verificada amb proves Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (**ZK-SNARKs**) [12, 13] i xifratge *threshold ElGamal* [10] per garantir privacitat del vot. Inclou també mecanismes de resistència a la coacció (*receipt-freeness*) [23] que impedeixen al votant demostrar a un tercer que ha votat, una propietat fonamental per prevenir la compra de vots i la coerció en entorns no controlats.

El projecte es publica sota llicència AGPL-3.0 i és, dels sistemes revisats, el més proper conceptualment al que es voldria arribar a llarg termini amb *Demochain*: materialitza, en producció, bona part de les propietats criptogràfiques que s'han hagut de deixar fora de l'abast del nostre MVP (vegeu secció 1.6 del capítol 1).

2.3.2 Lliçons recollides

La revisió del panorama actual permet identificar tres tendències clares en l'estat de l'art:

1. Els sistemes acadèmics consolidats (Helios, Belenios, ElectionGuard) aporten primitives criptogràfiques sòlides, però mantenen un component centralitzat operatiu.
2. Els projectes contemporanis sobre *blockchain* (DAVINCI) combinen aquestes primitives amb infraestructura distribuïda, però requereixen un disseny d'infraestructura significativament més complex (xarxes pròpies de *sequencers*, *zkVMs* dedicats).
3. Els intents d'adopció massiva amb tecnologia comercial sense rigor criptogràfic (Voatz, casos primerencs de SwissPost) han tendit a fracassar quan han estat sotmesos a auditoria externa.

El nostre projecte se situa, deliberadament, en una posició intermèdia: aprofita una infraestructura *blockchain* existent i pública (Algorand) com a capa de votació, en lloc

de construir-ne una de pròpia, i empra una segona *blockchain* pública (Ethereum) com a capa de notaria independent. La capa criptogràfica avançada queda explícitament identificada com a treball futur, sense ocultar-ho en cap moment ni reclamar propietats que el sistema actual no garanteix. La justificació detallada d'aquesta tria arquitectònica es recull al capítol de disseny.

PLANIFICACIÓ

Aquest capítol descriu com s'ha planificat i organitzat el projecte *Demochain* al llarg de la seva execució. S'hi exposa el cicle de vida adoptat, l'estructura del *backlog* amb la seva nomenclatura (èpiques, històries d'usuari, tasques i *spikes*), les definicions de Definition of Ready i Definition of Done, els *sprints* executats, els camins crítics identificats i la política de versionat del codi. L'objectiu és deixar constància d'una manera fidel de com s'ha fet la feina –no només del resultat final– per poder justificar les decisions de procés que han condicionat el producte entregat.

3.1 Cicle de vida del projecte

L'equip ha treballat amb un **model iteratiu i incremental** inspirat en Scrum [6], però adaptat a un context acadèmic amb tres restriccions importants: l'equip és reduït (quatre persones, sense rol de *product owner* extern), la disponibilitat és parcial (els membres compaginen el projecte amb la resta d'assignatures del grau) i l'horitzó temporal és tancat (l'entrega final té data fixa).

La conseqüència principal d'aquestes restriccions és que el projecte s'ha hagut de partir en dues fases conceptualment diferents:

1. **Fase de recerca i fonaments** (febrer-abril de 2026). L'equip ha hagut d'adquirir des de zero el coneixement tècnic sobre *blockchain*, Algorand, Ethereum, mètodes de votació ordinal i criptografia aplicada. Sense aquesta inversió inicial, qualsevol decisió de disseny posterior seria una conjectura.
2. **Fase de construcció iterativa** (abril-maig de 2026). Amb el coneixement adquirit, s'ha pogut planificar el *backlog* amb èpiques realistes, executar *sprints* curts i prendre decisions arquitectòniques justificades.

Aquesta divisió no és estricta: durant la fase de construcció s'han mantingut *spikes* puntuals de recerca (com la decisió de llenguatge per als *smart contracts*, recollida a l'ADR 000) sempre que ha calgut explorar un espai de solucions nou.

3.1.1 Influència de la recerca sobre la implementació

La fase de recerca ha condicionat el projecte de manera molt directa. Les conclusions documentades a [17], dins la carpeta docs/research/, han portat a les decisions següents:

- L'estudi comparatiu de plataformes *blockchain* (docs/research/ i ADR 001) va concloure que Algorand era la millor opció per a un MVP de votació, per la combinació de finalitat ràpida, cost negligible i suport per a *smart contracts* expressius. Aquesta decisió va condicionar totes les fases posteriors.
- L'estudi del mètode de Schulze (docs/research/schulze.md) i l'estudi comparatiu amb altres mètodes de votació ordinal va portar a triar Schulze pel seu compliment de propietats formals com la monotonicitat, la independència de clons i la consistència amb Condorcet [8]. L'algorisme s'implementa amb Floyd-Warshall [24] sobre la matriu de comparació per parelles.
- L'estudi dels llenguatges de *smart contracts* d'Algorand (ADR 000) va portar a adoptar *algopy* amb compilador Puya [3] en lloc de *TEAL* o *PyTEAL*, pel seu sistema de tipus fort i la integració nativa amb AlgoKit [25].
- L'exploració inicial dels mecanismes de privacitat criptogràfica (xifratge homomòrfic ElGamal, proves ZK-SNARKs Groth16, cerimònies de desxifrat amb *trustees*) va revelar una complexitat que excedia el pressupost temporal del projecte. Aquesta constatació –recollida explícitament al final del primer mes– ha portat a reduir l'abast de l'MVP a votació en clar amb verificació pública (vegeu secció 3.8).

3.2 Organització del backlog

Tot el treball s'ha gestionat amb **GitHub Projects** sobre el repositori demochain [17]. Es tracta d'una elecció pragmàtica: GitHub Projects ofereix integració directa amb les *issues*, les *pull requests* i les revisions de codi, sense introduir cap eina externa.

3.2.1 Jerarquia de *issues*

El *backlog* s'estructura en cinc nivells, amb plantilles d'*issue* i etiquetes (*labels*) que en formalitzen el tipus. Les plantilles són al directori .github/ISSUE_TEMPLATE/ del repositori i una acció de GitHub (.github/workflows/issue-title.yml) en valida el títol.

- **Èpica** (type/epic). Conjunt de treball que abasta diversos *sprints*. Cada èpica té un identificador (*p. ex.* E2, E7, E11), un objectiu, una justificació, un abast inclòs i exclòs i un criteri de tancament.
- **Història d'usuari** (type/user-story). Funcionalitat de valor visible per a un usuari, formulada en primera persona. Cada **US** té identificador format pel codi de l'èpica (*p. ex.* E2-US1) i un conjunt de criteris d'acceptació en format *Donat/Aleshores*.

- **Spike** (type/spike). Investigació limitada en el temps amb un objectiu d'aprenentatge concret i una pregunta a respondre. Els resultats d'un *spike* es materialitzen en un document de recerca a docs/research/ o en un **ADR**.
- **Tasca** (type/task). Unitat concreta de treball tècnic, idealment menor a un dia. Identificador format pel codi de la **US** mare (p. ex. E2-US1-T3). Inclou descripció, criteris d'acceptació i una llista explícita de dependències.
- **Sub-tasca** (type/subtask). Subdivisió d'una tasca quan la implementació demana una granularitat més fina.

Adicionalment, totes les *issues* es classifiquen segons la seva àrea funcional mitjançant etiquetes `area/*`: `area/smart-contract`, `area/front-end`, `area/network` (anchoring), `area/census`, `area/wallet`, `area/test`, `area/research`, `area/docs` i `area/devops`.

3.2.2 Definition of Ready (DoR)

Una tasca s'ha considerat *ready* per ser introduïda en un *sprint* quan complia simultàniament:

- Té un identificador i un títol que segueix el patró [TYPE] descripció.
- Té una descripció clara de què cal fer.
- Té criteris d'acceptació verificables en format *Donat/Quan/Aleshores*.
- Té llistades les dependències –tasques o decisions de les quals depèn– i totes elles estan resoltes o aturades a l'espera d'aquesta tasca.
- Està vinculada a una història d'usuari pare i, per transitivitat, a una èpica.
- Té assignat almenys un responsable.

3.2.3 Definition of Done (DoD)

Una tasca s'ha considerat com a *done* quan complia totes les condicions següents:

- Tots els *checkboxes* dels criteris d'acceptació estan marcats com a complerts.
- El codi viu en una branca específica i una **PR** està oberta contra dev.
- La **PR** té un títol que compleix *Conventional Commits* [7] i és validada per la *check PR checks / Conventional Commits title*.
- La **PR** passa la *check PR checks / Ruff lint*, que executa `ruff check` i `ruff format --check` sobre el codi Python del repositori. El *linting* del codi TypeScript del client (oxlint) s'executa als *hooks* locals abans del *push*.
- La **PR** té com a mínim una revisió aprovatòria d'una persona diferent del darrer *committer*.

- Els tests existents continuen passant i s'han afegit tests nous quan la tasca ho requereix.
- S'ha fet *merge* de la **PR** a main amb *squash and merge* (vegeu secció 3.7).

3.3 Inventari d'èpiques

El *backlog* consta de nou èpiques. La taula 3.1 en mostra una visió global, amb el seu estat actual al repositori i la indicació de si la seva funcionalitat ha quedat coberta per l'**MVP** entregat.

ID	Codi	Títol	Estat	Al MVP
#3	E1	Recerca i fonaments	Tancada	Sí
#5	E2	Propostes i aprovació	Tancada	Sí
#6	E3	Votació preferencial de l'elecció	Tancada	Parcial
#7	E4	Resultats i verificació del recompte	Tancada	Parcial
#340	E5	Elegibilitat de votants (ZK)	Oberta	No
#10	E6	Autenticació del votant i gestió de claus	Tancada	Parcial
#394	E7	Gestió d'organitzacions	Tancada	Sí
#413	E8	Ancoratge de resultats a Ethereum	Tancada	Sí
#402	E11	Setup del projecte i del repositori	Tancada	Sí
#520	E12	Entrega final del projecte	Tancada	Sí

Taula 3.1: Inventari d'èpiques del projecte.

Les marques *Parcial* corresponen a èpiques que es van tancar amb la part bàsica implementada (votació en clar, recompte amb Schulze, connexió de *wallet*) però que originalment incloïen funcionalitats criptogràfiques que es van moure a treball futur. La justificació es detalla a la secció 3.8.

3.4 Sprints executats

El projecte s'ha organitzat en iteracions de dues setmanes anomenades *sprints*. Els tres últims *sprints* estan formalment registrats com a *milestones* al repositori; els tres primers, executats abans de la consolidació del repositori actual el 17 d'abril de 2026, no hi consten explícitament però sí en la documentació de seguiment.

Sprint 1 (febrer-març). Recerca preliminar. Estudi d'Algorand, Ethereum, AlgoKit i mètodes de votació ordinal. Setup del primer repositori i prova de concepte sobre LocalNet d'Algorand.

Sprint 2 (març). Decisions arquitectòniques. Diagrames C4 inicials [1], primer esborrany del contracte *algopy* i selecció definitiva del mètode de votació.

Sprint 3 (març-12 abril). Primer prototip de votació. Lògica de propostes i de votació per pluralitat al contracte, mockup de les pantalles principals, configuració de *docker-compose*. Entrega intermèdia a la convocatòria del 12 d'abril.

Sprint 4 – Schulze, Results, Integration (17–30 abril). Implementació del mòdul de Schulze en TypeScript sobre la matriu desxifrada (issue #132 i #133), pàgina de resultats amb guanyador i rànquing (#136), matrius de comparació per parelles col·lapsables (#139), targeta «El meu vot» al detall de la proposta (#144) i panell de verificació del vot (#145). Aquest *sprint* consolida la presència del repositori actual i incorpora la majoria del codi del client.

Sprint 5 – Testing, Refinement i Robustesa (1–14 maig). Cobertura de proves unitàries del domini de propostes (#458), tests dels *mappers* de *queries* Algorand (#457), tests d'integració del *governance client* contra LocalNet (#456), correcció del *bug* d'accés a transaccions via Lora (#463) i refinament de la pantalla de votació amb edició d'hora/minuts amb el teclat (#470).

Sprint 6 – Documentació, Demo i Entrega (15–25 maig). Redacció de la memòria, preparació de la presentació oral, retrospectiva i empaquetament final amb la versió v1.0.0.

Els *sprints* 4 a 6 consten com a *milestones* actius al repositori, amb el detall complet de les *issues* associades a cada un.

3.5 Camins crítics

L'arquitectura del sistema imposa unes dependències estrictes entre components que han condicionat l'ordre de la implementació. Identificar-les amb antelació ha estat clau per evitar bloquejos i, en alguns casos, per prioritzar feina aparentment menor que altres tasques necessitaven.

3.5.1 Camí crític 1: Recerca → contracte Algorand

Cap línia de codi del contracte podia escriure's abans d'entendre el model d'execució de l'Algorand Virtual Machine (AVM), l'estructura de *boxes* d'emmagatzematge, el sistema de tipus d'*algopy* i les convencions ARC-4 [26]. Aquesta dependència és la justificació de la *fase de recerca* prèvia descrita a la secció 3.1. La duració d'aquesta fase ha tingut un cost d'oportunitat alt –un mes sense codi entregat– però sense aquest coneixement el contracte hauria estat ple d'errors de model.

3.5.2 Camí crític 2: Contracte Algorand → client web

El client només pot interactuar amb el contracte un cop es genera el fitxer de descripció ARC-56 [27] a partir del contracte compilat. Aquest fitxer, en el repositori `client/src/algorand/Demochain.arc56.json`, conté l'Application Binary Interface (ABI) dels mètodes públics i és consumit pel client TypeScript per construir i signar transaccions amb *algosdk*.

La conseqüència pràctica és que el client no podia avançar més enllà del *mockup* estàtic fins que el contracte tenia un primer conjunt de mètodes estables. La planificació ha donat prioritat, dins de cada *sprint*, a tancar primer els mètodes del contracte

de l'àrea funcional corresponent abans que les pantalles del client que els consumeixen. Quan, ocasionalment, el contracte introduïa canvis en una **PR**, el client havia de regenerar l'**ABI** i adaptar els *mappers*.

3.5.3 Camí crític 3: Contracte Algorand → ancoratge

El servei d'ancoratge llegeix directament les *boxes* del contracte d'Algorand per construir l'estat canònic de l'elecció. Concretament, llegeix les *boxes* amb prefix `pr_` (propostes) i `eb_` (paperetes d'elecció), tal com s'implementen al contracte. Si l'esquema d'aquestes *boxes* canvia, el servei d'ancoratge no pot interpretar les dades.

L'ancoratge, per tant, no es podia desenvolupar de manera realista fins que el contracte tenia un esquema d'emmagatzematge estabilitzat. La planificació ha tractat aquesta dependència posant el codi de l'ancoratge en una èpica pròpia (#413) executada un cop el contracte ja tenia versió estable, i ha inclòs a la **US E7-US1** (#414) la tasca explícita d'*adaptar* la capa d'ancoratge importada d'un repositori extern al nou contracte.

3.5.4 Camí crític 4: Infraestructura local de desenvolupament

Tots tres components anteriors necessiten un entorn local funcional. Aixecar un node Algorand LocalNet amb *indexer*, *conduit* i base de dades PostgreSQL no és una tasca trivial. El fitxer `docker-compose.yml` del repositori integra tots els serveis i automatitza la compilació i desplegament del contracte i el seu registre a `client/.env.local`.

Aquesta infraestructura ha estat un camí crític silenciós: cada vegada que un membre de l'equip incorporava una nova versió del codi havia de poder aixecar tota la pila en pocs minuts. La inversió en automatització d'aquest desplegament ha estat un dels factors que ha permès una velocitat sostinguda durant la fase de construcció.

3.6 Paquets de treball

Els paquets de treball es deriven de l'estructura del repositori i de les àrees funcionals etiquetades a les *issues*. Cada paquet té un responsable principal –el mateix membre de l'equip que ha liderat l'àrea durant tot el projecte– però totes les **PRs** han passat per revisió creuada.

Paquet	Ubicació al repo	Responsable principal
Smart contract Algorand	voting-contract/	Marc Link Cladera
Client web	client/	Jordi Vanyó Mestre
Ancoratge a Ethereum	network/anchoring/	Dylan Luigi Canning
Smart contract notari	network/ethereum/	Dylan Luigi Canning
Infraestructura local	docker-compose.yml, conduit.yml, Makefile	Jordi Vanyó / Dylan Luigi
Documentació i backlog	docs/, GitHub Projects	Antonio Contestí Coll

Taula 3.2: Distribució de paquets de treball.

3.7 Política de versionat del codi

S'ha adoptat una política de *feature branches* + **PR** + *squash and merge*, complementada amb *Conventional Commits* [7] i *git hooks* locals. Les regles concretes estan documentades a CONTRIBUTING.md del repositori i es resumeixen a continuació.

3.7.1 Estructura de branques

- **main**. Branca principal. Conté codi sempre desplegable. Cap *push* directe està permès; tots els canvis hi arriben via **PR**.
- **dev**. Branca d'integració per a treball en curs i artefactes de documentació que encara no estan a punt per ser publicats a **main** (com els capítols de la memòria).
- **feat/<num>**, **fix/<num>**, **docs/<num>**, etc. Branques de característica, una per cada *issue*. El número correspon a l'identificador GitHub de l'*issue*. *P. ex.* la branca **feat/392** resol l'*issue* #392.

3.7.2 Convencions de commit

Els missatges de *commit* segueixen l'estàndard *Conventional Commits* [7] amb el format:

```
<type>(<scope>): descripció
```

Els tipus permesos són: **feat**, **fix**, **docs**, **style**, **refactor**, **test**, **chore**, **ci**, **build**, **perf** i **revert**. El compliment es valida en dos punts:

- *Localment*, mitjançant els *hooks* de Git (**.githooks/commit-msg**, **.githooks/pre-commit**, **.githooks/pre-push**) que es registren executant **make hooks**.
- *Remotament*, mitjançant l'acció de GitHub **.github/workflows/pr-checks.yml**, que comprova el títol de cada **PR**.

3.7.3 Procés de *Pull Request*

1. Es crea una branca `feat/<num>` a partir de `main`.
2. S'hi fan els *commits* que calguin.
3. En fer *push*, els *hooks* locals executen Ruff (codi Python) i Biome (codi TypeScript) per assegurar el format.
4. S'obre una **PR** contra `main`. La plantilla de **PR** enllaça automàticament l'*issue* corresponent mitjançant la fórmula Closes #<num>.
5. GitHub Actions executa les *checks* de format i nomenclatura.
6. Almenys un altre membre de l'equip revisa la **PR** i n'aprova el contingut.
7. Es fa *squash and merge* per mantenir un historial lineal i una correspondència 1:1 entre *commits* a `main` i *issues* tancades.

Una conseqüència d'aquesta política, observada durant el projecte, és que ningú pot fer *merge* de la seva pròpia **PR**: la regla de *branch protection* de GitHub exigeix una aprovació d'algú diferent de l'últim *pusher*. Aquesta restricció ha estat un mecanisme natural de revisió creuada i ha contribuït a la qualitat del codi entregat.

3.8 Reducció d'abast: de l'MVP criptogràfic a l'MVP en clar

Una de les decisions de planificació més rellevants del projecte ha estat la **reducció d'abast** acordada al final del primer mes de desenvolupament.

L'estat objectiu inicial, recollit a les èpiques E3, E4, E5 i E6 del *backlog*, era un sistema amb:

- Vots xifrats amb ElGamal exponencial homomòrfic.
- Proves **ZK-SNARKs** Groth16 per acreditar l'elegibilitat del votant sense revelar-ne la identitat.
- Cerimònia de generació distribuïda de claus (DKG) i de desxifrat amb *trustees* i proves Chaum-Pedersen.
- Integració amb un **IdP** extern (Keycloak) per a l'autenticació prèvia del votant.

Aquesta arquitectura estava documentada com a estat objectiu i les èpiques corresponents s'havien redactat amb el detall necessari (vegeu les descripcions de #10, #340, #7 i #6 al repositori).

Després d'una sessió d'estimació realista cap al final del Sprint 3, l'equip va concloure que la criptografia avançada **no era assolible** dins l'horitzó del projecte sense comprometre la resta de l'entrega. La decisió adoptada va ser:

1. Reduir l'**MVP** a votació **en clar** amb verificació pública via consulta directa a la cadena Algorand i ancoratge *K-of-N* a Ethereum.

2. Tancar les èpiques E3, E4 i E6 amb el subconjunt de funcionalitats efectivament implementades –votació preferencial, recompte Schulze, connexió de *wallet*– i deixar la resta d'**USs** obertes com a treball futur explícit.
3. Mantenir oberta l'èpica E5 sencera, ja que tota la seva funcionalitat depèn de la capa criptogràfica.

Aquesta decisió ha estat un dels re-aprenentatges més importants del projecte. Vista en retrospectiva, és coherent amb les recomanacions de la literatura sobre gestió de projectes: davant un objectiu ambiciós i un horitzó tancat, és preferible entregar un subconjunt funcional i sòlid que un superconjunt incomplet o inestable. La criptografia avançada queda identificada explícitament al capítol de conclusions com a línia de treball futura amb el detall necessari perquè un altre equip pugui continuar-hi.

ANÀLISI DEL SISTEMA

En aquest capítol es detalla l'anàlisi de requisits, restriccions i decisions de disseny que han guiat el desenvolupament del projecte **Demochain**. L'objectiu és oferir una visió comprensible de la infraestructura tant per a auditors externs com per a perfils tècnics que no hagin participat directament en el procés de programació. A continuació, es descriuen els actors del sistema, la recerca teòrica en criptografia i xarxes de consens en contraposició a la implementació real de l'**MVP**, les especificacions funcionals i no funcionals, les restriccions imposades per l'entorn tecnològic i legal, i les decisions de disseny global de la plataforma.

4.1 Identificació de Parts Interessades (*Stakeholders*)

Per tal d'entendre l'abast i els objectius del sistema de votació, és imprescindible identificar quins són els actors clau que intervenen en el procés electoral i quin benefici directe o indirecte obtenen de la seva implantació. El sistema s'ha concebut des d'una perspectiva modular on es contraposen els interessos dels administradors, electors, custodis i verificadors per aconseguir un equilibri de seguretat.

1. Votants / electors que formen part del cens:

- **Descripció:** Membres actius de la institució que convoca la votació, com ara els estudiants i professors en l'àmbit universitari.
- **Benefici:** Disposar d'una plataforma digital accessible que els permet exercir el sufragi de manera remota, evitant desplaçaments físics i esperes. El sistema els assegura la total privacitat de les seves preferències electorals i els proporciona mecanismes pràctics contra la coacció, com és el dret a modificar el vot emès mentre l'elecció estigui oberta. A més, en emprar una xarxa permissionada a cost zero per transacció, no han d'adquirir criptomonedes ni pagar taxes per votar. Finalment, tenen la capacitat de verificar

individualment que la seva papereta s'ha registrat de forma íntegra a la xarxa d'Algorand mitjançant la interfície de consulta.

2. Candidats i les diferents opcions de votació:

- **Descripció:** Persones físiques que opten a un càrrec, llistes de partits o alternatives en un referèndum o consulta normativa.
- **Benefici:** Garantia que les regles del joc són immutables des de l'inici fins al final de la votació, ja que les dades s'emmagatzemen a una cadena de blocs (*blockchain*), eliminant el risc de manipulació física d'urnes o pèrdua de paperetes. Així mateix, es beneficien d'un recompte automàtic i immediat un cop tancat el procés electoral, reduint el cost de campanya associat a la mobilització de delegats o interventors a les meses electorals tradicionals.

3. Organitzadors (Comitè Electoral i Administradors):

- **Descripció:** Entitat o comitè electoral responsable de la gestió de les consultes, la qual cosa inclou crear l'organització, gestionar el cens d'adreces autoritzades, definir les opcions de les propostes i coordinar les dates d'inici i de tancament.
- **Benefici:** Reducció dràstica en els costos de gestió que acompanyen qualsevol sufragi tradicional (paperetes, urnes físiques, lloguer d'espais, dietes del personal de mesa, serveis de seguretat). L'automatització d'aquest procés elimina l'error humà i la necessitat d'escrutinis manuals llargs, oferint resultats transparents que reforcen la reputació democràtica de l'entitat.

4. Custodis (*Trustees*):

- **Descripció:** Actors independents amb rols contraposats dins de la comunitat (com ara membres de la junta de govern, representants dels estudiants o auditors externs) que reben un fragment de la clau de desxifratge del recompte global.
- **Benefici:** Control conjunt de l'escrutini final. Cap entitat per si sola pot desxifrar els vots individuals ni calcular els resultats abans d'hora. Gràcies a un esquema de llindar, poden participar de manera asíncrona per reconstruir el resultat, obtenint proves matemàtiques que validen la seva honestat i actuació davant els electors sense por de ser acusats de manipulació.

5. Auditors i observadors independents:

- **Descripció:** Entitats externes, periodistes o observadors internacionals que vetllen per la legalitat i legitimitat del sufragi.
- **Benefici:** Accés directe i en temps real a tot l'historial de transaccions d'Algorand i al segell d'Ethereum Sepolia sense necessitar permisos explícits ni demanar autorització als administradors. Això els permet auditar de cap a fi la integritat del procés reexecutant l'algorisme de Schulze directament sobre les caixes de dades públicament disponibles.

6. Entitats Democràtiques i DAO (Adoptants B2B):

- **Descripció:** Sindicats, associacions, cooperatives o organitzacions descentralitzades que cerquen una eina de govern electrònic segura.
- **Benefici:** Reutilització d'aquesta tecnologia de codi obert per a organitzar els seus processos de presa de decisions interns a cost zero de desenvolupament, adaptant el disseny modular a les seves estructures de govern.

7. Equip de desenvolupament i comunitat acadèmica:

- **Descripció:** Investigadors i programadors que dissenyen la solució i avaluen la viabilitat de noves primitives de xifratge o analitzen el comportament de l'algorisme de Schulze *on-chain*.
- **Benefici:** Disposen d'un cas pràctic i d'un repositori de codi modular escrit en Python i Solidity. Per a ells, aquest projecte ha constituït un gran entorn de formació pràctica, atès que no tenien pràcticament cap coneixement previ sobre tecnologia *blockchain* o criptografia abans de començar les catorze setmanes de desenvolupament.

4.2 Recerca Criptogràfica i de Consens per a un Disseny Robust

El disseny de qualsevol sistema de votació electrònica s'enfronta a un dilema fonamental: com garantir que només els electors autoritzats votin i que cadascú ho faci una sola vegada (integritat i autenticació), alhora que s'assegura que ningú, ni tan sols l'administrador del sistema, pugui vincular un vot amb la identitat de qui l'ha emès (privacitat), tot permetent que qualsevol ciutadà comprovi de forma independent que el recompte és correcte (auditabilitat pública). En els sistemes de votació tradicionals, aquest equilibri es confia a mesures físiques (urnes segellades, sobres opacs i meses electorals multipartidistes). Traslladar aquestes garanties al món digital sense dependre d'una autoritat central que podria ser corrupta requereix l'ús de primitives criptogràfiques avançades.

Durant la fase inicial del projecte, vàrem dur a terme una recerca profunda sobre l'estat de l'art en protocols de votació electrònica segura. En aquesta recerca, el projecte de codi obert **DaVinci** (disponible a <https://davinci.vote/>) ens va servir de guia i referència per a estructurar i entendre com coordinar diferents tecnologies de consens i privacitat. A partir d'aquest referent, vàrem identificar els conceptes criptogràfics clau que haurien de formar part d'una arquitectura teòrica robusta:

- **Proveïdor d'Identitat (IdP - Identity Provider):** Representa l'entitat de confiança que gestiona el
- cens real dels electors (per exemple, el sistema d'autenticació central d'una universitat). En un disseny plenament descentralitzat, l'**IdP** signa una credencial criptogràfica per a cada usuari registrat que demostra el seu dret al sufragi, sense vincular-lo directament a una adreça de bloc pública (*wallet*).
- **Proves de Coneixement Zero (ZK-SNARK) [12, 13]:** Són proves criptogràfiques que permeten a un elector provar davant el contracte intel·ligent que té una credencial vàlida signada per l'**IdP** (i, per tant, forma part del cens) sense revelar la

seva identitat real ni revelar la signatura de l'IdP. A nivell conceptual, el proveïdor executa un circuit amb entrades privades (les seves credencials) i paràmetres públics per a generar una prova compacta que qualsevol node pot verificar de forma instantània. Per evitar el doble vot sense comprometre l'anonimat, es va considerar l'ús d'un *nullifier* (anul·lador), un identificador determinista derivat de la clau privada del votant i de la proposta. El contracte intel·ligent rebutja transaccions amb anul·ladors ja utilitzats. No obstant això, el tema del *nullifier* no el varem acabar de concretar bé en la nostra recerca; dissenyar una estructura robusta i segura per a aquesta primitiva és extremadament complex i no varem trobar un disseny definitiu que s'adaptés de forma senzilla a la nostra infraestructura.

- **Xifratge Homomòrfic:** És un tipus de xifratge que permet efectuar operacions matemàtiques directament sobre dades xifrades (*ciphertexts*) de manera que, en desxifrar el resultat, s'obté la suma o l'operació en clar de les dades de partida. En el context electoral, les preferències de cada votant es xifren en el dispositiu client abans de ser enviades. La xarxa de blocs agrega de forma multiplicativa totes les paperetes xifrades directament *on-chain*, generant un únic resultat xifrat de toda la votació. Això assegura que les paperetes individuals mai es desxifren, protegint de forma absoluta el secret del sufragi.
- **Custodis o Trustees:** Són entitats de confiança encarregades de la custòdia de la clau de desxifratge global. Per evitar un únic punt de fallada o que l'administrador del sistema pugui desxifrar els vots abans de temps, s'empra un esquema de llindar. Mitjançant un protocol de generació de claus distribuïdes (DKG) [14], la clau privada de desxifratge es divideix en fragments (*shares*) repartits entre diversos custodis. Per desxifrar el recompte homomòrfic acumulat un cop tancat el procés, es requereix que un nombre mínim o llindar de custodis (com a manco la majoria absoluta) aportin la seva *share* parcial.

4.2.1 Desviació del Disseny i Limitacions del Prototip MVP

Malgrat haver definit aquest disseny teòric, l'equip va haver de fer front a una important limitació pràctica: la manca de coneixements previs profunds sobre criptografia avançada i la dificultat extrema que suposa implementar i auditar tot aquest flux de forma totalment segura. Dissenyar circuits de coneixement zero, coordinar la cerimònia de DKG entre custodis i desplegar operacions homomòrfiques a la màquina virtual de forma eficient no és gens trivial i requereix mesos de desenvolupament especialitzat.

Per mor d'això, es va decidir orientar el desenvolupament del prototip MVP cap a una arquitectura simplificada que fos viable dins els terminis del projecte. Aquesta desviació de disseny implica els següents canvis:

1. **Vots en clar:** Les paperetes de preferències ordinals es registren i s'emmagatzemen directament en clar a les *boxes* d'Algorand, sense xifratge homomòrfic. Qualsevol pot auditar els vots reals consultant la *blockchain*.
2. **Cens identificat:** La gestió d'electors s'efectua a través d'una llista blanca d'adreces de carteres físiques (*wallets*) gestionada per l'organitzador, prescindint de

les proves **ZK-SNARK** i de l'anul·lador. Els usuaris s'identifiquen i signen els seus vots amb la seva pròpia clau pública d'Algorand.

3. **Càlcul fora de la cadena:** Ateses les limitacions de *bytecode* d'Algorand, el recompte i l'algorisme de Schulze es realitzen al navegador (*client-side*) obtenint els vots directament de les *boxes* del contracte. La cerimònia de desxifratge de custodis s'ha posposat completament com a treball futur.

Aquesta arquitectura de l'**MVP** permet disposar d'un sistema funcional que garanteix la immutabilitat física dels vots emesos a Algorand i l'ancoratge del resultat final a Ethereum Sepolia, al preu d'ajornar el secret criptogràfic del vot i l'anònim absolut de l'elector per a futures iteracions de producció de la plataforma.

4.3 Especificació Funcional

El comportament del sistema es defineix a partir de les interaccions entre els diferents rols de la plataforma, detallant un flux de treball continu que guia tot el procés.

4.3.1 Cicle de Vida del Sufragi i Casos d'Ús

El cicle de vida d'una consulta electoral a la plataforma es defineix a partir de les interaccions dels diferents actors i del flux lògic del sistema, el qual es pot esquematitzar en els següents vuit casos d'ús principals:

- **UC-1: Crear Organització:** Qualsevol usuari registrat que connecti la seva cartera de criptomonedes (*wallet*) pot registrar una nova organització a la xarxa d'Algorand proporcionant un nom i una descripció, esdevenint automàticament l'organitzador i propietari del registre.
- **UC-2: Gestionar Cens:** L'organitzador administra els electors autoritzats afegint o eliminant adreces de *wallet* de la llista blanca de l'entitat. La interfície web gestiona aquesta càrrega en lots de com a màxim set adreces per transacció per evitar un consum excessiu de memòria de la màquina virtual.
- **UC-3: Crear Proposta:** Qualsevol membre registrat al cens de l'organització pot iniciar una consulta detallant el títol, la descripció, la llista de candidats ordinals (que ha de ser com a mínim de dos), la data d'inici i la data de finalització del sufragi.
- **UC-4: Emetre Vot d'Aprovació:** Durant una fase preliminar, els electors aproven o rebutgen la viabilitat de la proposta. Aquest vot d'aprovació és immutable i irrevocable, i només si s'assoleix un quòrum de dues tercers parts (2/3) de vots favorables abans de la data d'inici, la proposta passa a la fase electoral.
- **UC-5: Emetre/Modificar Vot Preferencial:** Un cop s'arriba a la data d'inici i durant la finestra activa, els votants ordenen els candidats per preferències mitjançant una interfície d'arrossegat i deixar anar (*drag & drop*). Es permet enviar noves paperetes per sobreescriure el registre anterior a les *boxes* del contracte i així evitar coaccions.

- **UC-6: Recompte i Escrutini:** En tancar-se la votació, es realitza el recompte. Conforme al disseny simplificat de l'**MVP**, les paperetes es llegeixen directament de les *boxes* d'Algorand en clar i l'escrutini es resol mitjançant l'aplicació client que executa l'algorisme de Schulze.
- **UC-7: Ancoratge postelectoral:** Un servei independent recopila les paperetes finals d'Algorand, les ordena lexicogràficament per adreça, genera un JavaScript Object Notation (**JSON**) canònic sense espais, calcula el *hash* Secure Hash Algorithm (**SHA**)-256 de 32 bytes i l'envia al contracte notari d'Ethereum Sepolia per certificar el recompte.
- **UC-8: Auditabilitat i Verificació Pública:** Qualsevol ciutadà o observador extern pot consultar de manera individual la seva papereta a Algorand i comprovar si el *hash* del resultat global coincideix amb el *hash* d'ancoratge validat pel consens de les universitats a Ethereum Sepolia.

A continuació, es mostra el diagrama de flux que representa gràficament la interacció dels diferents rols amb cada cas d'ús:

4.3.2 Metodologia del Backlog i Històries d'Usuari

El disseny de la lògica funcional i dels rols descrits s'ha planificat mitjançant metodologies àgils utilitzant GitHub CLI, connectant de manera directa amb el repositori de codi. La jerarquia de treball s'ha estructurat en èpiques, històries d'usuari, *spikes* de recerca, tasques específiques i gestió d'errors o *bugs*, cosa que ha facilitat la distribució de feines durant les catorze setmanes de durada del projecte.

A continuació es detallen les històries d'usuari clau que s'han emprat per a guiar el desenvolupament:

- **US-1 (Gestió d'organitzacions i cens):** Com a organitzador, vull poder crear una organització i gestionar el cens d'electors mitjançant càrregues en lot o fitxers **CSV**, per tal de preparar les votacions de manera àgil i sense errors.
- **US-2 (Autenticació i connexió):** Com a elector, vull poder vincular la meua cartera Pera Wallet de forma transparent des del navegador, per tal d'autenticar-me i signar les transaccions de votació de forma nativa i senzilla.
- **US-3 (Interfície de vot preferencial):** Com a votant, vull disposar d'una interfície d'arrossegar i deixar anar (*drag & drop*) per tal d'ordenar els candidats segons les meves preferències, fent que el meu vot sigui més expressiu que en un sistema majoritari simple.
- **US-4 (Protecció contra la coacció):** Com a elector, vull poder efectuar nous enviaments que sobreescriguin totalment la meua papereta anterior a les *boxes* del contracte de manera opaca per a un observador extern, per tal de protegir-me contra la coacció o la compravenda de vots.
- **US-5 (Seguretat de la custòdia):** Com a trustee o custodi, vull poder registrar-me en el sistema per a custodiar el procés, entenent que en futures versions de la plataforma podré aportar proves de coneixement zero i shares de desxifratge.

- **US-6 (Auditabilitat de l'ancoratge):** Com a auditor acadèmic, vull poder verificar directament a Ethereum Sepolia si un resultat de votació té la certificació de consens del consorci d'universitats, per tal de validar formalment la legitimitat del sufragi.

Els *spikes* de recerca inicials es van utilitzar per estudiar la viabilitat de dissenyar un cens totalment anònim *on-chain* i coordinar de forma eficient la cerimònia de desxifratge dels custodis abans de desglossar les tasques de desenvolupament i programar els diferents components de la plataforma.

4.3.3 Requisits de Producte

Els requisits de producte del sistema asseguruen la consistència jurídica i tècnica del sufragi:

- **RP-1 (Marge temporal de creació):** La data d'inici d'una proposta ha d'estar, com a manco, a tres dies de distància de la seva creació en el contracte intel·ligent.
- **RP-2 (Finestra electoral mínima):** La durada activa del procés de votació preferencial ha de tenir una durada d'almenys vint-i-quatre hores per garantir la participació.
- **RP-3 (Cens de dades atòmic):** El registre del cens ha de validar que no s'introdueixen adreces duplicades ni adreces nul·les (adreça zero) a la llista blanca de l'organització.
- **RP-4 (Consistència de la papereta):** El contracte intel·ligent d'Algorand ha de rebutjar de forma estricta paperetes de preferències que no continguin exactament el mateix nombre d'elements que candidats té la proposta.
- **RP-5 (Estat terminal de l'urna):** Un cop calculat el guanyador i tancat l'escrutini de la proposta, l'elecció canvia al seu estat immutable terminal, tancant de forma completa qualsevol escriptura posterior.

4.4 Especificació No Funcional i Paràmetres de Qualitat

Els requisits no funcionals del sistema de votació estan orientats a garantir la seguretat, la disponibilitat i l'auditabilitat de les consultes, alhora que busquen optimitzar l'eficiència computacional.

4.4.1 Requisits No Funcionals

- **RNF-1 (Seguretat i secret del sufragi):** Els vots individuals s'han de protegir per a mantenir-ne l'anonimat i el secret de les paperetes a la xarxa d'Algorand mitjançant adreces de cartera pseudònimes.
- **RNF-2 (Immutabilitat):** Qualsevol transacció confirmada (vot emès, canvi de cens, creació d'organització) esdevé completament inalterable gràcies al protocol de consens immediat d'Algorand, implicant qualsevol modificació posterior.

- **RNF-3 (Disponibilitat):** L'urna digital ha de ser accessible de forma permanent i ininterrompuda (24/7) durant tota la finestra de votació, independentment d'eventuals caigudes dels servidors de la institució convocant.
- **RNF-4 (Auditabilitat de cap a fi):** Es fomenta l'auditabilitat *end-to-end*, de manera que qualsevol observador o elector pot descarregar i verificar de forma asíncrona que la seva papereta s'ha comptabilitzat correctament i que el *hash* coincideix amb el segell d'Ethereum Sepolia.
- **RNF-5 (Eficiència Computacional i cost):** El disseny de l'execució s'optimitza per evitar un consum elevat d'operacions *on-chain* a Algorand i redueix l'escriptura a Ethereum a una única transacció global per elecció finalitzada per evitar uns costos de gas inacceptables.

4.4.2 Paràmetres de Qualitat

Per mesurar la qualitat i la resiliència del sistema de seguretat distribuïda s'apliquen tres paràmetres clau:

- **Llindar de Seguretat de Custòdia (K -de- N):** Per poder desxifrar la matriu agregada final de preferències sota l'esquema de seguretat de llindar del disseny teòric, cal la participació de com a mínim la majoria absoluta dels custodis autoritzats, la qual cosa és causada per la relació:

$$K = \left\lceil \frac{N+1}{2} \right\rceil$$

Aquesta equació assegura que fa falta el consens de la majoria absoluta dels custodis per desbloquejar els resultats de la votació, evitant que una minoria o un únic administrador pugui comprometre el procés de forma unilateral.

- **Consens d'Ancoratge a Ethereum:** El contracte notari de Sepolia exigeix que el *hash* enviat del resultat final rebí el consens de dues terceres parts (2/3) dels nodes universitaris autoritzats abans de marcar l'elecció com a oficialment ancorada. El llindar K_{notary} es congela mitjançant la fórmula:

$$K_{notary} = \left\lceil \frac{2N+2}{3} \right\rceil$$

- **Temps de Confirmació:** Aprofitant les característiques tècniques de la xarxa permissionada d'Algorand, qualsevol transacció de vot o d'aprovació s'ha de veure confirmada en un temps màxim de quatre segons, evitant retards en l'escriptura de dades i sense el risc de patir reorganitzacions probabilístiques de blocs.

4.5 Restriccions

El disseny arquitectònic de Demochain ha hagut d'adaptar-se a una sèrie de limitacions imposades pel *software* de les xarxes utilitzades, els reglaments legals de protecció de dades i el context d'aprenentatge de l'equip.

4.5.1 Restriccions Tecnològiques

El disseny de la lògica *on-chain* s'ha enfrontat a límits físics estrictes imposats per la Màquina Virtual d'Algorand (AVM) i el protocol d'Ethereum. Inicialment, l'AVM limita la mida màxima del *bytecode* dels contractes intel·ligents a només 8KB de memòria. Aquesta severa restricció de mida impedeix codificar directament *on-chain* algorismes complexos de cerca de camins ponderats en grafs de preferències, com ara l'algorisme de Floyd-Warshall que fa servir el mètode Schulze, el qual té una complexitat computacional de triple bucle de tipus $O(n^3)$. Per mor d'aquest límit de *bytecode*, el sistema s'ha vist obligat a delegar la resolució del guanyador a l'aplicació client (fora de la cadena de blocs), de manera que el navegador llegeix les paperetes, calcula el mètode Schulze i les *boxes* del contracte intel·ligent es limiten a persistir les dades de forma estructurada.

En segon lloc, l'emmagatzematge en *boxes* d'Algorand implica un cost anomenat Saldo Mínim Requerit (MBR) per byte per evitar la saturació del registre d'estat de la xarxa. Aquest cost en microAlgos que el mateix contracte ha de mantenir com a fons de reserva es calcula mitjançant la fórmula:

$$\text{MBR (microAlgos)} = 2500 + 400 \times (\text{len}(\text{clau}) + \text{mida_box})$$

Com que cada organització, membre del cens, proposta i vot emès genera noves caixes de dades al contracte, cal gestionar l'MBR com a paràmetre crític per no deixar el compte del contracte sense saldos suficients. A més, hi ha una restricció que impedeix que una transacció faci referència a més de vuit caixes simultàniament, la qual cosa condiciona la manera en què el client ha d'enviar les transaccions a la xarxa. Finalment, el cost econòmic de fer crides a Ethereum L1 ens fa elegir emprar la xarxa de proves Sepolia, i el protocol només envia un únic *hash* de 32 bytes (`bytes32`) per cada elecció un cop s'ha completat tot l'escrutini.

Sense segones capes o agregació de transaccions (*rollups*), Algorand té un límit físic d'unes 6.000 transaccions per segon. És una xifra molt bona per a entitats mitjanes com universitats o empreses, però insuficient per a gestionar eleccions estatals simultànies on participen milions de ciutadans.

4.5.2 Restriccions Legals i de Privacitat

En l'àmbit legal, el disseny s'enfronta directament a la normativa europea de protecció de dades (RGPD) [28]. Atès que la cadena de blocs és històricament immutable per disseny, és tècnicament impossible esborrar les dades un cop confirmades a la xarxa, fet que s'oposa al dret a l'oblit de la normativa legal. Per resoldre aquesta restricció, el sistema prohibeix de manera absoluta guardar *on-chain* qualsevol tipus de dada personal identificable, com ara noms, correus o documents nacionals d'identitat. La identificació física del votant es realitza exclusivament al Proveïdor d'Identitat (IdP) centralitzat de la institució convocant.

D'altra banda, el secret del vot s'oposa de forma frontal a la total transparència d'una *blockchain* on l'historial de transaccions de qualsevol adreça és pública per a tothom. Com s'ha explicat a la secció de recerca criptogràfica, la privacitat teòrica mitjançant *ZK-SNARK* i xifratge homomòrfic ha hagut de ser simplificada en l'MVP actual, on l'anonimat dels electors es recolza únicament en l'ús d'adreces de cartera genèriques i pseudònims, esdevenint una limitació de seguretat assumida pel prototip.

4.5.3 Restriccions d'Aprenentatge i Terminis del Projecte

La principal restricció que ha condicionat el disseny i el calendari del projecte ha estat la manca total d'experiència prèvia en tecnologia *blockchain* i criptografia per part de l'equip de desenvolupament abans de començar les catorze setmanes de termini establertes:

- Es va haver de dur a terme una intensa recerca inicial (*spikes* de recerca) des de zero per copsar conceptes de distribució de dades, signatura de transaccions, gestió de carteres i funcionament de les màquines virtuals (AVM i Ethereum Virtual Machine (EVM)).
- Aquest escenari d'aprenentatge intensiu va obligar a prendre decisions realistes. Com s'ha detallat a la Secció 4.2, el desenvolupament segur de circuits ZK-SNARK, el model d'anul·lador i el protocol de desxifratge homomòrfic amb DKG de custodis requereix competències altament especialitzades i un temps de programació inexistent en el calendari. Per consegüent, per assegurar el lliurament d'un prototip funcional de cap a fi (MVP), es va decidir ajornar la implementació criptogràfica com a feina futura, centrant els esforços a establitzar els fluxos de cens, votació preferencial i ancoratge inter-blockchain.

4.6 Decisions de Disseny Global

A l'hora de conceptualitzar Demochain, s'han pres decisions clau sobre l'arquitectura global que prioritzen la transparència i la resistència a la manipulació per sobre de models centralitzats convencionals. La decisió d'utilitzar Algorand en lloc d'Ethereum o altres plataformes de primera capa es va basar en criteris econòmics i de facilitat de desenvolupament:

1. **Comissions extremadament econòmiques:** Les comissions d'Algorand són de només 0,001 ALGO per transacció, la qual cosa feia factible el projecte fins i tot amb xarxes públiques.
2. **Eliminació de barreres d'entrada gràcies a Python:** Poder programar els contractes intel·ligents directament en Python natiu mitjançant el compilador Puyapy (algopy) va permetre a un equip que no coneixia la *blockchain* començar a desenvolupar des del primer dia, estalviant-se l'aprenentatge de llenguatges complexos com Solidity o Rust.

Durant el cicle de desenvolupament de Demochain, el model de xarxa ha patit una evolució molt interessant arran de la identificació de noves limitacions. Inicialment, es va plantejar un model on els mateixos telèfons mòbils dels electors actuessin com a nodes validadors de la xarxa. Es pensava que mitjançant el mecanisme de VRF (Funció Aleatòria Verificable) [29] d'Algorand es podria obligar cada votant a mantenir l'aplicació activa almenys dos minuts per poder votar, obligant-los a participar del consens de forma efímera. L'estudi tècnic posterior va demostrar que això era completament inviable a la pràctica per mor de la inestabilitat de les xarxes mòbils, el desgast de

bateria dels terminals i els exigents requisits de bloqueig de fons (*staking*) que té el protocol de consens de la xarxa.

Davant d'aquesta impossibilitat, es va dissenyar una infraestructura de xarxa privada permissionada. Els nodes validadors passaven a ser gestionats per un consorci d'entitats de confiança recíproca amb interessos contraposats (per exemple, les diferents universitats de la comunitat autònoma). Això permetia un control de participació i fixar les comissions de la xarxa a zero de manera nativa ($\text{min_fee} = 0$), eliminant la necessitat de finançar les carteres dels votants.

Més endavant, es va intentar adaptar el *software* per a funcionar directament sobre la xarxa pública d'Algorand (testnet/mainnet). Ens vàrem adonar que la xarxa pública oferia grans avantatges de *liveness*, seguretat contra atacs de denegació de consens locals i una transparència absoluta per a auditors mundials. No obstant això, es va topa amb un obstacle: com que les comissions són obligatòries a la xarxa pública (malgrat que siguin negligibles), l'única forma d'evitar que el votant hagués d'adquirir criptomonedes era crear un servei centralitzat de patrocini (*sponsor/relayer*) que cobrés el cost de les transaccions a través d'un grup atòmic. Això trencava la naturalesa descentralitzada de la votació (el patrocinador centralitzat podia censurar vots lliurement no signant-ne el patrocini). A més a més, l'equip ja tenia enllestit i testejat tot el mòdul d'ancoratge (*anchoring*) a Ethereum Sepolia, dissenyat precisament per a compensar la menor resiliència de seguretat d'una xarxa privada.

Finalment, es va decidir mantenir el disseny de xarxa privada permissionada controlada pel consorci d'universitats. Tanmateix, es va decidir conservar la integració amb **Pera Wallet** a la interfície client. Aquesta decisió de disseny manté viva la possibilitat de publicar l'aplicació en un futur a la mainnet pública d'Algorand, ja que el *software* és plenament compatible amb carteres externes; només caldria implementar un mètode de distribució inicial de saldos o acceptar un relayer distribuït de comissions.

4.7 Mètode de Recompte

Una de les eleccions clau en el disseny de Demochain és el mètode per avaluar les preferències ordinals i determinar-ne el guanyador de la consulta.

4.7.1 El Teorema d'Impossibilitat d'Arrow

Dins de la teoria de l'elecció social, el **Teorema de la Impossibilitat d'Arrow** [30] demostra matemàticament que cap sistema de votació ordinal pot complir simultàniament un conjunt de cinc criteris o condicions de justícia electoral quan hi ha tres o més candidats:

1. **Domini no restringit (Universalitat):** El sistema ha d'admetre qualsevol perfil o combinació possible de preferències ordinals que els electors decideixin introduir. Els votants tenen llibertat absoluta per a ordenar els candidats com desitgin.
2. **Absència de dictador (No dictadura):** No hi ha d'haver cap elector que determini de forma unilateral el resultat del sistema independentment de les preferències de la resta de votants. El comportament col·lectiu ha de reflectir múltiples veus.

3. **Eficiència de Pareto (Unanimitat):** Si absolutament tots els electors prefereixen de forma estricta el candidat X per sobre del candidat Y, l'ordre col·lectiu del recompte ha de situar X abans que Y.
4. **Independència d'alternatives irrelevantes (IIA):** La posició de preferència relativa entre dos candidats X i Y en el recompte final només ha de dependre de com els electors valorin X i Y entre ells, sense que s'alteri per la presència, introducció o eliminació d'un tercer candidat Z.
5. **Monotonicitat:** Si un elector decideix modificar la seva papereta per a elevar la posició o el suport a una candidatura concreta, la posició col·lectiva final d'aquesta candidatura no ha de decaure de cap de les maneres (és a dir, donar més suport no ha de perjudicar mai el candidat).

Aquest teorema demostra matemàticament que **no existeix un sistema electoral ordinal perfecte**, i que qualsevol decisió de disseny representa una tria de compromís (*trade-off*) on s'ha de sacrificar qualche criteri de justícia teòrica per a obtenir viabilitat pràctica.

4.7.2 Comparativa dels Mètodes de Votació i el Criteri de Condorcet

El **criteri de Condorcet** és un principi clau en la teoria d'elecció social. Estipula que si un candidat és preferit de forma majoritària en tots els enfrontaments directes o per parelles (duels en un-contrà-un) davant de cadascun dels altres candidats, aquest candidat s'anomena **guanyador de Condorcet**. En conseqüència, qualsevol mètode electoral que es defineixi com a just hauria de seleccionar aquest guanyador com a vencedor de l'elecció.

Per justificar l'ús del **mètode Schulze**, varem fer una comparació detallada amb altres sistemes comuns:

- **Sistema majoritari simple (vot únic):** Molt inexpressiu. L'elector només pot triar un candidat, perdent tota informació sobre les seves altres preferències. Sovint força els votants al vot estratègic i no sol triar el guanyador de Condorcet.
- **Mètode de Borda:** Tot i ser ordinal i senzill (es donen punts en funció de la posició), viola el criteri de Condorcet. A més, és extremadament vulnerable a atacs de manipulació de clons, on s'introdueixen candidats pràcticament idèntics per a inflar artificialment la puntuació d'una opció i distorsionar el resultat.
- **Vot Transferible o Desfilament Instantani (IRV):** S'elimina el candidat amb manco vots de primera preferència i es transfereixen els vots fins a trobar majoria. El defecte clau és que no és monòton (viola el criteri de monotonicitat), el qual pot provocar que un candidat perdi l'elecció de forma paradoxal pel simple fet d'obtenir més suport en primera preferència per part d'un grup d'electors. Tampoc compleix necessàriament amb el criteri de Condorcet.

Davant d'aquestes alternatives, s'ha seleccionat el **mètode Schulze**. Aquest algorisme realitza comparacions per parelles directes entre tots els candidats i construeix un graf de victòries orientades. Schulze és un mètode de Condorcet (sempre selecciona el

guanyador de Condorcet si existeix) i compleix simultàniament amb la monotonicitat, la independència de clons i la simetria de reversió, convertint-se en el mètode més robust i equitatiu per al domini de Demochain.

4.7.3 Inconvenients del Mètode Schulze

Tot i ser excel·lent des del punt de vista de la justícia matemàtica, varem identificar quatre inconvenients principals en emprar Schulze per a un sistema real:

1. **Elevada complexitat computacional ($O(n^3)$):** La determinació dels camins més forts en el graf de preferències per a resoldre els cicles (paradoxa de Condorcet) es basa en l'algorisme de Floyd-Warshall. El cost computacional creix de forma cúbica amb el nombre de candidats (n). Per mor d'això, calcular el recompte directament dins d'un contracte d'Algorand és completament inviable a causa dels límits de cost d'*opcode* i temps d'execució de la màquina virtual, forçant a delegar el càlcul a l'aplicació client.
2. **Baixa llegibilitat cognitiva per a la ciutadania:** La majoria de votants està familiaritzada amb recomptes senzills (sumar vots directes). Entendre la lògica de camins de preferència més forts en un graf de duels Directed Acyclic Graph (DAG) i verificar que els resultats són correctes sense dependre d'un ordinador és gairebé impossible per al públic general. Això pot generar desconfiança o distanciament psicològic cap a la legitimitat de l'urna electrònica.
3. **Manca d'escalabilitat del missatge de vot:** Cada elector ha d'ordenar les seves preferències. La quantitat d'informació necessària per a codificar el vot creix de forma quadràtica amb relació al nombre de candidats. Per a eleccions amb molts candidats, la *payload* de la transacció i l'emmagatzematge en *boxes* de la *blockchain* creixen de forma ineficient, augmentant l'MBR a Algorand.
4. **Complexitat en les regles de desempat:** Malgrat que l'algorisme està dissenyat per a definir clarament els camins de victòria, en escenaris reals es poden produir empats on els valors dels camins més forts de dos candidats siguin exactament idèntics. Resoldre aquests empats de manera justa exigeix implementar protocols deterministes addicionals preestablerts (com analitzar preferències de primer nivell o sortejos pseudoaleatoris basats en *hashes on-chain*), augmentant la dificultat del codi.

4.8 Llenguatge Ubic (DDD)

Per tal de garantir que tot l'equip de programació, els custodis i els auditors utilitzin el mateix significat de les dades, s'ha adoptat una aproximació de **Domain-Driven Design (DDD)** [31] definint un llenguatge comú per a tot el projecte. El domini de Demochain s'estructura a partir de la gènesi d'una **Organització**, la qual representa l'entitat jurídica o el col·lectiu registrat al sistema per un **Organitzador** que és l'administrador amb drets d'escriptura per gestionar les adreces del **Cens** d'aquella entitat. Cada membre autoritzat d'aquest cens rep el rol de **Votant** i té dret a participar en les consultes de les propostes. La **Proposta** constitueix la consulta electoral *on-chain*, que s'inicia sota

una **Fase d'Aprovació** on es requereix un quòrum de dues tercers parts favorables per transitar cap a la **Fase d'Elecció** activa, durant la qual s'admet l'enviament de paperetes.

La manifestació del sufragi del votant s'anomena **Papereta**, la qual conté el missatge digital signat amb les seves preferències, ja sigui com un vot de si/no o com un *array* d'índexs ordinals. En finalitzar el procés, es duu a terme el **Recompte**, que en el disseny teòric representa l'agregació homomòrfica multiplicativa *on-chain* de les paperetes xifrades sota una clau de desxifratge de llindar controlada per diversos **Custodis o Trustees**. Cada custodi aporta la seva **Share de Desxifratge** de forma independent sota un esquema de **Consens K-of-N**. Per prevenir el doble vot de manera anònima, es preveu l'ús d'un **Nullifier o Anul·lador** determinista. En l'**MVP** simplificat actual, las paperetes es desen en clar i el recompte es calcula *client-side*, mantenint l'estructura terminològica com a model per a futures iteracions. Tot el cicle es tanca amb l'**Ancoratge**, que és l'acció determinista de publicar el *hash* de resultats finals d'Algorand a la xarxa d'Ethereum Sepolia per certificar-ne la integritat a llarg termini.

4.9 Model de Dades

El disseny de les dades del projecte s'ha estructurat de manera que encaixi amb les limitacions d'emmagatzematge transaccional d'ambdues xarxes.

4.9.1 Estructures d'Algorand (ARC-4 Structs en algopy)

Al contracte intel·ligent d'Algorand escrit en Python, s'utilitzen estructures de dades **ARC-4** que es codifiquen de forma compacta en l'estat de les *boxes*. Les estructures estan definides a `contract.py`:

L'estructura `Organization` s'encarrega d'emmagatzemar les metadades generals de cada col·lectiu registrat al sistema. Conté un identificador numèric de l'entitat, el nom textual de l'entitat, una breu descripció, l'adreça de *wallet* del seu organitzador i el nombre total de membres que formen el cens. Aquestes estructures es classifiquen i es desen en un mapa de caixes de dades persistents associades a l'estat del contracte sota el prefix de clau d'organització:

```
class Organization(arc4.Struct):
    org_id: arc4.UInt64
    name: arc4.String
    description: arc4.String
    organizer: arc4.Address
    member_count: arc4.UInt32
```

Per altra banda, l'estructura `Proposal` recull el cicle de vida de cada elecció. Emmagatzema l'identificador de l'organització a la qual pertany, el títol, la descripció del sufragi, un *array* dinàmic de cadenes de text amb la llista de candidats o opcions, i les dates d'inici i de tancament de la finestra de votació representades en format UNIX:

```
class Proposal(arc4.Struct):
    org_id: arc4.UInt64
    title: arc4.String
    description: arc4.String
    options: arc4.DynamicArray[arc4.String]
    starting_date: arc4.UInt64
```



```
ending_date: arc4.UInt64
```

Per dur el registre dels sufragis d'aprovació i controlar la transició cap a la fase electoral s'empra l'estructura `ApprovalTally`, que conté el nombre total de vots emesos a favor de la proposta i el nombre acumulat d'electors participants en la fase d'aprovació. Finalment, es defineixen les estructures `BallotId` i `CensusId` per actuar com a claus compostes *on-chain*, les quals s'encarreguen d'enllaçar de forma unívoca l'adreça del votant amb la proposta de votació preferencial o amb la llista del cens de l'organització:

```
class ApprovalTally(arc4.Struct):
    votes_for: arc4.UInt32
    total_votes: arc4.UInt32

class BallotId(arc4.Struct):
    sender: arc4.Address
    proposal_id: arc4.UInt64

class CensusId(arc4.Struct):
    org_id: arc4.UInt64
    member: arc4.Address
```

En l'**MVP** desenvolupat, el cens s'emmagatzema com un `BoxMap` que fa *mapping* de cadascun dels `CensusId` amb un valor booleà, i les paperetes preferencials en clar es desen directament com un `BoxMap` que enllaça cadascun dels `BallotId` amb un *array* dinàmic d'enters sense signe (`arc4.DynamicArray[arc4.UInt8]`) que reflecteix el rànquing.

4.9.2 Estructures de Dades d'Ethereum (`NotaryContract.sol`)

Al contracte intel·ligent escrit en Solidity d'Ethereum, s'emmagatzema l'estat del consorci electoral per a cada sufragi finalitzat mitjançant l'estructura `Election`. Aquesta estructura recull el llindar `thresholdK` necessari per segellar el resultat, un *array* dinàmic amb les adreces públiques de les universitats certificadores del consorci, i mapes de correspondència per validar quins nodes estan autoritzats a firmar, quines submissons s'han registrat des de les seves adreces de Sepolia i l'acumulació de vots que té cada variant de *hash*. L'estructura finalitza amb dos valors booleans que indiquen si l'elecció es troba actualment oberta a l'ancoratge o si ja ha assolit el quòrum i es troba oficialment ancorada a la xarxa:

```
struct Election {
    uint256 thresholdK;
    address[] authorizedNodes;
    mapping(address => bool) isAuthorized;
    mapping(address => bytes32) submissions;
    mapping(bytes32 => uint256) \textit{hash}Count;
    bool open;
    bool anchored;
}
```

4.9.3 Procés de Derivació del *hash* Canònic

El *hash* que s'envia a Ethereum Sepolia representa el compromís d'integritat del resultat electoral. Per calcular-lo de manera idèntica a tots os nodes validadors s'aplica el següent procés determinista:

1. S'escanegen totes les *boxes* d'Algorand com a llista de paperetes amb prefix `eb_` vinculades a la proposta.
2. Es classifiquen les paperetes ordenant les adreces dels electors de forma lexicogràfica. Aquest pas és vital, ja que l'ordre físic de rebuda de transaccions depèn de la latència de la xarxa i canviaria el format en bytes de cada node.
3. Se serialitza la informació en una cadena de text tipus **JSON** canònica (claus ordenades de la A a la Z, sense cap espai en blanc, tabulacions o salts de línia):

```
{
  "ballots": [
    {
      "preference": [2, 0, 1],
      "voter": "ADDR_A"
    },
    {
      "preference": [0, 2, 1],
      "voter": "ADDR_B"
    }
  ],
  "options": ["Opció A", "Opció B", "Opció C"],
  "proposal_id": 1,
  "title": "Rector 2026"
}
```

4. Es calcula el *hash* **SHA**-256 d'aquesta representació en bytes, produint un valor únic hexadecimal de 32 bytes (bytes32).

CAPÍTOL 5

DISSENY

Aquest capítol presenta el disseny tècnic i d'interacció de DemoChain. Es descriu primer l'arquitectura global seguint el model C4 (context, contenidors i components), després el patró de desplegament basat en Docker, tot seguit el disseny de la interfície web amb què interactua el votant, i finalment els fluxos de comunicació principals representats com a diagrames de seqüència.

5.1 Arquitectura: visió C4

Per descriure l'arquitectura s'ha seguit el model C4 de Simon Brown, que ofereix quatre nivells de detall progressius. En aquest capítol s'inclouen els tres primers nivells i el quart queda implícit al codi de l'aplicació. Cada nivell s'orienta a un públic diferent: el primer dona una vista de conjunt útil per a qualsevol part interessada, el segon situa les peces de *software* que cal mantenir, i el tercer entra al detall dels components que conviuen dins cada contenidor.

5.1.1 Nivell 1 - Context

El primer nivell mostra el sistema des de fora i n'identifica els actors humans i els sistemes externs amb què interacciona ([Figura 5.1](#)). Hi ha dos rols humans diferenciats: el **Votant**, que proposa, vota eleccions des de la interfície web o inclús gestiona organitzacions, i l'**Administrador** de nodes, que gestiona la llista blanca d'adreces autoritzades a publicar *hashes* a Ethereum. El sistema en si interactua amb dues xarxes (Algorand i Ethereum) i amb una cartera externa que custodia les claus privades del votant.

La decisió de mantenir aquestes dues xarxes diferenciades es justifica al capítol d'Anàlisi: Algorand allotja la lògica de votació en una xarxa permissionada de baix cost, i Ethereum actua únicament com a notari públic per a la prova final, sense participar en cap pas intermedi del recompte.

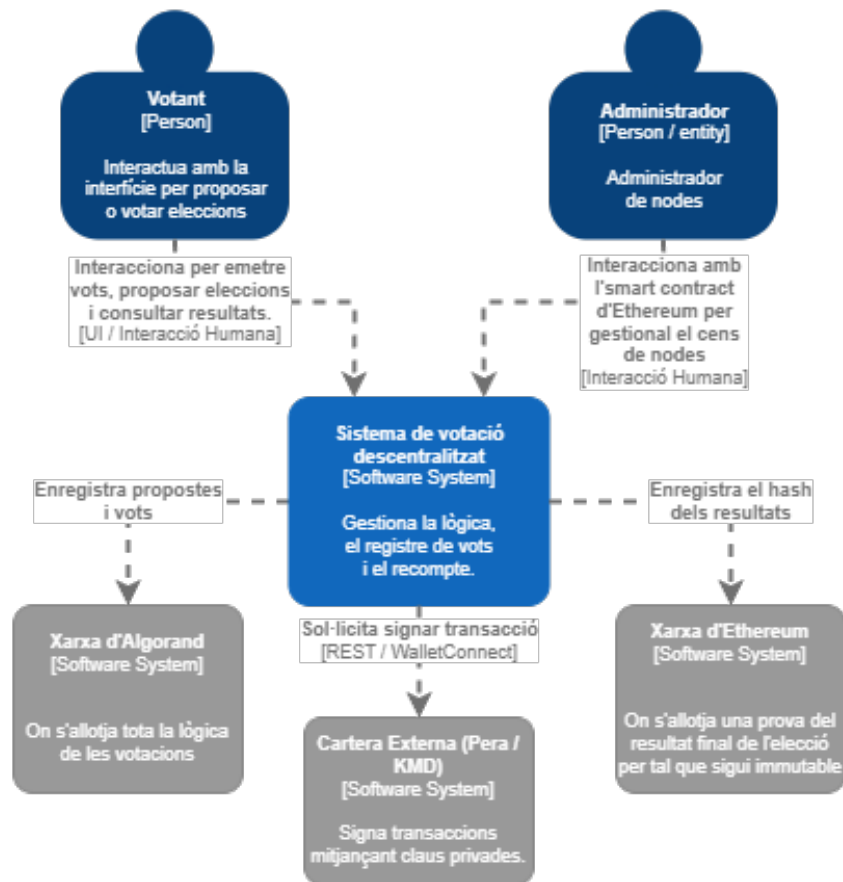


Figura 5.1: Diagrama C4 de nivell 1: context de DemoChain.

5.1.2 Nivell 2 - Contenidors

El segon nivell descompon el sistema en quatre contenidors de *software*, cadascun amb una tecnologia i una responsabilitat clares (Pàgina 43):

- **Client - Interfície web** (React). Punt d'entrada de l'usuari. Construeix transaccions **ABI**, demana signatures a la cartera, consulta l'estat de la cadena i hi escriu propostes, vots i operacions de gestió d'organitzacions (si es el cas).
- **Contracte intel·ligent d'Algorand** (Algopy/TEAL). Conté la lògica de propostes, eleccions i organitzacions. Manté l'estat persistent (cens, paperetes, finestres temporals) a Box Storage.
- **Servei d'ancoratge** (Python). Procés autònom que s'executa paral·lelament a cada node. Detecta eleccions tancades, en deriva un *hash* determinista i l'envia al contracte d'Ethereum.
- **Contracte intel·ligent d'Ethereum** (Solidity). El *NotaryContract*: acumula els *hashes* enviats pels nodes i registra l'elecció com a ancorada quan se n'arriba al consens de *K-of-N*.

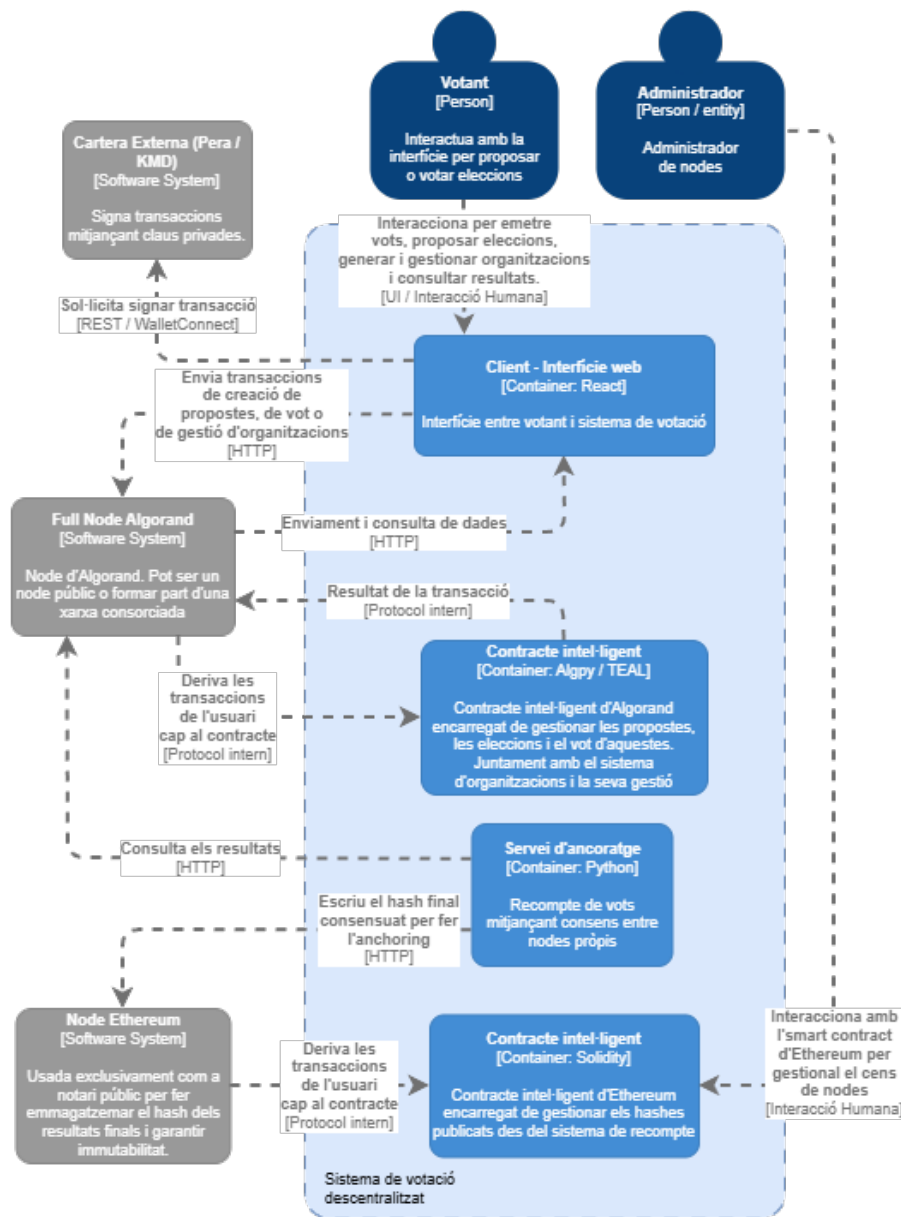


Figura 5.2: Diagrama C4 de nivell 2: contenidors de DemoChain.

El client no parla directament amb el contracte: tota interacció passa per un **Full Node Algorand**, que actua de gateway HTTP cap a la xarxa. Aquesta separació permet que diferents universitats puguin oferir el seu propi node sense haver de tocar el client. De manera anàloga, el servei d'ancoratge llegeix paperetes del node Algorand i escriu al *NotaryContract* a través d'un node Ethereum. En producció aquest últim queda fora del *software* del projecte (s'utilitza un proveïdor de la testnet d'Ethereum, Sepolia); per al desenvolupament local s'inclou un node Hardhat dins del mateix docker-compose que desplega automàticament el contracte.

5.1.3 Nivell 3 - Components

El tercer nivell entra dins de cada contenidor i hi descompon els components funcionals. Es presenten quatre vistes, una per contenidor.

Client web

El client és una aplicació de pàgina única (SPA) que organitza la responsabilitat en capes ben delimitades, de manera que els canvis a la interfície no impacten la lògica de domini ni la integració amb la cadena.

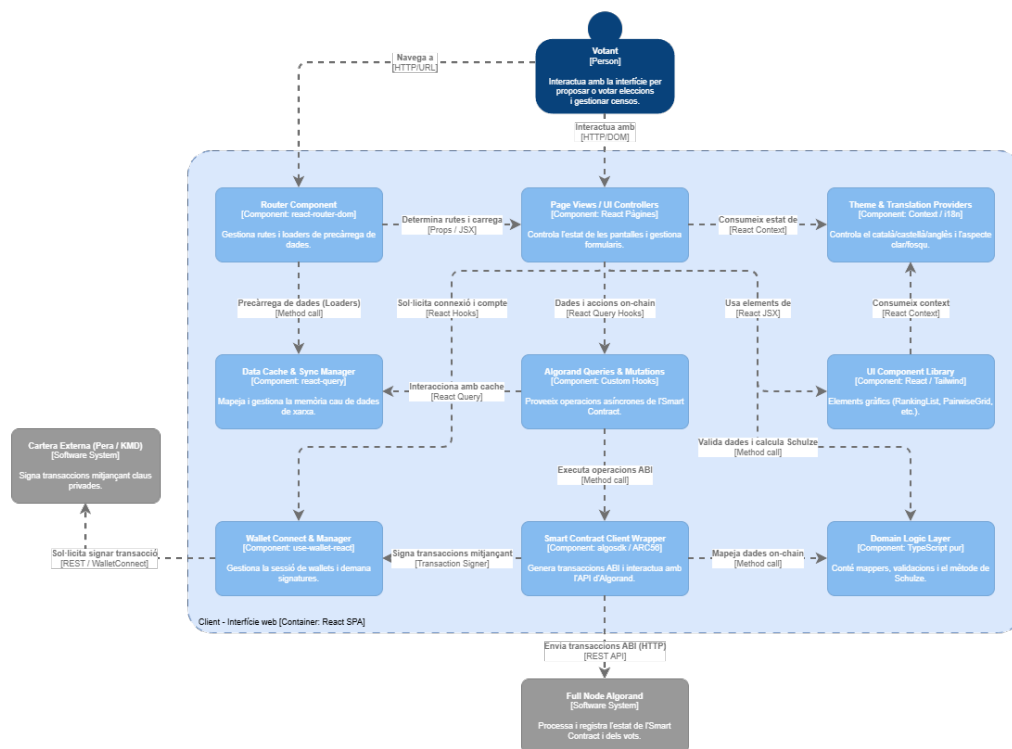


Figura 5.3: Components del client web.

Els components principals són:

- **Router**, que gestiona les rutes i els *loaders* de precàrrega de dades amb `react-router-dom`.
- **Page Views**, que controlen l'estat de cada pantalla i orquestren formularis amb `react-hook-form` i `zod`.
- **Theme & Translation Providers**, basats en `React Context` i `i18next`, que mantenen el tema clar/fosc i les tres llengües suportades.
- **Data Cache & Sync Manager** (`react-query`), que ofereix una caché coherent sobre les consultes a la cadena i facilita la invalidació després de cada transacció.
- **Algorand Queries & Mutations**, conjunt de *custom hooks* que encapsulen les operacions asíncrones del contracte.

- **UI Component Library**, els elements gràfics reutilitzables (`RankingList`, `PairwiseGrid`, etc.) construïts amb TailwindCSS.
- **Wallet Connect & Manager** (`use-wallet-react`), responsable de la sessió de cartera i de demanar signatures.
- **Smart Contract Client Wrapper**, que utilitza `algosdk` per generar transaccions **ABI** a partir del descriptor **ARC56** del contracte.
- **Domain Logic Layer**, codi TypeScript pur que conté *mappers*, validacions i el càlcul del mètode de Schulze.

El càlcul del guanyador amb Schulze viu al client perquè el contracte d'Algorand no emmagatzema cap resultat precalculat: només paperetes individuals. Això simplifica el contracte i permet que qualsevol persona pugui reproduir el recompte de manera independent.

Contracte intel·ligent d'Algorand

El contracte té com a punt d'entrada un **encaminador principal** que classifica cada transacció rebuda i la deriva al mòdul adient. Els mòduls coexisteixen al mateix *application*, però estan separats lògicament per responsabilitats (**Figura 5.4**):

- **Lògica de votació**: rep el vot, comprova que la direcció pertany al cens i que la proposta està en una finestra vàlida, i finalment emet o sobreescriu el vot.
- **Lògica de propostes**: rep i valida les propostes (dates, opcions, organització) i les desa a l'emmagatzematge.
- **Emmagatzematge d'estat**: utilitza Algorand Box Storage per persistir propostes, eleccions, vots d'aprovació, vots d'elecció, organitzacions i censos.

Servei d'ancoratge

El servei d'ancoratge és un procés Python autònom dissenyat per executar-se paral·lelament a cada node (**Figura 5.5**). La seva responsabilitat és detectar quan una elecció es tanca, calcular-ne un *hash* determinista i enviar-lo al *NotaryContract* d'Ethereum.

La clau d'aquest disseny és el determinisme: tots els nodes han de produir exactament el mateix *hash* a partir del mateix estat de la cadena. Per garantir-ho, el servei ordena lexicogràficament les paperetes per direcció del votant i en genera una representació **JSON** canònica (claus ordenades, sense espais) abans d'aplicar **SHA-256**. Així s'evita que diferències accidentals d'ordre o format facin divergir nodes honestos.

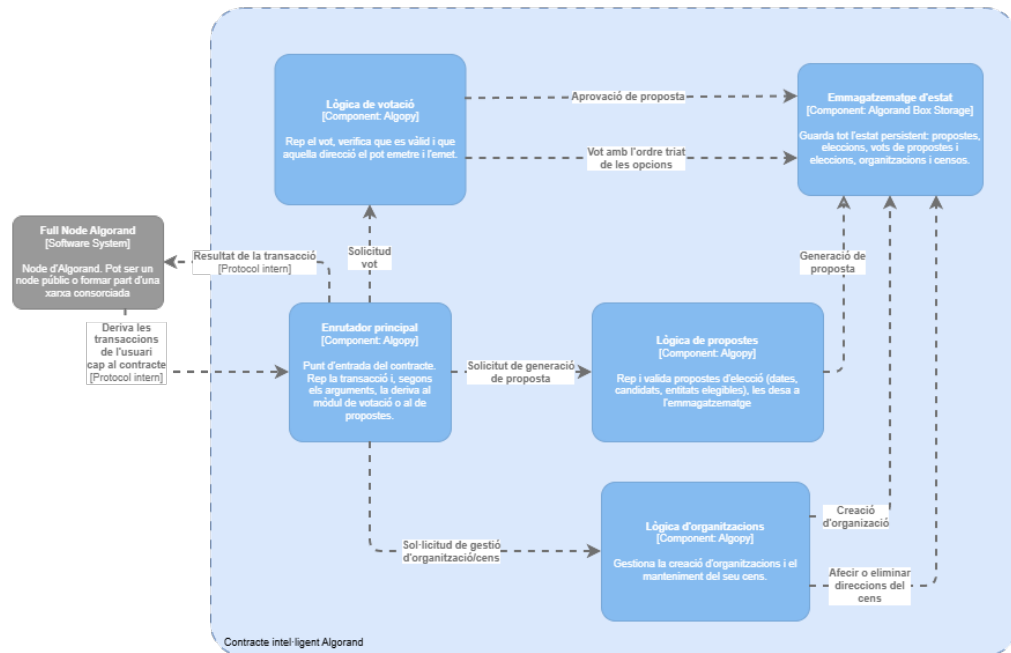


Figura 5.4: Components del contracte intel·ligent d'Algorand.

Contracte intel·ligent d'Ethereum

El *NotaryContract* (Figura 5.6) implementa el consens K -of- N entre els nodes universitaris autoritzats. L'administrador manté una llista blanca de direccions Ethereum i, abans que comenci l'ancoratge d'una elecció, l'obre al contracte. En aquest moment es congelen tant la llista de nodes autoritzats com el llindar K ($\lceil 2/3 N \rceil$), de manera que altes o baixes posteriors no afecten les eleccions ja obertes.

Cada node pot enviar un únic *hash* per elecció. Quan K nodes coincideixen en el mateix valor, el contracte emet un esdeveniment públic que registra l'elecció com a ancorada de manera immutable. La consulta posterior (quin *hash* va enviar cada node, quan, i si finalment s'ha consensuat) és lliure i no requereix cap autorització.

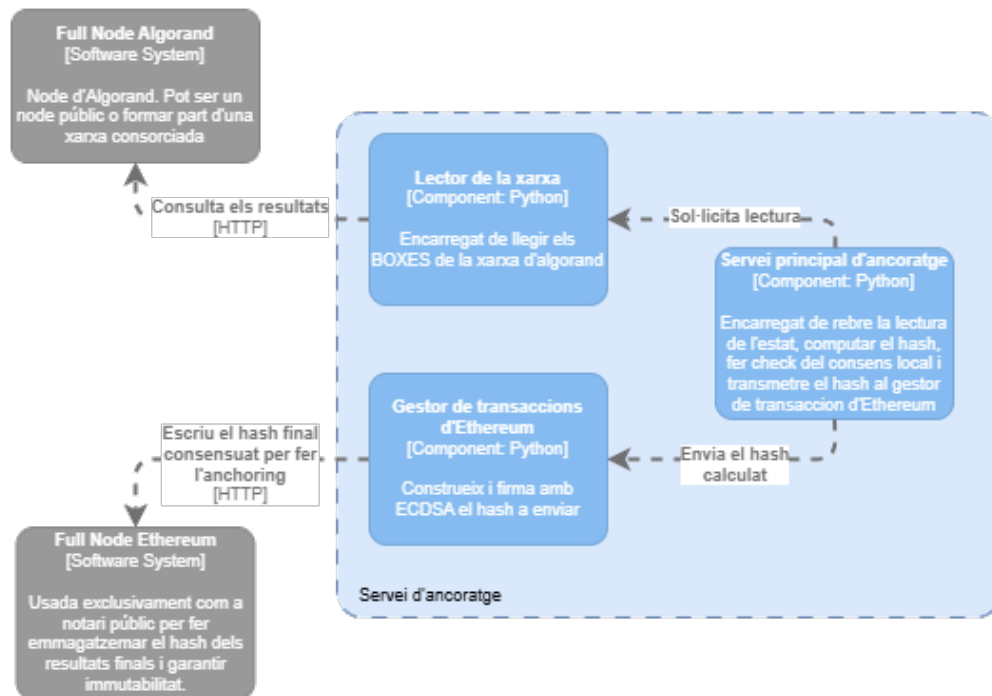


Figura 5.5: Components del servei d'ancoratge.

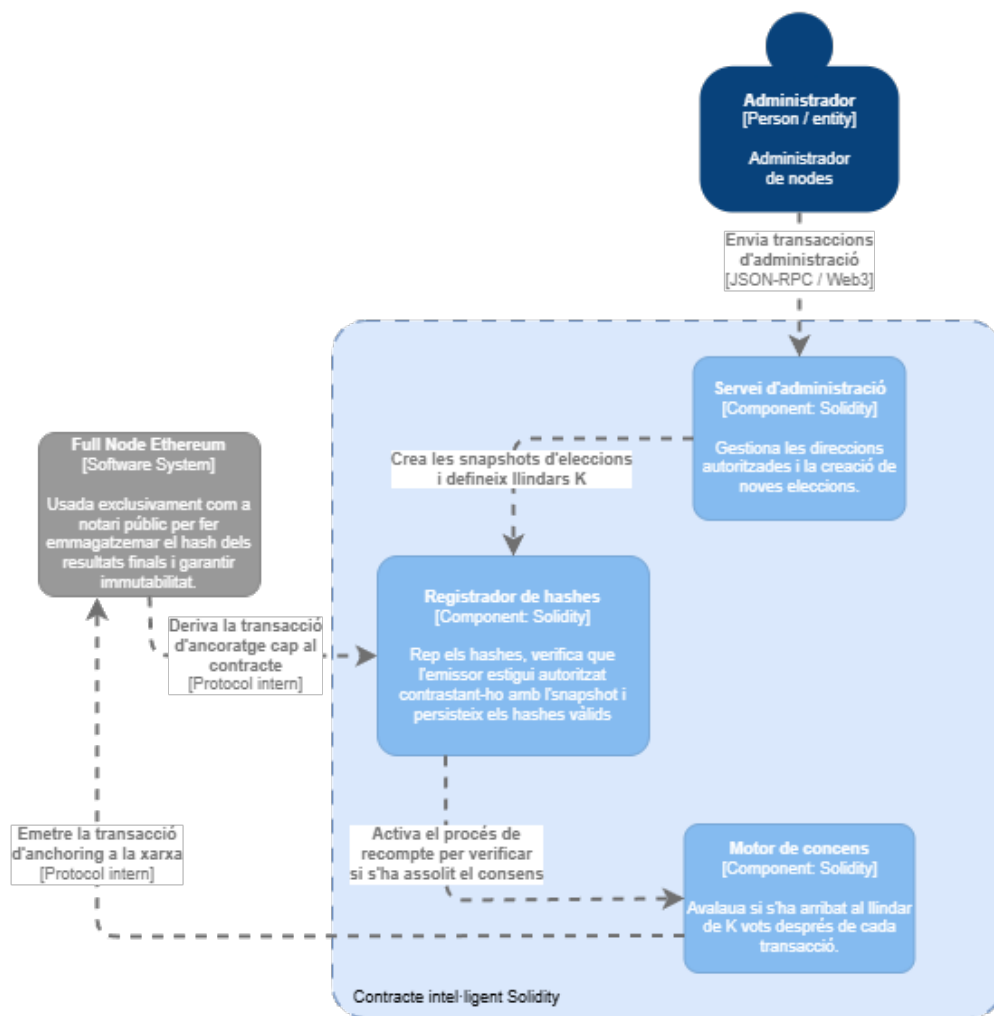


Figura 5.6: Components del contracte intel·ligent d'Ethereum.

5.2 Patró de desplegament

5.2.1 Stack Docker Compose

Tot l'entorn local de DemoChain es desplega amb un únic fitxer `docker-compose.yml`. Aquesta decisió persegueix tres objectius: (1) que qualsevol desenvolupador o universitat pugui aixecar el sistema amb una sola comanda, (2) aïllar les versions concretes d'Algorand, l'*indexer* i el client, i (3) reproduir en local la mateixa topologia que s'espera en producció a cada node universitari.

La pila de serveis és la següent:

- **algod**: el node d'Algorand. A dins conté dos nodes interns (un *lead* a 8080 i un *follower* a 8081) i el dimoni Key Management Daemon (**KMD**) que custodia les claus de prova.
- **contract-deploy**: contenidor efímer basat en uv (Python 3.12) que compila el contracte amb PuyaPy i el desplega contra algod.
- **indexer-db**: una instància PostgreSQL que actua de magatzem per a l'*indexer*.
- **conduit**: lector de *state deltas* del *follower* d'Algorand que alimenta la base de dades.
- **indexer**: l'*indexer* d'Algorand, que ofereix una Application Programming Interface (**API**) HTTP de cerca sobre la cadena.
- **client**: el contenidor del client web, basat en Bun, que serveix Vite en mode *hot-reload*.

5.2.2 Inicialització i flux APP_ID

Un punt rellevant del disseny és com s'enllaça el client amb el contracte desplegat. El servei `contract-deploy` no és un servei de llarga durada: només existeix per construir el contracte, desplegar-lo a algod i deixar el seu identificador disponible al client. El flux és:

1. algod arrenca i el *healthcheck* confirma que respon.
2. `contract-deploy` sincronitza les dependències amb `uv sync`, executa la *build* del contracte i el desplega.
3. De la sortida del desplegament n'extreu el valor `app_id` i el persisteix com a `VITE_APP_ID` a `client/.env.local`.
4. Només llavors el servei `client` arrenca, ja sabent contra quina aplicació ha d'apuntar.

Quan es vol reiniciar la cadena (per exemple amb `docker compose down -v`), el cicle es repeteix i el client rep automàticament el nou APP_ID. Així no cal cap intervenció manual per mantenir client i contracte sincronitzats.

5.2.3 Aixecar un node d'Algorand

El mateix `docker-compose.yml` serveix de plantilla per desplegar un node real. Cada entitat participant en el consorci aixeca la seva pròpia còpia del `compose`, però apuntant el seu `algorand` a la xarxa permissionada compartida i afegint-hi el servei d'ancoratge amb les seves credencials Ethereum. El client web és estàtic i, per tant, qualsevol node pot servir-lo sense que això comprometi la integritat del sistema: la confiança recau en el consens *K-of-N* del *NotaryContract*, no en la disponibilitat d'un node concret.

5.3 Disseny de la interfície d'usuari

La interfície s'ha dissenyat amb un objectiu principal: que un votant que es connecti per primera vegada puguin entendre què està a punt de fer i en quin punt del cicle es troba cada proposta, sense necessitat de conèixer els detalls de la cadena. Per aconseguir-ho s'han aplicat tres principis transversals:

- **Vocabulari uniforme.** Els termes *proposta*, *elecció*, *aprovació*, *cens* i *organització* es fan servir de manera consistent a totes les pantalles i corresponen al llenguatge ubic definit a l'anàlisi.
- **Múltiples idiomes i tema personalitzable.** La interfície està traduïda a català, castellà i anglès, i suporta tema clar i fosc.
- **Estat visible.** Cada proposta indica de manera explícita en quin estat del cicle es troba mitjançant *badges* i filtres, evitant que el votant hagi d'inferir-ho a partir de dates.

5.3.1 Llengües, tema i navegació global

La pantalla d'inici presenta la proposta de valor del producte i serveix com a entrada al sistema. La capçalera global, present a totes les rutes, ofereix accés directe a les seccions principals (*Inici*, *Organitzacions*, *Propostes*, *Nova proposta*), al selector d'idioma, al canvi de tema i al botó de connexió de cartera ([Figura A.1](#)).

5.3.2 Gestió d'organitzacions

Les organitzacions són l'arrel de qualsevol activitat de votació: una proposta sempre pertany a una organització i només els membres del seu cens hi poden participar. La pantalla de llistat mostra les organitzacions a què pertany o que administra l'usuari connectat, amb una insígnia d'*Organitzador* quan és el cas ([Figura A.2](#)).

El detall d'una organització presenta dues pestanyes complementàries: **Propostes** (les propostes obertes o tancades d'aquesta organització) i **Cens** (les adreces autoritzades a participar) ([Figura A.3](#)). L'administrador pot afegir o eliminar membres des de la pestanya de cens, sigui d'un en un o per càrrega massiva mitjançant un fitxer **CSV**. La interfície amaga al votant la complexitat de la transacció subjacent: el client trosseja internament les adreces en lots compatibles amb el límit del contracte (set adreces per transacció) i guia el votant pel procés de signar les transaccions consecutives.

5.3.3 Propostes i cicle de vida

La pantalla de propostes presenta totes les propostes visibles per a l'usuari, amb cercador i filtres per estat: *Actives*, *Pendent d'aprovació*, *Aprovada*, *Votació oberta*, *Tancada* i *Totes*. Aquests estats segueixen exactament la màquina d'estats descrita a la documentació funcional del sistema (*PendingApproval*, *PendingStart*, *Open*, *Closed*, *Rejected*) i serveixen perquè el votant detecti d'un cop d'ull en quines propostes pot actuar i en quines no (Figura A.4).

5.3.4 Creació d'una proposta

La creació d'una proposta s'ha modelat com un assistent de cinc passos: *Organització* (A.7), *Bàsics* (A.8), *Dates* (A.9), *Opcions* (A.10) i *Revisió* (A.11). La divisió en passos persegueix dos objectius: dividir mentalment la decisió (cada pas demana un únic tipus d'informació) i poder validar restriccions complexes -com la finestra temporal mínima- abans d'avançar al pas següent. Tot el formulari es valida amb zod, de manera que cap dada arriba al contracte si no compleix les regles bàsiques. Una vegada es crea la proposta es reencamina a la pantalla per aprovar-la o revocar-la (Figura A.5). Quan la proposta s'aprova es veu tal com a la Figura A.6).

5.3.5 Emissió de vot

Quan una proposta és aprovada, els membres del cens poden ordenar les opcions arrossegant-les fins a compondre el seu ranking de preferències. El component es construeix amb @dnd-kit i admet tant ratolí com teclat, garantint accessibilitat (Figura A.12). El vot d'elecció es pot modificar tantes vegades com calgui mentre la finestra estigui oberta; en canvi, el vot d'aprovació previ (binari, visible a la Figura A.5) és immutable un cop emès. Una vegada emès dona feedback com es veu en la Figura A.13 i després passa a una pàgina on pot veure el seguiment de quanta gent ha votat (Figura A.14).

5.3.6 Resultats i verificació pública

Un cop tancada l'elecció, la pantalla de resultats mostra el ranking final calculat amb el mètode de Schulze (Figura A.15), així com la matriu de comparacions parelles que permet entendre per què una opció n'ha vençut una altra (Figura A.16). El càlcul es fa íntegrament al client a partir de les paperetes llegides de la cadena: el contracte d'Algorand no en guarda cap resultat precalculat.

A la mateixa pantalla s'hi inclou un panell de verificació obert a qualsevol persona, fins i tot a qui no és membre del cens. Introduint una direcció de cartera, el client consulta la blockchain i mostra el ranking que aquella direcció va emetre o bé indica que no va votar (Figura A.17). Aquesta funcionalitat, combinada amb l'ancoratge a Ethereum, permet auditar el resultat de manera independent.

5.4 Flux de comunicació amb l'usuari

Aquesta secció recull els fluxos de comunicació més rellevants entre l'administrador, el votant, el client, la cartera i la cadena. Estan representats com a diagrames de seqüència UML i s'han generat a partir de les especificacions Mermaid incloses al repositori.

5.4.1 Gestió d'organitzacions

Aquest flux agrupa les tres operacions que pot fer un administrador sobre una organització: crear-la, donar d'alta noves adreces al cens i eliminar-ne (Figura 5.7). La part més rellevant del disseny és com el client amaga al votant la limitació tècnica del contracte, que només admet un màxim de set adreces per transacció de cens. Quan l'administrador carrega un CSV amb molts membres, el client el trosseja internament en lots de set i sol·licita la signatura de tantes transaccions com calgui, mostrant una barra de progrés. D'aquesta manera, l'única conseqüència visible per a l'administrador és que els censos molt grans tarden una mica més a confirmar-se.

A més, el contracte aplica dues regles invariants que també queden reflectides al diagrama: només l'administrador pot modificar el cens i mai pot eliminar-se ell mateix, ja que tota organització ha de tenir com a mínim un membre.

5.4. Flux de comunicació amb l'usuari

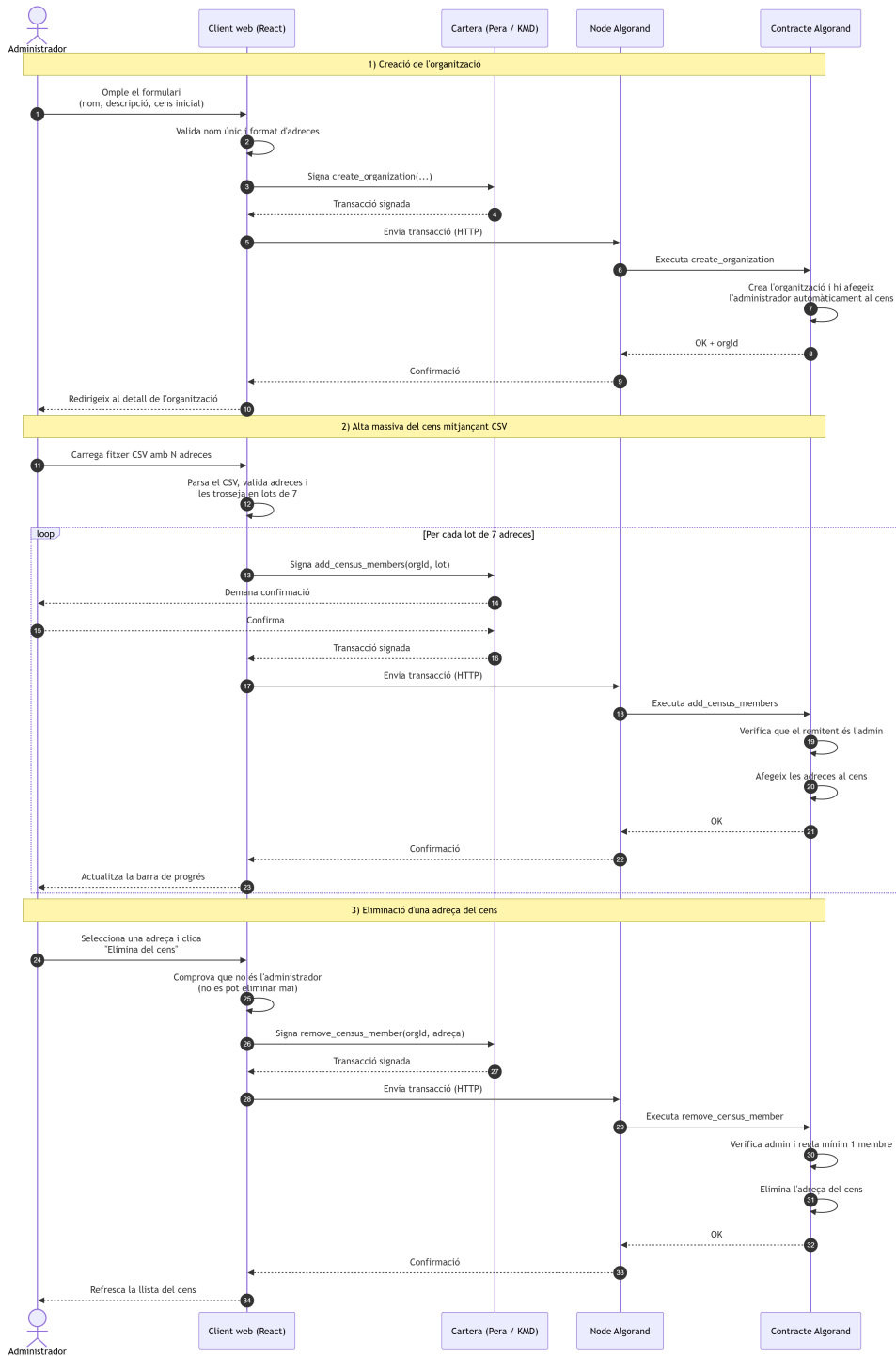


Figura 5.7: Diagrama de seqüència: creació d'una organització i gestió del seu cens (alta massiva via CSV i baixa individual).

5.4.2 Crear una proposta

El flux comença al formulari guiat del client. Un cop l'usuari ha completat els cinc passos, el client construeix la transacció **ABI** corresponent i en sol·licita la signatura a la cartera. La transacció signada s'envia al node Algorand i, en cas d'èxit, el contracte valida les restriccions (cens, dates, opcions) i registra la proposta a Box Storage. El client invalida la seva caché per recollir l'estat actualitzat i mostra la proposta acabada de crear en estat *PendingApproval*.

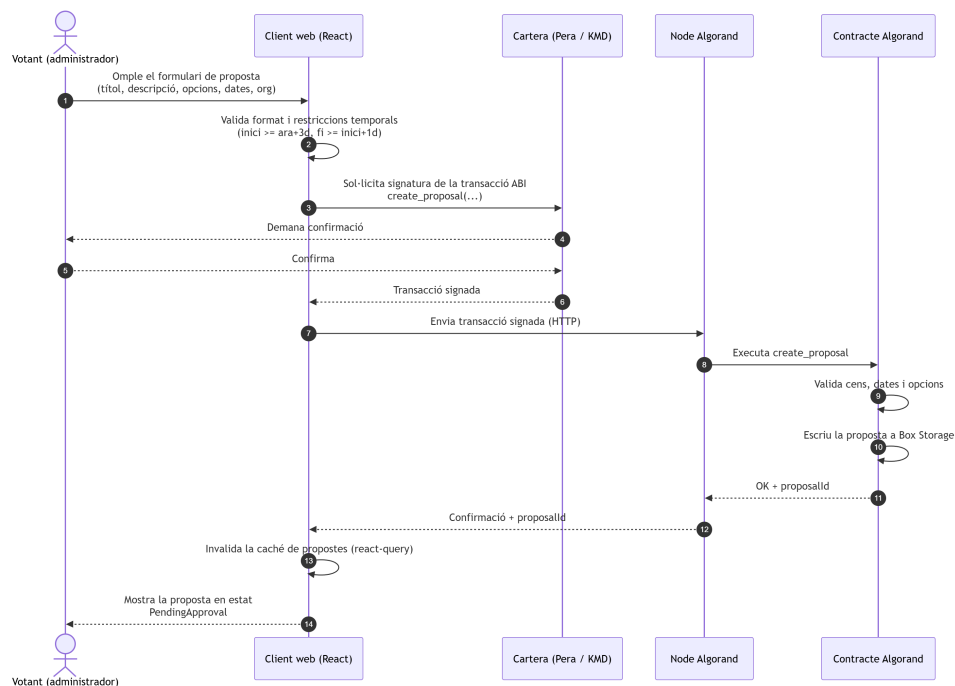


Figura 5.8: Diagrama de seqüència: creació d'una proposta.

5.4.3 Votar l'aprovació d'una proposta

El vot d'aprovació és binari (a favor o en contra) i immutable. El client comprova que la proposta es troba en estat *PendingApproval* i que falten més de tres dies per a la data d'inici; en cas contrari, no permet emetre el vot. El contracte verifica que el votant pertany al cens i que no ha votat amb anterioritat. Aquest disseny és deliberadament conservador: vol garantir que el resultat de la fase d'aprovació es coneix amb prou antelació perquè ningú es pugui sorprendre que una proposta passi (o no passi) a elecció a última hora.

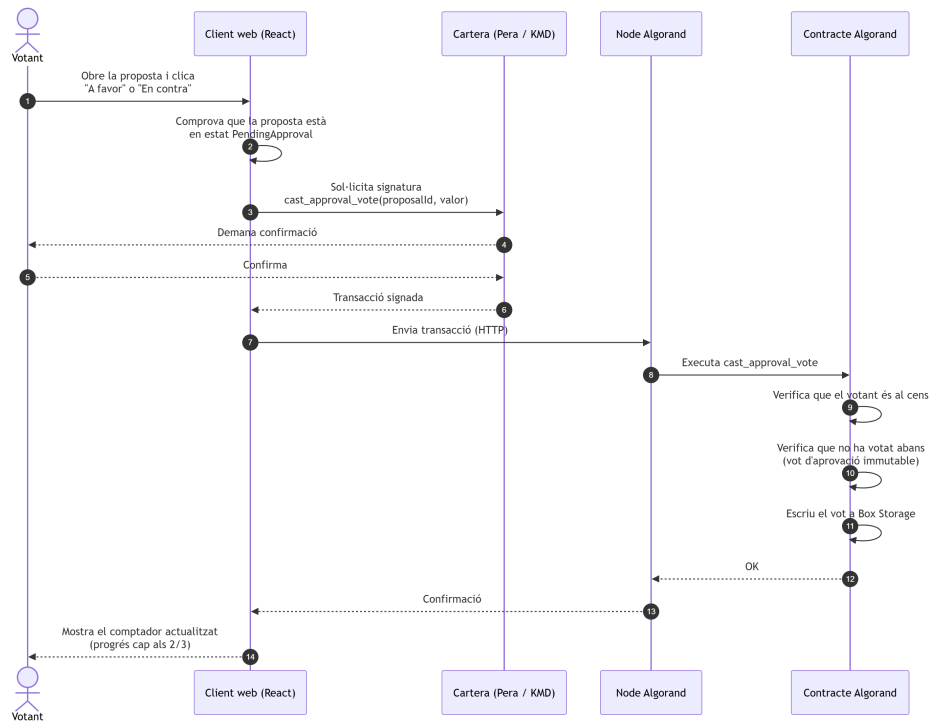


Figura 5.9: Diagrama de seqüència: vot d'aprovació.

5.4.4 Emetre el vot d'elecció (ranking)

Quan la proposta entra a l'estat *Open*, el votant pot ordenar les opcions segons les seves preferències. A diferència del vot d'aprovació, el ranking sí que es pot modificar tantes vegades com calgui mentre l'elecció estigui oberta. El contracte sobreescriu el vot anterior amb cada nova transacció, però en deixa traça a la cadena perquè qualsevol verificació externa només considera l'última versió.

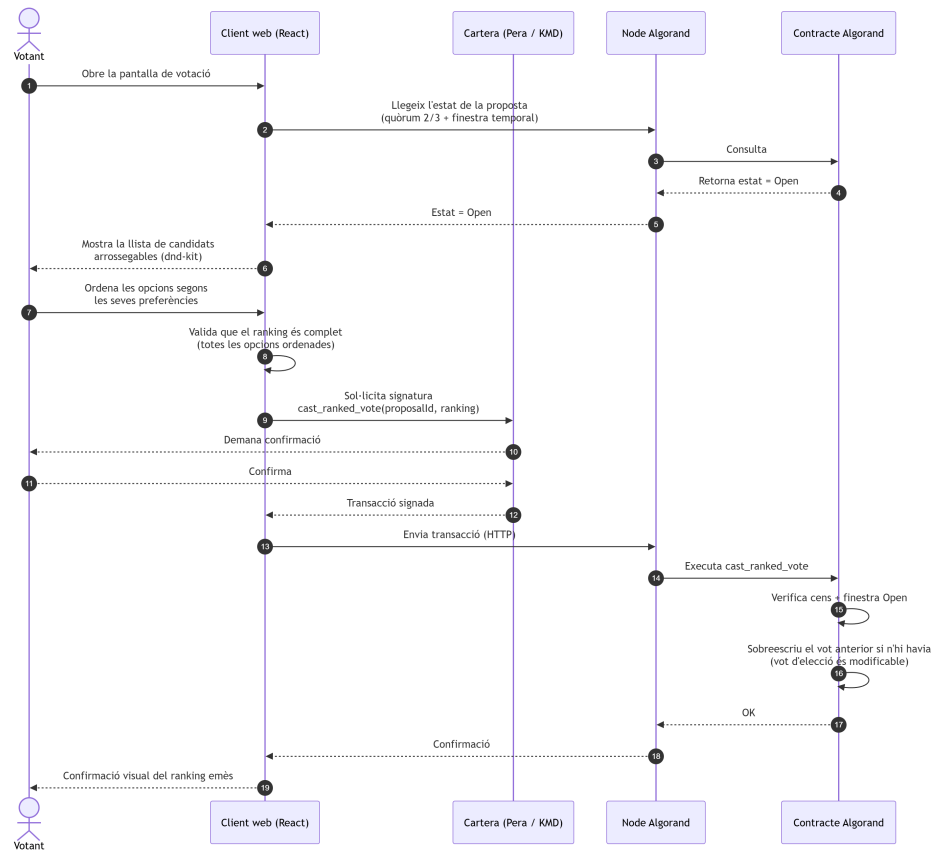


Figura 5.10: Diagrama de seqüència: vot d'elecció amb ranking.

5.5 Flux autònom i automàtic

5.5.1 Ancoratge a Ethereum

Aquest flux s'executa automàticament i paral·lelament a cada node un cop una elecció es tanca. Tots els nodes llegeixen les paperetes del contracte d'Algorand, en deriven independentment el mateix *hash* determinista i l'envien al *NotaryContract* d'Ethereum. El contracte acumula els enviaments fins que K nodes coincideixen en el mateix *hash*; en aquell moment registra l'elecció com a ancorada de manera permanent i emet un esdeveniment públic.

Aquest disseny garanteix dues propietats essencials. Primer, que cap node individual pugui falsejar el resultat: necessita la coincidència de K -of- N . Segon, que el resultat sigui verificable sense necessitat de confiar en cap entitat central: qualsevol persona pot consultar el contracte i comprovar quins nodes han signat, què han signat i quan.

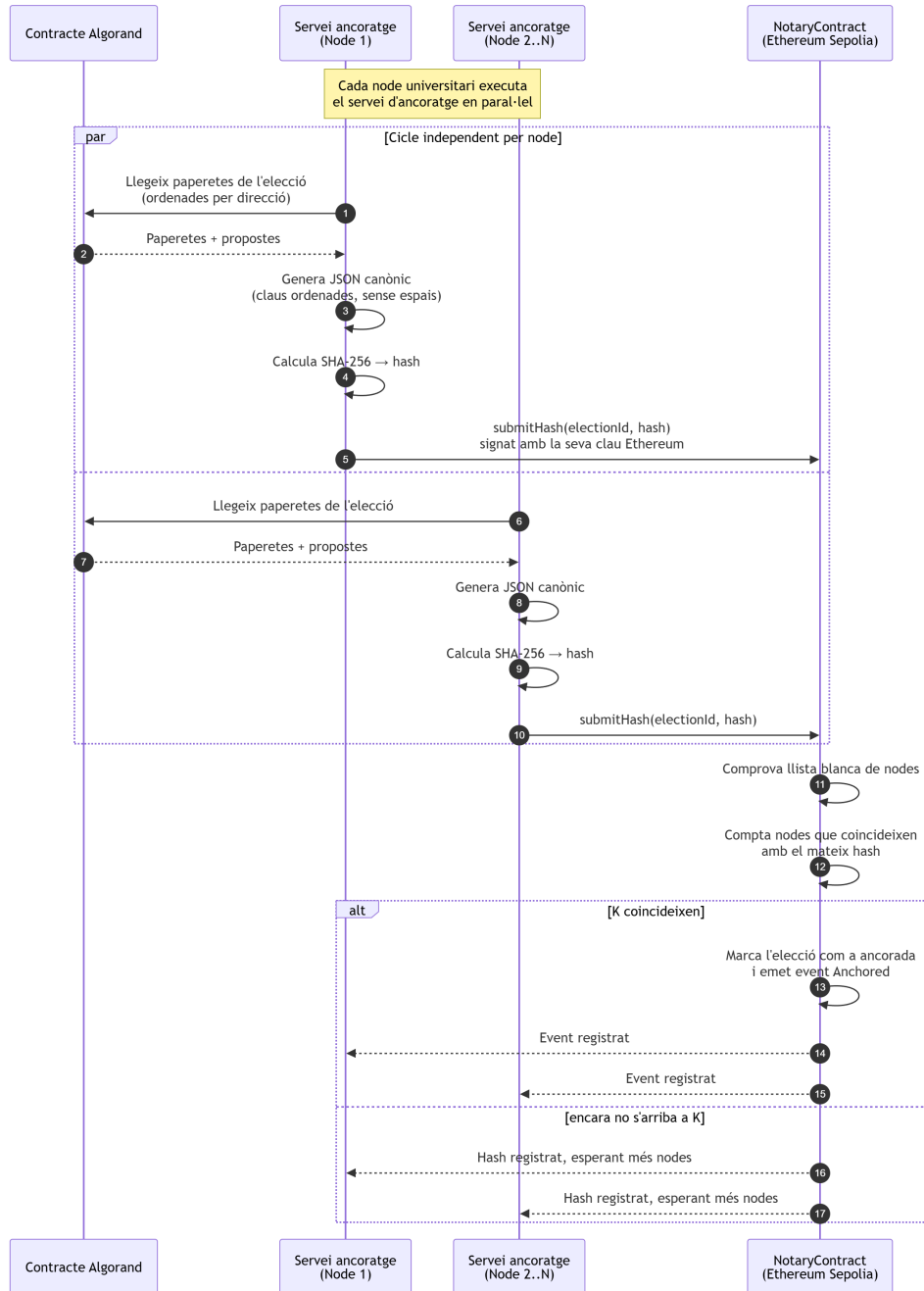


Figura 5.11: Diagrama de seqüència: ancoratge a Ethereum.

DESENVOLUPAMENT

Aquest capítol descriu com s'ha executat el projecte durant la seva fase de construcció. Mentre que el capítol 3 ha exposat la *planificació* (cicle de vida, jerarquia del *backlog*, Definition of Ready i Definition of Done, política de versionat), aquí es relata el desenvolupament real: com s'ha organitzat l'equip dia a dia, com s'ha materialitzat la jerarquia del *backlog* en *issues* concretes, què ha entregat cada *sprint*, quines versions intermèdies s'han generat, quins problemes s'han trobat i com s'han demostrat les funcionalitats. L'objectiu és deixar constància fidel de *què* ha passat, no del *que s'havia previst*.

6.1 Organització de l'equip

L'equip està format per quatre estudiants del grau en Enginyeria Informàtica de la Universitat de les Illes Balears (UIB), sense experiència prèvia en *blockchain* ni en criptografia aplicada (vegeu secció 1.1 del capítol 1). Aquesta homogeneïtat de perfils ha facilitat la presa de decisions col·lectiva i la revisió creuada del codi, però ha obligat l'equip a adquirir des de zero el coneixement tècnic necessari (vegeu secció 6.5).

6.1.1 Rols i responsabilitats

Cada membre de l'equip ha liderat una àrea funcional del producte durant tot el projecte, mantenint la mateixa responsabilitat des de la fase de recerca fins a l'entrega final. Aquesta estabilitat ha permès acumular coneixement profund sobre cada paquet de treball i evitar el cost de canvis freqüents de context. La taula 6.1 sintetitza els rols.

Aquesta distribució per àrees no ha estat estricta. La regla de protecció de main –que impedeix fer *merge* de la mateixa PR– ha forçat que cada membre revisi codi de les altres àrees i, en conseqüència, hi adquireixi un coneixement funcional bàsic. Les decisions arquitectòniques rellevants s'han pres en reunions setmanals conjuntes, inicialment presencials i progressivament híbrides.

Membre	Àrea principal	Responsabilitats concretes
Jordi Vanyó Mestre	Client web i integració Algorand	Interfície React, integració amb <code>algosdk</code> , codificació i descodificació ARC-4 , lectura de <i>boxes</i> , signatura de transaccions des del navegador.
Dylan Luigi Canning Garcia	Ancoratge i <i>NotaryContract</i>	Servei Python d'ancoratge (<code>network/anchoring/</code>), <i>hash</i> determinista (JSON canònic + SHA-256), submissió al contracte d'Ethereum, contracte <i>NotaryContract</i> a Sepolia.
Antonio Contestí Coll	Coordinació i documentació	Gestió del <i>backlog</i> a GitHub Projects, seguiment de <i>sprints</i> , coherència de la documentació tècnica, redacció de la memòria.
Marc Link Cladera	<i>Smart contract</i> Algorand i arquitectura	Disseny i implementació del contracte <i>Demochain</i> en <i>algopy</i> , diagrames C4 [1], redacció dels ADRs .

Taula 6.1: Distribució de rols i responsabilitats durant el desenvolupament. Cap rol ha estat exclusiu: totes les **PRs** han passat per revisió creuada (vegeu secció 3.7 del capítol 3).

6.1.2 Dinàmica de treball

L'equip ha funcionat amb una cadència setmanal estable. Cada setmana s'ha celebrat una reunió de coordinació amb tres punts fixos: revisió del que s'ha fet, identificació de bloqueigs i planificació de la setmana següent. A més, dins de cada *sprint* s'ha celebrat una *retrospectiva* curta per identificar millores de procés. Les decisions de caràcter tècnic s'han documentat als **ADRs** corresponents (**ADR 000** sobre *algopy* i **ADR 001** sobre la plataforma *blockchain*), de manera que la discussió no s'ha hagut de repetir.

6.2 Organització del backlog

El *backlog* del projecte s'ha gestionat amb **GitHub Projects** sobre el repositori *demochain* [17], seguint l'estructura jeràrquica de cinc nivells descrita a la secció 3.2.1 del capítol 3. Aquesta secció n'il·lustra l'aplicació real amb exemples concrets del repositori.

6.2.1 Materialització de la jerarquia

La cadena èpica → **història d'usuari** / *spike* → **tasca** s'ha aplicat de manera consistent durant tot el projecte. La figura conceptual següent mostra un exemple complet extret del *backlog*:

- **Èpica E2 – Propostes i aprovació** (*issue* #5). Conjunt de funcionalitats per a la creació, validació i aprovació de propostes per quòrum de dos terços.

- **Història d'usuari E2-US1** – *Com a membre del cens vull crear una proposta dins de la meua organització.* Inclou els criteris d'acceptació en format *Donat/Quan/Aleshores* sobre validacions de dates, opcions mínimes i pertinença al cens.
 - * **Tasca E2-US1-T3** – Implementació del mètode `create_proposal` al contracte amb validació de la finestra temporal mínima (3 dies abans de l'inici, 1 dia mínim de votació) i del nombre d'opcions (≥ 2).
- **Spike E1-SP1** – *Quin llenguatge cal triar per al contracte d'Algorand: TEAL, PyTEAL o algopy?* El resultat d'aquest *spike* s'ha materialitzat al document [ADR 000](#) (`docs/decisions/000-smart-contract-language.md`).

A cada *issue* se li ha assignat, a més del tipus (`type/epic`, `type/user-story`, `type/spike`, `type/task`, `type/subtask`), una etiqueta d'àrea funcional: `area/smart-contract`, `area/front-end`, `area/network`, `area/census`, `area/wallet`, `area/test`, `area/research`, `area/docs` i `area/devops`. Aquesta combinació de tipus i àrea ha permès filtrar el *backlog* segons diferents perspectives durant les reunions de planificació.

6.2.2 Volum i estat final del backlog

A l'entrega final, el *backlog* comprèn **nou èpiques** i més de **cent issues** traçades, tal com es detalla a la taula [3.1](#) del capítol [3](#). La majoria de les èpiques estan tancades; només dues queden obertes explícitament:

- **E5 – Elegibilitat de votants amb Zero-Knowledge (ZK)** (*issue* #340), que depèn íntegrament de la capa criptogràfica no implementada.
- **E11 – Setup del projecte i del repositori** (*issue* #402), oberta com a *epic* contenidor de tasques de manteniment continu del repositori.

Els tres últims *sprints* es registren formalment com a *milestones* a GitHub, amb les *issues* corresponents assignades a cadascun. Una de les retrospectives ha identificat que els tres primers *sprints* no es van classificar de manera estricta i caldria assignar-hi els *issues* corresponents per a la traçabilitat completa. Aquesta tasca queda recollida com a punt pendent a l'*epic* E11.

6.3 Resum de les iteracions de desenvolupament

El desenvolupament s'ha estructurat en sis iteracions (*sprints*) de dues setmanes. Aquesta secció en relata l'execució real, complementant la planificació exposada a la secció [3.4](#) del capítol [3](#).

6.3.1 Sprint 1 – Recerca preliminar (febrer-març)

Iteració íntegrament dedicada a la fase de recerca. L'equip ha estudiat el funcionament d'Algorand [\[2, 25\]](#), les alternatives a Ethereum [\[5\]](#) i els mètodes de votació ordinal. S'ha aixecat un primer *LocalNet* d'Algorand i s'ha provat el desplegament d'un

contracte mínim per familiaritzar-se amb la cadena d'eines. **Entrega:** primera versió del repositori amb prova de concepte sobre LocalNet i tres documents de recerca (docs/research/algorand.md, ethereum.md, voting-methods-comparison.md).

6.3.2 Sprint 2 – Decisions arquitectòniques (març)

S'han fixat les decisions tècniques clau del projecte: tria del llenguatge del contracte (*algopy* amb compilador Puya [3], formalitzada a l'ADR 000) i tria de la plataforma *blockchain* (Algorand permissionada amb ancoratge a Ethereum, formalitzada a l'ADR 001). Paral·lelament, s'han redactat els primers diagrames C4 [1] i s'ha seleccionat el mètode de Schulze [8] com a algorisme de recompte, amb el corresponent document de recerca (docs/research/schulze.md). **Entrega:** primer esborrany del contracte *algopy* amb les operacions bàsiques d'organització i diagrames C4 a nivells 1 i 2.

6.3.3 Sprint 3 – Primer prototip de votació (març – 12 abril)

Implementació del primer prototip funcional. Al contracte d'Algorand s'hi ha afegit la lògica de propostes (creació, validació de dates i opcions) i una versió inicial de votació per pluralitat. Al client web s'ha construït el *mockup* de les pantalles principals amb React i *TailwindCSS*, i s'ha configurat el primer `docker-compose.yml` amb *algod*, *indexer* i client. **Entrega intermèdia** a la convocatòria del 12 d'abril.

Aquest *sprint* acaba amb la *sessió d'estimació realista* en què l'equip pren la decisió de **reduir l'abast del MVP** i deixar la criptografia avançada com a treball futur (vegeu secció 6.5 i secció 3.8 del capítol 3).

6.3.4 Sprint 4 – Schulze, resultats i integració (17–30 abril)

Aquest *sprint* consolida la presència del repositori actual i incorpora la majoria del codi del client. S'implementa el mòdul de Schulze en TypeScript sobre la matriu de comparació per parelles (*issues* #132 i #133) amb l'algorisme de Floyd-Warshall [24], la pàgina de resultats amb guanyador i *rànquing* (#136), les matrius de comparació per parelles col·lapsables (#139), la targeta «*El meu vot*» al detall de la proposta (#144) i el panell de verificació pública del vot (#145). **Entrega:** versió integrada client + contracte capaç d'executar el cicle complet d'una elecció sobre LocalNet.

6.3.5 Sprint 5 – Testing, refinament i robustesa (1–14 maig)

Sprint orientat a la qualitat i estabilitat. S'incorporen cobertures de proves unitàries del domini de propostes (#458), tests dels *mappers* de consultes Algorand (#457) i tests d'integració del *governance client* contra LocalNet (#456). Es corregeix el *bug* d'accés a transaccions via Lora (#463) i es refina la pantalla de votació amb edició d'hora i minuts mitjançant teclat (#470). **Entrega:** cinc fitxers de test unitari al contracte (voting-contract/tests/: `approval_unit_test.py`, `cast_election_vote_unit_test.py`, `census_unit_test.py`, `organization_unit_test.py`, `proposal_unit_test.py`) i tests al client.

6.3.6 Sprint 6 – Documentació, demo i entrega (15–25 maig)

Sprint final dedicat a redacció de la memòria, preparació de la presentació oral, retrospectiva i empaquetament. S'han generat els diagrames C4 a nivell 3 (un per contenidor) i els diagrames de seqüència Mermaid dels fluxos principals. **Entrega:** versió v1.0.0 del producte i memòria final.

6.4 Versions generades i entregades

A banda de la versió final del producte, durant tot el projecte s'han generat entregables intermedis amb cadència setmanal. Aquesta secció n'inventaria els més rellevants, distingint entre entregables de recerca, entregables d'enginyeria i la versió final del producte.

6.4.1 Recerca obtinguda setmanalment

Durant tot el projecte, l'equip ha mantingut una pràctica setmanal de documentar el coneixement adquirit. Aquesta pràctica ha estat especialment intensa durant la fase de recerca (*sprints* 1 i 2), però s'ha mantingut tot al llarg del projecte cada vegada que ha sorgit un *spike* de recerca. Els resultats es materialitzen en dos llocs del repositori:

- **docs/research/**: documents tècnics amb les conclusions d'estudis comparatius. A l'entrega final n'hi ha quatre:
 - `algorand.md` – Estudi en profunditat d'Algorand: model de consens, [AVM](#), [boxes](#), [ARC-4](#) [26] i [ARC-56](#) [27].
 - `ethereum.md` – Anàlisi d'Ethereum com a opció alternativa i com a destinació de l'ancoratge, incloent-hi Ethereum Improvement Proposal ([EIP](#))-1559 [32].
 - `voting-methods-comparison.md` – Comparativa de mètodes de votació ordinal (Borda, Condorcet, IRV, Schulze) amb les propietats formals respectives.
 - `schulze.md` – Estudi en detall del mètode de Schulze [8] i del seu càlcul amb Floyd-Warshall [24].
- **docs/decisions/**: Architecture Decision Records amb les decisions arquitectòniques formals derivades dels estudis anteriors:
 - [ADR 000](#) – Tria del llenguatge de *smart contract* (`000-smart-contract-language.md`).
 - [ADR 001](#) – Tria de la plataforma *blockchain* (`001-blockchain-choice.md`).

Aquesta pràctica ha resultat especialment valuosa: cada document de recerca i cada [ADR](#) ha servit per evitar que la mateixa discussió es repetís en *sprints* posteriors i ha proporcionat justificació formal per a les decisions de disseny.

6.4.2 Versions intermèdies del producte

Pel que fa al producte de *software*, s'han generat dues versions significatives intermèdies abans del lliurament final:

- **Prototip Sprint 3** (12 d'abril): *mockup* del client i contracte amb votació per pluralitat. Entrega intermèdia formal a la convocatòria del 12 d'abril.
- **Integració Sprint 4** (30 d'abril): primera versió capaç d'executar el cicle complet d'una elecció (organització, proposta, aprovació, votació preferencial, recompte Schulze) sobre LocalNet.

6.4.3 Versió final entregada

La versió final entregada és la v1.0.0, generada al final del Sprint 6. Comprèn:

- *Smart contract* d'Algorand Demochain en *algopy* (voting-contract/).
- *Smart contract* d'Ethereum NotaryContract en Solidity (network/ethereum/).
- Servei d'ancoratge en Python (network/anchoring/).
- Client web React 19 + TypeScript (client/).
- Infraestructura Docker Compose (docker-compose.yml, conduit.yml).
- Documentació completa: docs/ amb subdirectoris research/, decisions/, diagrams/ i memory/.

6.5 Problemes trobats i la seva resolució

Durant el desenvolupament s'han trobat diversos obstacles. Aquesta secció recull els dos més rellevants pel seu impacte sobre l'abast i sobre la planificació, ja que la majoria d'incidències tècniques puntuals queden documentades a les *issues* de GitHub i als *pull requests* corresponents.

6.5.1 Criptografia avançada fora de l'abast de l'MVP

El problema de planificació més important del projecte ha estat la constatació que la **capa de criptografia avançada** prevista inicialment no era assolible dins l'horitzó temporal del Treball de Fi de Grau (TFG). L'arquitectura inicial preveia:

- Vots xifrats amb ElGamal exponencial homomòrfic.
- Proves **ZK-SNARKs** Groth16 [16, 11] per acreditar l'elegibilitat del votant sense revelar-ne la identitat.
- Cerimònia de generació distribuïda de claus (DKG) i desxifrat amb *trustees* amb proves Chaum-Pedersen.
- Integració amb un **IdP** extern (Keycloak) per a l'autenticació prèvia del votant.

Detecció del problema. La detecció es va produir al final del Sprint 3, en una sessió d'estimació realista en què es va quantificar el cost d'implementar cada primitiva. Les estimacions indicaven que sense un mes addicional de recerca específica per primitiva, l'equip no tenia les bases per construir-les amb les garanties necessàries (vegeu secció 3.8 del capítol 3).

Resolució adoptada. La decisió, presa en grup i documentada a les *issues* corresponents, va ser:

1. Reduir l'**MVP** a votació **en clar** amb verificació pública via consulta directa a la cadena Algorand i ancoratge *K-of-N* a Ethereum.
2. Tancar les èpiques E3, E4 i E6 amb el subconjunt de funcionalitats efectivament implementades (votació preferencial, recompte Schulze, connexió de *wallet*) i deixar la resta d'**USs** obertes com a treball futur explícit.
3. Mantenir oberta l'èpica E5 sencera, ja que tota la seva funcionalitat depèn de la capa criptogràfica.

Lliçó apres. Vista en retrospectiva, la decisió és coherent amb les recomanacions de la literatura: davant un objectiu ambiciós i un horitzó tancat, és preferible entregar un subconjunt funcional i sòlid que un superconjunt incomplet o inestable. La criptografia avançada queda identificada explícitament al capítol de conclusions com a línia de treball futura.

6.5.2 Falta de coneixement inicial

Cap membre de l'equip tenia experiència prèvia en *blockchain* ni en criptografia aplicada. Aquesta absència de base ha estat el segon problema rellevant del projecte: cap línia de codi del contracte podia escriure's abans d'entendre el model d'execució de l'**AVM**, l'estructura de *boxes* d'emmagatzematge, el sistema de tipus d'*algopy* i les convencions **ARC-4** [26].

Resolució adoptada. Es va adoptar una **fase de recerca i fonaments explícita** (febrer-abril de 2026), durant la qual no es va comprometre cap entrega de producte. La fase va consistir en:

- Lectura sistemàtica de la documentació oficial d'Algorand, AlgoKit [25] i Puya [3].
- Redacció dels documents de recerca a docs/research/ com a mecanisme de consolidació del coneixement.
- Formalització de les decisions clau als **ADRs** 000 i 001, perquè el coneixement adquirit quedés codificat i no s'hagués de tornar a discutir.
- Provatura pràctica: aixecament d'un *LocalNet* d'Algorand i desplegament d'un contracte mínim per familiaritzar-se amb la cadena d'eines.

Resultat. El cost d'oportunitat d'aquesta fase ha estat alt –un mes sense codi de producte entregat– però en retrospectiva ha resultat la decisió més encertada del projecte: el contracte que es va escriure després estava lliure d'errors conceptuals que altrament haurien requerit reescriptures costoses.

6.6 Demostracions funcionals

Cada *sprint* ha culminat amb una demostració funcional interna de les funcionalitats incorporades. Aquesta secció en sintetitza l'evolució i descriu la demostració de la versió final.

6.6.1 Demostracions intermèdies

- **Sprint 3 (12 abril).** Demostració del prototip de votació per pluralitat: creació d'organització, creació de proposta, votació simple i visualització del resultat sobre el *mockup* del client.
- **Sprint 4 (30 abril).** Demostració del cicle complet sobre LocalNet: creació d'organització amb cens, redacció d'una proposta, fase d'aprovació per quòrum, votació preferencial amb *rànquing* arrossegable, càlcul del guanyador amb Schulze i visualització de la matriu de comparació per parelles.
- **Sprint 5 (14 maig).** Demostració de la robustesa: execució de tota la bateria de tests del contracte, càrrega massiva de cens via **CSV** amb trossejament automàtic en lots de set adreces, panell de verificació pública del vot per a una adreça arbitrària, i correcció del *bug* d'accés a transaccions via Lora.

6.6.2 Demostració de la versió final

La demostració de la versió v1.0.0 a l'entrega final cobreix el cicle complet d'una elecció sobre l'entorn local i és **reproducible** per qualsevol persona amb el repositori i Docker.

Preparació de l'entorn. Una sola comanda aixeca tota la infraestructura: `docker compose up`. Això arrenca seqüencialment el node Algorand (*lead* i *follower*), el desplegament automàtic del contracte (que persisteix l'`APP_ID` a `client/.env.local`), la base de dades PostgreSQL, Conduit, l'*indexer*, el node Ethereum local (Hardhat) amb el desplegament del NotaryContract, el servei d'ancoratge i el client web servit per Vite (vegeu secció 3.1 del capítol 3 i detalls a la secció corresponent de Disseny).

Recorregut funcional. La demostració recorre les pantalles principals del client (vegeu les figures de la secció de disseny):

1. Connexió de cartera (Pera Wallet o Lute).
2. Creació d'una organització i càrrega de cens via **CSV** (amb trossejament automàtic en lots de set adreces).

3. Redacció d'una proposta mitjançant l'assistent de cinc passos.
4. Vot d'aprovació binari per part dels membres del cens.
5. Vot d'elecció amb *rànding* arrossegable (component @dnd-kit, accessible amb ratolí i teclat).
6. Visualització dels resultats finals amb el mètode de Schulze, la matriu de comparació per parelles col·lapsable i el panell de verificació pública del vot.

Verificabilitat . La demostració inclou la verificació externa del resultat: introduint una adreça arbitrària al panell de verificació, el client consulta directament la cadena i mostra el *rànding* emès per aquesta adreça, sense necessitat de cap permís. En paral·lel, el servei d'ancoratge publica el *hash* determinista de l'elecció tancada al contracte `NotaryContract` desplegat al node Ethereum del *LocalNet*, fent visible el consens *K-of-N* entre nodes universitaris (en producció, el mateix flux apunta a Sepolia).

Temps total . Sobre el *LocalNet*, el cicle complet –des de `docker compose up` fins a la visualització dels resultats– es pot executar en menys de **cinc minuts**, fet que n'ha facilitat l'ús durant les demostracions de cada *sprint* i la repetició davant un públic acadèmic a l'entrega final.

CONCLUSIONS

Aquest capítol tanca la memòria valorant el que s'ha entregat respecte al que s'havia plantejat al començament del projecte. La primera part revisa el producte final –el que s'ha fet, el que s'ha decidit no fer, el que queda per fer més endavant i les condicions de posada en producció i manteniment. La segona part és una reflexió de l'equip sobre el procés: què ha funcionat, què no, com s'ha treballat conjuntament i quin valor real aporta el resultat a les diferents parts interessades.

7.1 Producte final entregat

7.1.1 Què s'ha fet

El projecte entrega un Minimum Viable Product (**MVP**) **funcional i demostrable** d'un sistema de votació descentralitzada amb les peces següents:

- **Smart contract d'Algorand** escrit en *algopy* [3], amb gestió d'organitzacions (creació, cens d'adreces autoritzades amb càrrega massiva que es gestiona realitzant grup de transaccions o cada transacció carrega set adreces), propostes amb fase d'aprovació per quòrum de dos terços, votació preferencial amb *rànquings* complets, i prevenció de doble vot.
- **Smart contract d'Ethereum** (NotaryContract) en Solidity, desplegable a la *test-net* Sepolia, que implementa un protocol de consens *K-of-N* entre nodes universitaris. La llista de nodes autoritzats i el llindar K es congelen en obrir una elecció, garantint que cap canvi posterior pugui alterar els paràmetres de validació.
- **Sistema d'ancoratge** en Python que llegeix l'estat final d'una elecció des d'Algorand, produeix un *hash* determinista (**JSON** canònic + **SHA**-256) i el publica al contracte notari d'Ethereum. La determinació de l'ordre dels vots per direcció lexicogràfica garanteix que tots els nodes generen exactament el mateix *hash* si no hi ha manipulació.

- **Client web** amb React 19 i TypeScript, amb interfície completa per a la creació d'organitzacions, càrrega de cens via **CSV**, gestió de propostes, votació preferencial amb reordenació per arrossegament, recompte amb el mètode de Schulze [8] computat localment a partir de les paperetes individuals –el contracte d'Algorand no emmagatzema cap resultat precalculat, només els vots, fet que reforça l'argument de verificabilitat pública–, i panell de verificació pública que permet consultar el vot emès per qualsevol adreça.
- **Infraestructura local reproducible** amb Docker Compose: un únic docker compose up aixeca una *LocalNet* d'Algorand amb dos nodes (*lead* i *follower*), l'*Indexer* amb la seva base de dades PostgreSQL, el servei Conduit, compila i desplega el contracte, i serveix el client web a través de Vite.
- **Pràctiques d'enginyeria de software** consolidades: *backlog* jerarquitzat amb nou èpiques i més de 100 *issues* traçades a GitHub Projects, control de versions amb *feature branches* i revisió creuada obligatòria, *Conventional Commits* [7] validats automàticament tant en local com en Continuous Integration (**CI**), dos Architecture Decision Records formals i diagrames C4 [1].

El sistema permet executar el cicle complet d'una elecció –crear una organització, carregar un cens, redactar una proposta, obtenir-ne l'aprovació per quòrum, votar preferencialment, calcular el guanyador amb Schulze i ancorar el resultat a Ethereum– en menys de cinc minuts sobre l'entorn de desenvolupament local.

7.1.2 Què s'ha decidit no fer (i no es farà)

Una de les decisions arquitectòniques que ha pres l'equip mereix una explicació explícita perquè condiciona l'evolució futura del projecte: **els nodes de la xarxa permissionada no seran els mateixos votants**. L'arquitectura final assumeix que els nodes Algorand estan operats per entitats institucionals –universitats, entitats cíviques, col·legis professionals– i que els votants accedeixen al sistema com a usuaris finals d'un client que es connecta a aquests nodes a través d'una **API**.

Aquesta decisió s'ha pres pels motius següents:

- Operar un node Algorand requereix recursos de maquinari, coneixement tècnic i una amplada de banda constant. Esperar que cada votant operi un node és tècnicament irreal i pràcticament exclouent per a usuaris no tècnics.
- El model de nodes institucionals s'alinea amb el cas d'ús natural del projecte: cada universitat opera un node, contribueix a la xarxa permissionada i participa en el consens *K-of-N* d'ancoratge.
- La distribució de confiança ja queda garantida pel fet que múltiples entitats independents operen els nodes; no cal que cada votant en sigui un per assolir el mateix grau de descentralització.

Aquesta orientació diferencia clarament el projecte de plataformes com DAVINCI [9], on l'usuari final pot participar activament a la xarxa de *sequencers*. Optar per nodes institucionals és una elecció conscient: aposta per la verificabilitat pública sense exigir una participació activa a la infraestructura per part del votant.

7.1.3 Què no s'ha fet, (però es farà més endavant)

Les funcionalitats següents queden identificades explícitament com a treball futur. No s'han abandonat: són part del projecte a llarg termini i corresponen a èpiques del *backlog* que es van prioritzar a la fase de planificació, però que s'han hagut de moure fora de l'abast de l'**MVP**.

Criptografia avançada del vot . Incorporar xifratge homomòrfic ElGamal exponencial [10, 11] i proves *Groth16* de coneixement zero [12, 13] per a la verificació de l'elegibilitat sense revelar la identitat del votant. Aquesta línia està documentada amb detall a les èpiques E3, E4 i E5 del repositori [17] i requereix una dedicació estimada de diversos mesos addicionals per ser duta a producció amb les garanties necessàries.

Cerimònia distribuïda de desxifrat amb *trustees* . Una vegada introduït el xifratge, caldrà implementar el protocol de generació distribuïda de claus (DKG) [14] i de desxifrat en fases amb proves Chaum-Pedersen [15] per evitar que cap *trustee* individual pugui revelar els vots. Aquesta funcionalitat correspon a la **US E4-US3** (*Cerimònia de desxifrat dels trustees*), encara oberta.

Integració amb un Identity Provider (IdP) extern . L'autenticació del votant, via un proveïdor d'identitat (per exemple Keycloak, o el directori d'identitats d'una universitat), i la derivació determinista de la clau de votació Ed25519 a partir de les credencials del votant és l'objectiu de l'èpica E6, amb les històries d'usuari E6-**US1** i E6-**US2** encara obertes al repositori.

Mecanismes de resistència a la coacció(*receipt-freeness*) . Impedir que el votant pugui demostrar a un tercer que ha votat, una propietat fonamental per prevenir compra de vots i coerció [23]. Aquest mecanisme està documentat a la **US E3-US4** del repositori [17] i està implementat al *state-of-the-art* per sistemes com DAVINCI; és imprescindible per a desplegaments amb riscos socials reals.

7.1.4 Posada en producció

La transició de l'**MVP** actual a un sistema desplegable per una universitat real requereix una sèrie de passos addicionals:

1. **Operació d'un node Algorand permissionat per cadascuna de les universitats participants.** Cada entitat hauria d'aixecar un node Algorand connectat a la xarxa permissionada del consorci, amb la configuració de seguretat adequada (TLS, *firewalling*, monitoratge, còpies de seguretat).
2. **Servei d'ancoratge dedicat per node.** Cada node universitari hauria d'operar el seu propi servei d'ancoratge amb una clau privada d'Ethereum donada d'alta a la *whitelist* del contracte notari. El servei hauria de córrer com a *daemon* de sistema (`systemd`) amb reinici automàtic, monitoratge del cost de gas i alertes operatives.

3. **Desplegament estable a Ethereum.** Per a un desplegament real, el contracte `NotaryContract` hauria de passar de la *testnet* Sepolia a la *mainnet* d'Ethereum. Això implica costos reals de *gas*, una auditoria de seguretat del contracte i una gestió curiosa de les claus dels administradors.
4. **Distribució del client web als votants.** Cada votant es descarrega el codi obert del client des del repositori públic [17] i l'executa localment a la seva pròpia màquina. Aquesta decisió reforça la verificabilitat –qualsevol pot inspeccionar el codi que executa– i elimina un punt central de compromís en què un tercer podria servir una versió alterada del client a un subconjunt de votants. Per a producció caldria, doncs, publicar versions *tagged* i reproduïbles del client, amb instruccions clares per a la construcció local (`bun install + bun run dev` o `bun run build`) i, opcionalment, *builds* verificables firmats criptogràficament pels mantenidors.
5. **Procediments operatius compartits.** Documentació operativa: com es coordinen les universitats per actualitzar el contracte, què es fa si un node es perd, com es porten a terme les incorporacions i baixes de membres del consorci.

7.1.5 Manteniment

El manteniment a llarg termini del sistema cobreix tres àmbits diferenciats:

Manteniment del codi . Actualitzacions periòdiques de les dependències (`algopy`, `react`, `web3.py`, `algosdk`) per incorporar correccions de seguretat i compatibilitat. La cobertura de tests automàtics existent (*p. ex.* els cinc fitxers de test unitari del contracte i els tests del client) ha de mantenir-se i estendre's.

Auditoria de seguretat del contracte . Els *smart contracts* d'Algorand i d'Ethereum són essencialment immutables un cop desplegats. Qualsevol incidència crítica requereix migrar a una nova versió i fer una transició coordinada. És imprescindible sotmetre el contracte d'Algorand a una auditoria de seguretat externa abans d'un desplegament real, especialment quan s'incorporin les capes criptogràfiques.

Monitoratge operatiu . En producció, l'estat de la xarxa Algorand, el servei d'ancoratge i el client web s'haurien de monitorar amb eines estàndard (Prometheus, Grafana) i amb alertes configurades sobre indicadors crítics: lag dels nodes, fallades en les submissions a Ethereum, despeses anòmales de *gas*.

7.2 Lliçons apreses

Aquesta secció recull la reflexió de l'equip sobre el procés seguit. No pretén ser exhaustiva sinó assenyalar els punts que, en retrospectiva, han marcat de manera més clara el desenvolupament del projecte.

7.2.1 Què ha funcionat

La inversió en recerca prèvia . Dedicar el primer mes gairebé en exclusiva a entendre l'ecosistema Algorand, el llenguatge *algopy* i els mètodes de votació ordinal va semblar lent mentre durava. En retrospectiva, ha estat la decisió més encertada del projecte: el contracte que es va escriure després estava lliure d'errors conceptuals que altrament haurien requerit reescriptures costoses. Els dos **ADRs** formalitzats al final d'aquesta fase (Architecture Decision Record 000 sobre *algopy* i Architecture Decision Record 001 sobre la plataforma) van fixar el rumb i van evitar discussions repetides.

L'stack d'enginyeria del software . Les pràctiques adoptades –*Conventional Commits* [7], *feature branches*, *pull requests* amb revisió creuada obligatòria, GitHub Actions, *git hooks* locals que autoformaten el codi– han creat un entorn de treball en què la qualitat del codi s'ha mantingut alta amb molt poc esforç conscient. En particular, la regla de protecció de `main` que impedeix fer *merge* la mateixa **PR** ha forçat de manera natural la revisió creuada entre membres de l'equip.

Docker Compose com a punt de pivot . Tenir un fitxer `docker-compose.yml` que aixecava tot el sistema (*LocalNet* Algorand, Indexer, Conduit, base de dades, client web, desplegament del contracte) amb una sola comanda ha estat decisiu. Cada membre de l'equip podia provar canvis dels altres en pocs minuts, i la incorporació de noves dependències no requeria documentació manual: era automàtica.

Diagrames C4 mantinguts al repositori . Els diagrames a tres nivells (context, contenidors, components per a cada peça) han servit com a llenguatge comú per a les decisions arquitectòniques. Quan un membre nou ha de tocar una àrea que no és la seva, els diagrames li donen l'orientació mínima per saber on i què mirar.

7.2.2 Què no ha funcionat

L'ambició criptogràfica inicial . Plantejar la criptografia avançada (ElGamal homomòrfic + **ZK**-SNARK + *trustees*) al mateix temps que tots els altres components del sistema va resultar inviable. Sense un mes addicional de recerca específica sobre cada una d'aquestes primitives, l'equip no tenia les bases per implementar-les amb correcció. La decisió de reduir l'abast a una votació en clar amb verificabilitat pública (vegeu secció 3.8 del capítol 3) va ser dolorosa però necessària. La lliçó és coneguda, però val la pena recordar-la: *millor entregar menys, però sòlid, que més i trencat*.

La gestió del temps en paral·lel amb la resta del grau . Tot l'equip combinava el projecte amb altres assignatures. Els *sprints* de dues setmanes han resultat, en pràctica, més curts del que sembla quan es resten les setmanes amb càrrega laboral o el treball final de grau. Una planificació amb *sprints* de tres setmanes hauria estat probablement més realista.

7.2.3 El funcionament com a equip

L'equip ha funcionat de manera notablement cohesionada. Quatre membres amb perfils similars (frau d'enginyeria Informàtica, sense experiència prèvia en *blockchain*) i una distribució natural de responsabilitats –un per àrea– ha permès paral·lelitzar el treball sense generar conflictes d'autoria. Les reunions setmanals, inicialment presencials i progressivament híbrides, han servit tant per a la coordinació tècnica com per a la presa de decisions col·lectiva.

Cap decisió rellevant s'ha pres unilateralment. Els canvis d'abast, la reorganització de *sprints* i la incorporació o eliminació d'eines tècniques s'han discutit en grup i s'han documentat posteriorment (al *backlog*, als **ADRs** o en aquesta memòria). Aquest patró ha estat un dels actius més clars del projecte i s'hauria de mantenir si l'equip continua treballant en versions posteriors del sistema.

La revisió creuada de **PRs**, forçada per la configuració del repositori, ha contribuït a una comprensió compartida del codi: encara que cada membre lideri una àrea, tots tenen un coneixement funcional bàsic de les altres tres. Aquesta propietat és especialment valuosa per a la continuïtat del projecte.

7.2.4 El producte: orgull i autocrítica

Globalment, l'equip considera que el producte entregat compleix amb suficiència els objectius plantejats al capítol 1. El sistema funciona, és reproduïble, està documentat amb rigor i les decisions arquitectòniques són defensables. Les renúncies fetes al llarg del projecte –notablement la criptografia avançada– s'han pres amb consciència i s'han documentat amb transparència, sense ocultar-les ni dissimular-les.

L'autocrítica principal és que el sistema, en la seva forma actual, encara no està preparat per un desplegament en un context amb riscos socials reals. Sense la capa de privacitat criptogràfica i sense una auditoria de seguretat externa del contracte, recomanar el sistema per a unes eleccions serioses seria irresponsable. Aquesta limitació no és un defecte amagat sinó una propietat coneguda de l'**MVP**: el producte actual és, com el seu nom indica, una *prova de viabilitat*, no un sistema productiu.

7.3 Aportació a les parts interessades

Finalment, val la pena articular què aporta el projecte a cada una de les parts interessades identificades al començament:

A les universitats i entitats educatives . Una base de codi oberta, documentada i reproduïble que es pot avaluar com a punt de partida per a un sistema propi de votació interna (eleccions de rector, claustre, representació estudiantil). El sistema actual no és apte per a aquest desplegament sense la capa criptogràfica, però la infraestructura –Algorand permissionat + ancoratge a Ethereum + client web obert– és transferible immediatament.

A entitats cíviques i col·lectius professionals . La demostració que és tècnicament possible construir un sistema de votació descentralitzada de baix cost (les comissions

d'Algorand són de l'ordre de 0,1 cèntims per transacció [2]) sense renunciar a la verificabilitat pública. El projecte mostra una alternativa viable als sistemes comercials privats emprats actualment per moltes entitats.

A la comunitat acadèmica i de recerca en votació electrònica . Un cas d'estudi documentat sobre la combinació Algorand + Ethereum + Schulze. Tot i no aportar primitives criptogràfiques noves, el projecte documenta amb rigor les restriccions reals de construir un sistema de votació amb *blockchain* en un horitzó temporal acotat, una informació útil per a futurs equips que afrontin un projecte similar.

A l'equip mateix . Quatre estudiants amb una experiència formativa significativa sobre tecnologies emergents (*blockchain*, *smart contracts*, criptografia aplicada), pràctiques d'enginyeria del *software* (GitHub Projects, Architecture Decision Records, C4, *Conventional Commits*) i treball en equip estructurat. Les competències adquirides són transferibles immediatament a contextos professionals.

7.4 Tancament i línies de treball futures

Si l'equip continués el projecte més enllà de l'assignatura, l'ordre lògic de prioritats seria el següent:

1. Incorporar la capa de **privacitat criptogràfica** (xifratge homomòrfic ElGamal + ZK-SNARK Groth16) com a primer pas indispensable abans de qualsevol desplegament real. Aquesta és la barrera més clara que separa el sistema actual d'un sistema productiu.
2. Integrar la **cerimònia de trustees** amb DKG i desxifrat distribuït, completant el model d'amenaques de manera coherent amb l'arquitectura criptogràfica.
3. Integrar un **Identity Provider** extern per a l'autenticació dels votants i la derivació de claus determinista.
4. Sotmetre el sistema a una **auditoria de seguretat externa** dels dos *smart contracts*.
5. Realitzar un **desplegament pilot** amb una entitat universitària real per a una elecció de baix risc (per exemple, l'elecció dels delegats d'una assignatura) que serveixi de banc de proves abans d'eleccions amb implicacions més grans.

El sistema actual representa un punt de partida sòlid des del qual es pot construir sense haver de recomençar. Aquesta és, en última instància, la millor mètrica de l'èxit del projecte: no és el sistema definitiu, però és una base sobre la qual el sistema definitiu es pot construir amb confiança.

ÚS DE LA INTEL·LIGÈNCIA ARTIFICIAL

L'equip ha adoptat un conjunt de principis inspirats en les recomanacions de la Unió Europea sobre intel·ligència artificial fiable (*Trustworthy AI*) i en els principis recollits a l'*AI Act* europeu.

Els principis aplicats durant el desenvolupament del projecte són els següents:

- **Supervisió humana (Human oversight)** Les eines d'intel·ligència artificial s'han utilitzat exclusivament com a assistents tècnics. Totes les decisions d'arquitectura, disseny i implementació han estat preses pels membres de l'equip de desenvolupament.
- **Responsabilitat del desenvolupador** L'equip assumeix la responsabilitat completa sobre el codi final desenvolupat i sobre les decisions tècniques adoptades en el sistema.
- **Verificació del contingut generat** Qualsevol suggeriment generat per eines d'intel·ligència artificial ha estat analitzat críticament, validat i, si escau, adaptat pels desenvolupadors abans d'incorporar-se al projecte.
- **Transparència en l'ús d'eines** L'ús d'eines d'intel·ligència artificial es documenta explícitament en aquesta memòria amb l'objectiu de garantir transparència acadèmica i evitar qualsevol ambigüitat sobre l'autoria del treball fet.
- **Protecció de dades i privacitat** Durant l'ús de models de llenguatge no s'ha introduït informació confidencial, claus criptogràfiques, credencials d'accés ni dades personals reals.

8.1 Eines d'intel·ligència artificial utilitzades

Durant el desenvolupament del projecte s'han utilitzat diverses eines d'intel·ligència artificial generativa per assistir en tasques d'aprenentatge, documentació tècnica i exploració de solucions.

8.1.1 Models de llenguatge (LLM)

S'han utilitzat models de llenguatge de gran mida com a assistents tècnics per a:

- aclariment de conceptes tècnics complexos
- exploració de patrons d'arquitectura
- generació d'esborranys de documentació
- revisió conceptual de fragments de codi
- generació d'exemples d'implementació

Entre els models utilitzats es troben:

- **ChatGPT (OpenAI)** Utilitzat com a assistent conversacional per a la comprensió de conceptes tècnics relacionats amb la cadena de blocs, contractes intel·ligents i desenvolupament d'aplicacions descentralitzades.
- **Gemini (Google DeepMind)** Utilitzat com a eina complementària per contrastar explicacions tècniques i explorar diferents enfocaments d'implementació.

És important destacar que els models de llenguatge s'han utilitzat únicament com a eines de suport conceptual i mai com a substitut del treball de desenvolupament fet per l'equip.

8.1.2 Eines de prototipatge i generació d'interfícies

Durant les primeres fases del projecte també s'han utilitzat eines de generació de prototips i assistents visuals basats en intel·ligència artificial.

- **Antigravity** Eina utilitzada per generar prototips ràpids d'interfície i explorar possibles estructures visuals durant la fase de disseny del *front-end*.

Aquestes eines han permès accelerar la fase inicial de disseny sense substituir el desenvolupament manual del sistema final.

8.2 Limitacions dels models de llenguatge

Els models de llenguatge presenten limitacions tècniques importants que han de ser considerades durant la seva utilització:

- poden generar informació incorrecta o incompleta
- no posseeixen coneixement actualitzat de tots els entorns tecnològics
- no comprenen completament el context d'un sistema complex
- no garanteixen la correcció del codi generat

Per aquest motiu, qualsevol contingut generat ha estat tractat com un suggeriment inicial i no com una font de veritat tècnica.

8.3 IA com a eina de suport a l'aprenentatge

En el context acadèmic d'aquest projecte, les eines d'intel·ligència artificial han contribuït principalment a:

- accelerar el procés d'aprenentatge de tecnologies emergents
- facilitar l'exploració d'alternatives arquitectòniques
- millorar la qualitat de la documentació tècnica
- donar suport a la resolució de dubtes conceptuals

Aquest enfocament reflecteix la realitat actual de l'enginyeria de *software* moderna, on els assistents basats en intel·ligència artificial s'estan integrant progressivament en els fluxos de treball dels desenvolupadors.

No obstant això, el valor intel·lectual del projecte, incloent-hi el disseny del sistema, la implementació dels contractes intel·ligents i l'arquitectura final de la solució, correspon íntegrament al treball fet pels membres de l'equip de desenvolupament.



ANNEXOS

A.1 Captures de pantalla de la interfície

Aquest annex recull totes les captures de pantalla referenciades al capítol de Disseny (**Capítol 5**), agrupades per àrea funcional de la interfície.

A.1.1 Pantalla d'inici

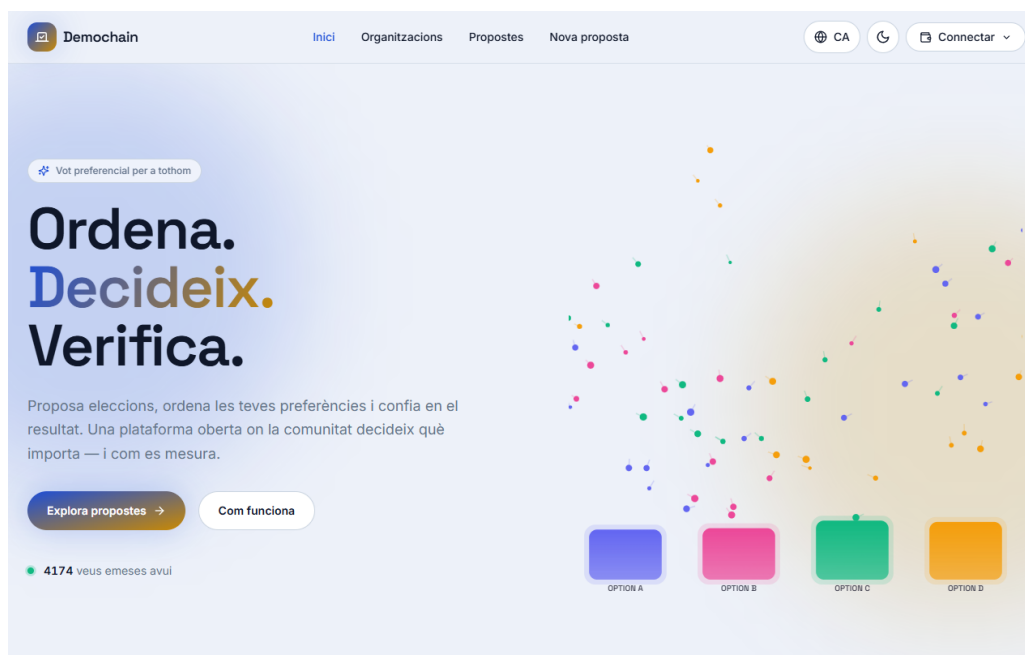


Figura A.1: Pantalla d'inici de DemoChain. Capçalera de navegació, selector d'idioma i connexió de cartera.

A.1.2 Gestió d'organitzacions

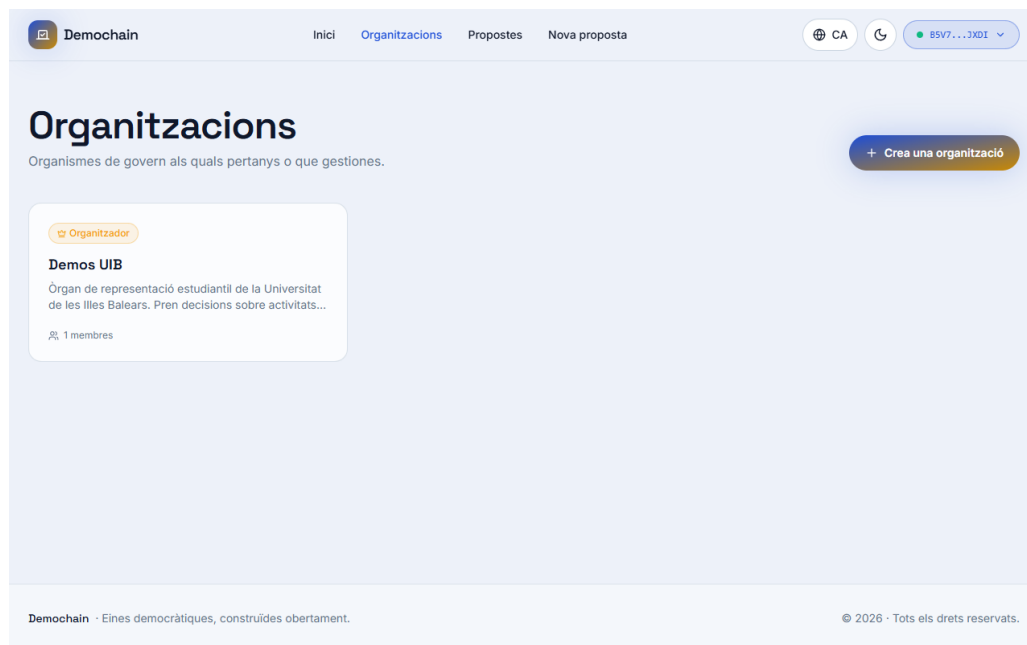


Figura A.2: Llistat d'organitzacions a què pertany el votant connectat.

A.1. Captures de pantalla de la interfície

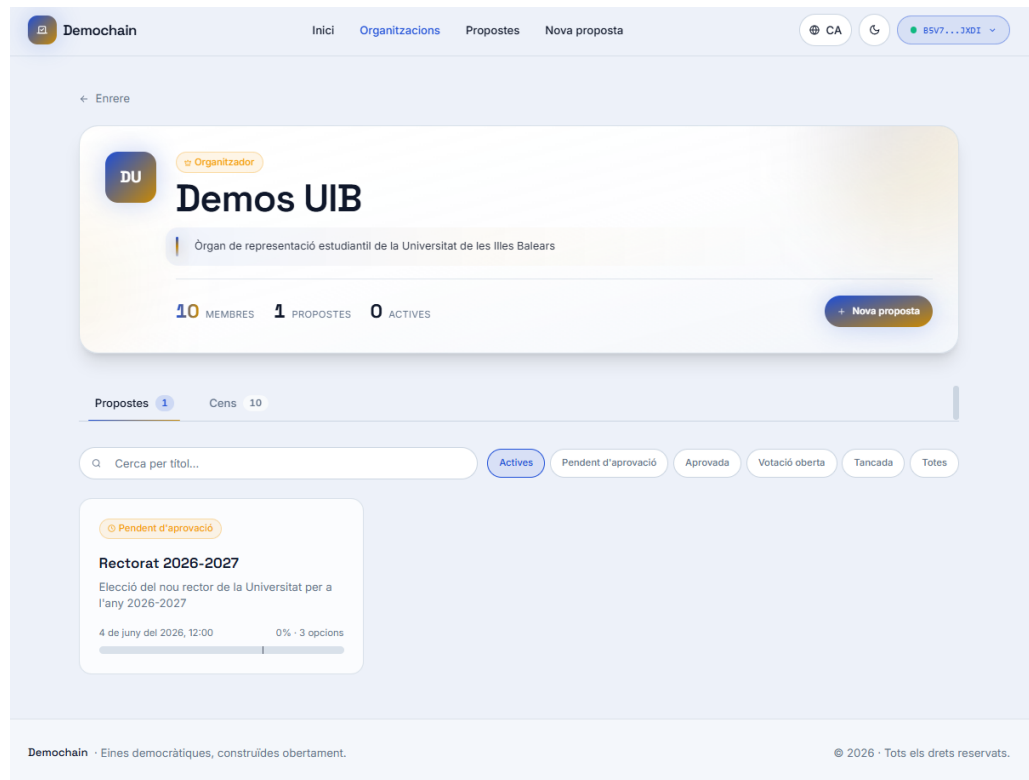


Figura A.3: Detall d'una organització, amb pestanyes de propostes i cens. L'organització té una proposta activa, encara sense vots d'aprovació.

A.1.3 Propostes

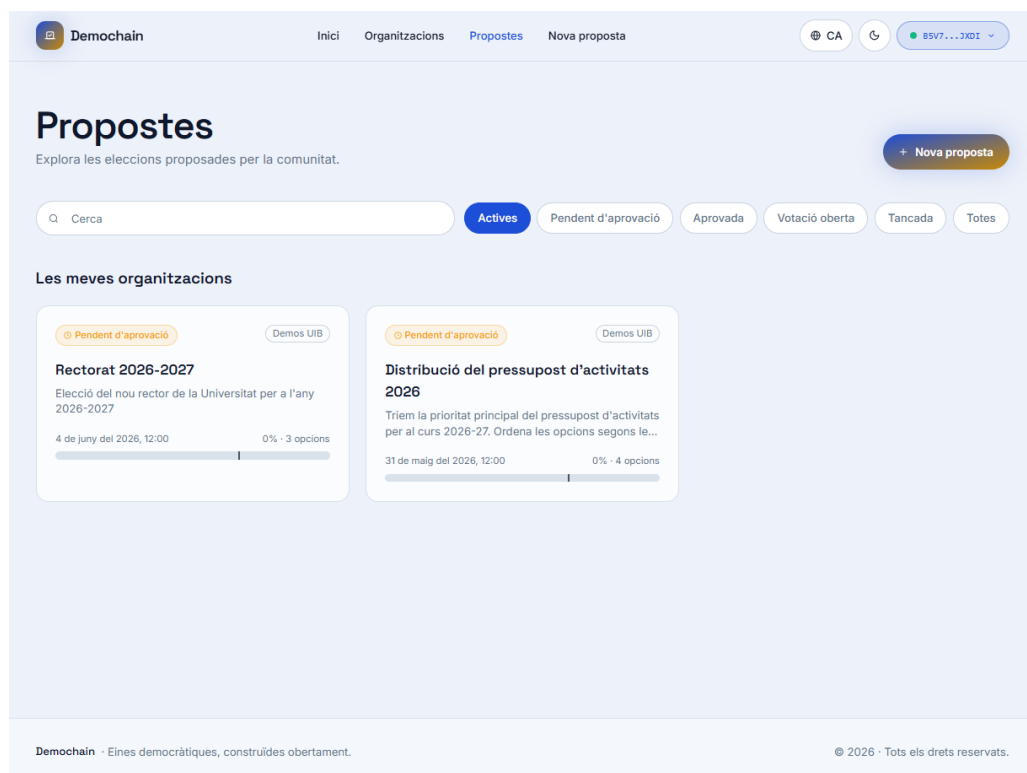


Figura A.4: Llistat de propostes amb filtres per estat. L'usuari que veu aquesta pantalla té dues propostes que estan *pendents d'aprovació* de l'organització “*Demos UIB*” (organització a la que pertany).



Figura A.5: Informació concreta d'una proposta: títol, resum, opcions, recompte de vots i formulari per aprovar o rebutjar la proposta.



Figura A.6: Informació concreta d'una proposta que s'ha aprovat i encara no ha començat el període de votació.

A.1.4 Creació d'una proposta

Demochain

Inici Organitzacions Propostes Nova proposta

CA 85V7...3XDT

PAS 1 DE 5

Crea una nova proposta

1 Organització 2 Bàsics 3 Dates 4 Opcions 5 Revisió

Selecciona l'organització a la qual pertanyerà aquesta proposta.

☒ **Demos UIB**
Òrgan de representació estudiantil de la Universitat de les Illes Balears

☐ **Estudiants EPS**
Associació d'estudiants de l'Escola Politècnica Superior

← Anterior Següent →

Demochain · Eines democràtiques, construïdes obertament.

© 2026 · Tots els drets reservats.

Figura A.7: Assistent de creació d'una proposta, pas *Organització*.

A. ANNEXOS

The screenshot shows the 'Crea una nova proposta' (Create a new proposal) form in the 'Bàsics' (Basics) step. The progress bar indicates the current step is 2 of 5. The form includes a title field with the text 'Distribució del pressupost d'activitats 2026' and a description field with the text 'Triem la prioritat principal del pressupost d'activitats per al curs 2026-27. Ordena les opcions segons les teves preferències.' Navigation buttons 'Anterior' and 'Següent' are at the bottom.

PAS 2 DE 5

Crea una nova proposta

Organització — 2 Bàsics — 3 Dates — 4 Opcions — 5 Revisió

TÍTOL DE LA PROPOSTA

Distribució del pressupost d'activitats 2026

DESCRIPCIÓ

Triem la prioritat principal del pressupost d'activitats per al curs 2026-27. Ordena les opcions segons les teves preferències.

← Anterior Següent →

Demochain · Eines democràtiques, construïdes obertament. © 2026 · Tots els drets reservats.

Figura A.8: Assistent de creació d'una proposta, pas *Bàsics*.

The screenshot shows the 'Crea una nova proposta' (Create a new proposal) form in the 'Dates' step. The progress bar indicates the current step is 3 of 5. The form includes a 'LA VOTACIÓ COMENÇA' (Voting starts) field with the text '25 may 2026, 12:00' and a 'LA VOTACIÓ ACABA' (Voting ends) field with the text 'Selecciona fi'. A calendar modal is open, showing the date '25' selected. Navigation buttons 'Anterior' and 'Següent' are at the bottom.

PAS 3 DE 5

Crea una nova proposta

Organització — Bàsics — 3 Dates — 4 Opcions — 5 Revisió

LA VOTACIÓ COMENÇA

25 may 2026, 12:00

LA VOTACIÓ ACABA

Selecciona fi

Següent →

May 2026

SU	MO	TU	WE	TH	FR	SA
26	27	28	29	30	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

12 : 00

Cancel·la Confirma

Demochain · Eines democràtiques, construïdes obertament. © 2026 · Tots els drets reservats.

Figura A.9: Assistent de creació d'una proposta, pas *Dates*.

A.1. Captures de pantalla de la interfície

The screenshot shows a web application interface for creating a proposal. The header includes the 'Demochain' logo, navigation links for 'Inici', 'Organitzacions', 'Propostes', and 'Nova proposta', and a user profile section with a language selector set to 'CA' and a balance of '85V7...3XD1'.

The main content area is titled 'PAS 4 DE 5' and 'Crea una nova proposta'. A progress bar indicates the current step is '4 Opcions', with previous steps '1 Organització', '2 Bàsics', '3 Dates', and '5 Revisió' marked as completed.

The 'Opcions' section contains a list of four options, each with a numbered circle and a trash icon:

- 1 Concerts
- 2 Pistes de padel
- 3 Llibreria
- 4 Ampliar l'estacionament

Below the list is a button labeled '+ Afegeix opció'. At the bottom of the form are two navigation buttons: '← Anterior' and 'Següent →'.

The footer contains the text 'Demochain · Eines democràtiques, construïdes obertament.' and '© 2026 · Tots els drets reservats.'

Figura A.10: Assistent de creació d'una proposta, pas *Opcions*.

The screenshot shows the 'Crea una nova proposta' (Create a new proposal) form in the Demochain application. The form is at step 5 of 5, labeled 'PAS 5 DE 5'. The title is 'Crea una nova proposta'. The progress bar shows five steps: Organització, Bàsics, Dates, Opcions, and Revisió (current step). The form fields are as follows:

- ORGANITZACIONS:** Demos UIB
- TÍTOL:** Distribució del pressupost d'activitats 2026
- DESCRIPCIÓ:** Triem la prioritat principal del pressupost d'activitats per al curs 2026-27. Ordena les opcions segons les teves preferències.
- DATA D'INICI:** 25 may 2026, 12:00
- DATA DE FI:** 31 may 2026, 12:00
- OPCIONS:**
 - 1 Concerts
 - 2 Pistes de padel
 - 3 Llibreria
 - 4 Ampliar l'estacionament

At the bottom, there is a button 'Anterior' (Previous) and a button 'Publica la proposta' (Publish the proposal) with a checkmark icon.

Figura A.11: Assistent de creació d'una proposta, pas *Revisió*.

A.1.5 Emissió de vot

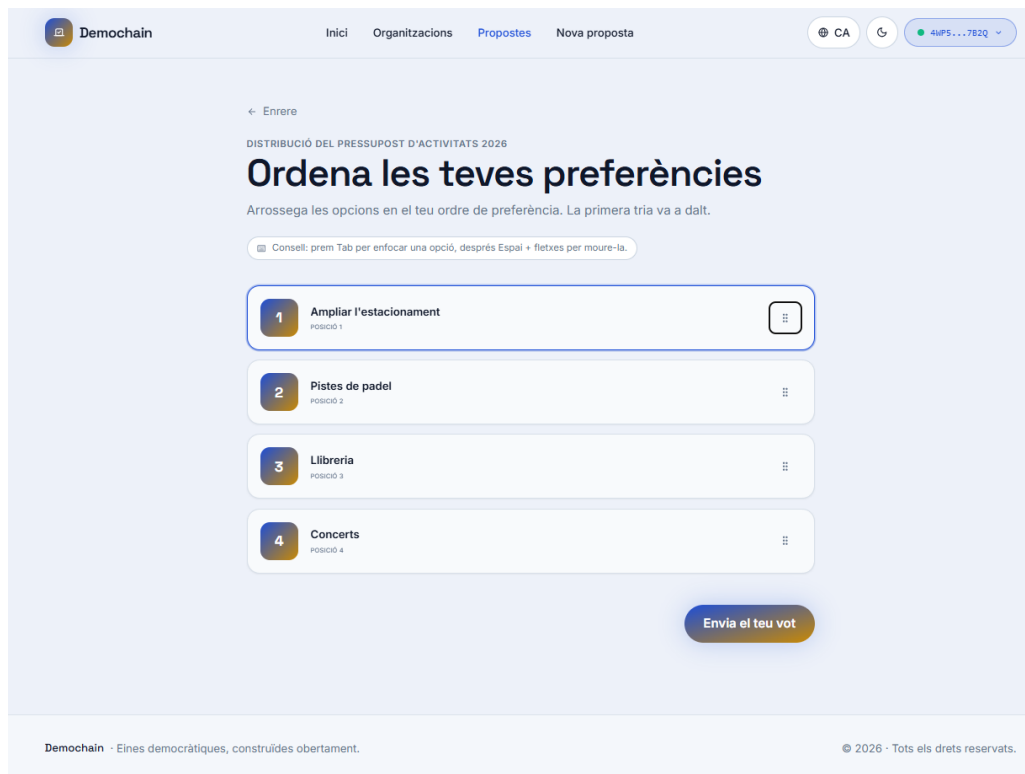


Figura A.12: Pantalla d'emissió de vot: ranking de preferències arrossegables.

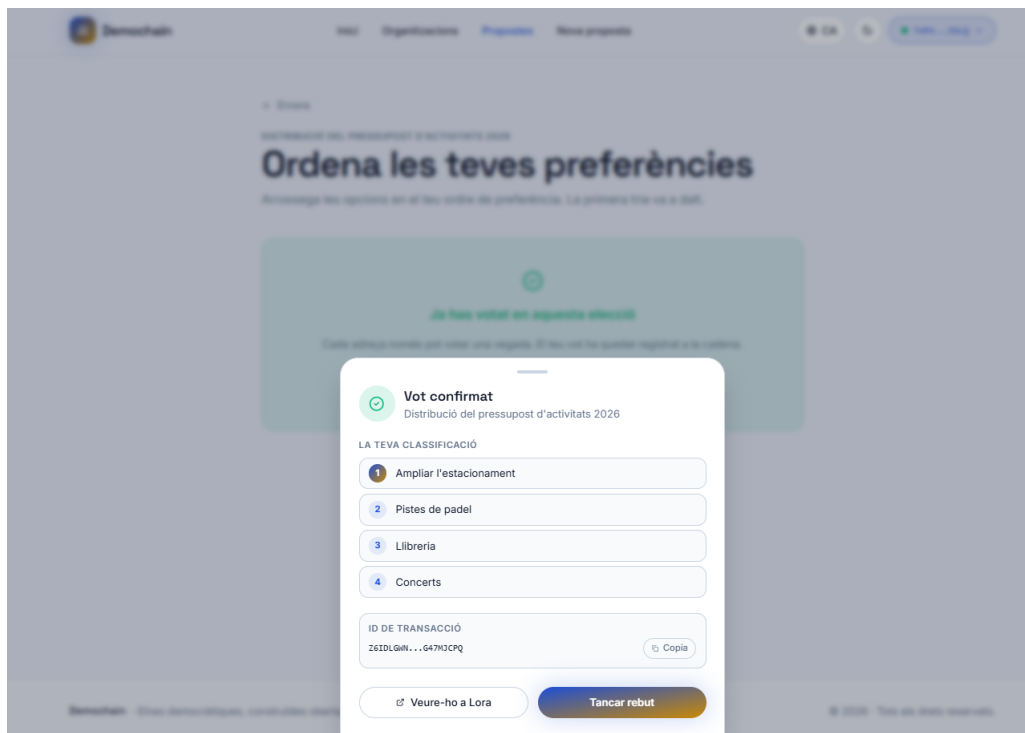


Figura A.13: Pantalla dde vot confirmat.

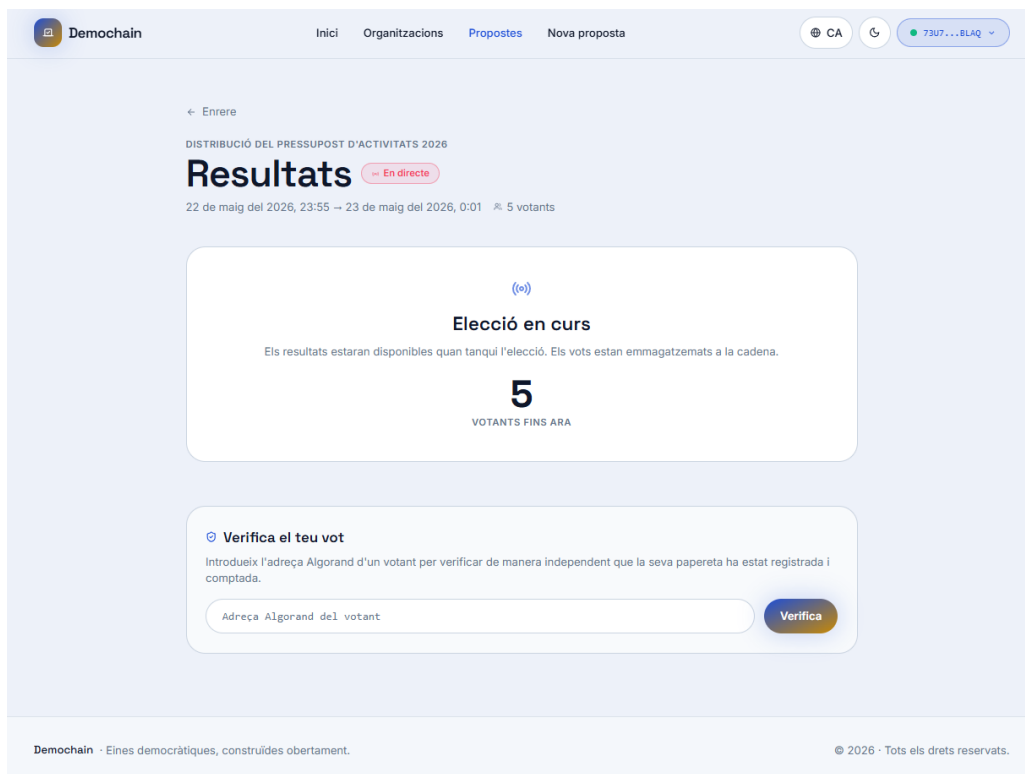


Figura A.14: Pantalla de seguiment d'enviament de vots

A.1.6 Resultats i verificació pública

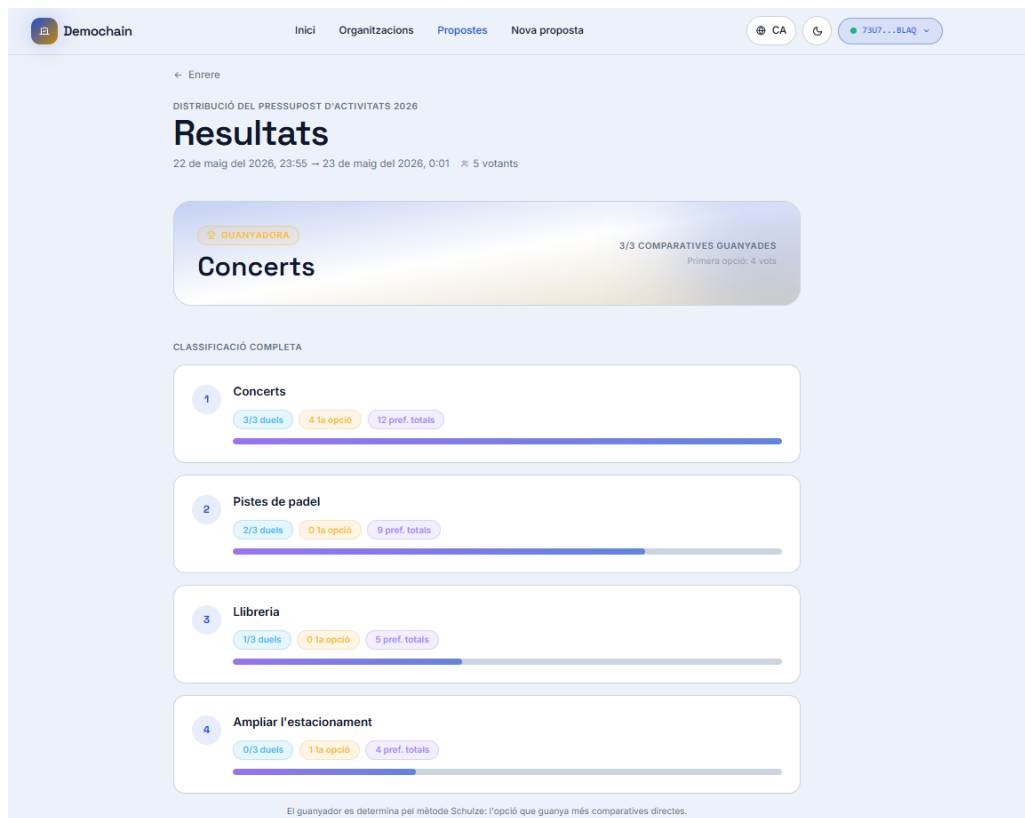


Figura A.15: Pantalla de resultats finals i verificació pública.

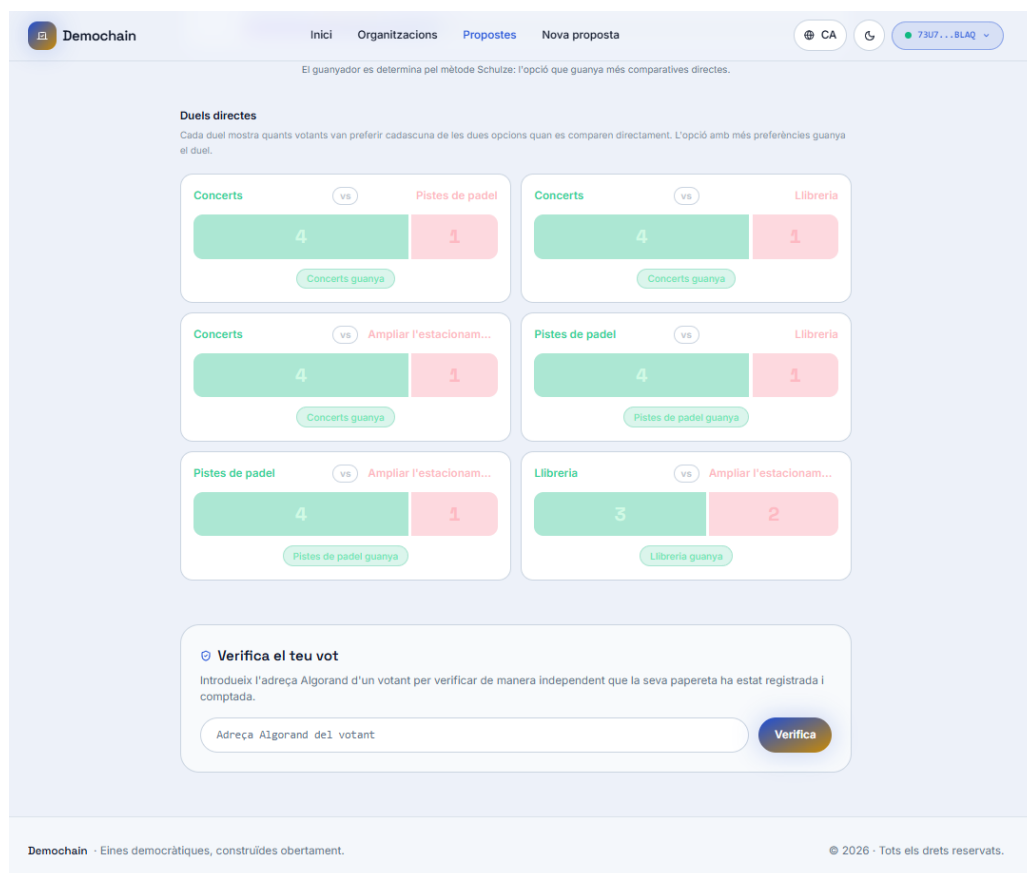


Figura A.16: Pantalla de resultats amb els duels directes.

A.1. Captures de pantalla de la interfície

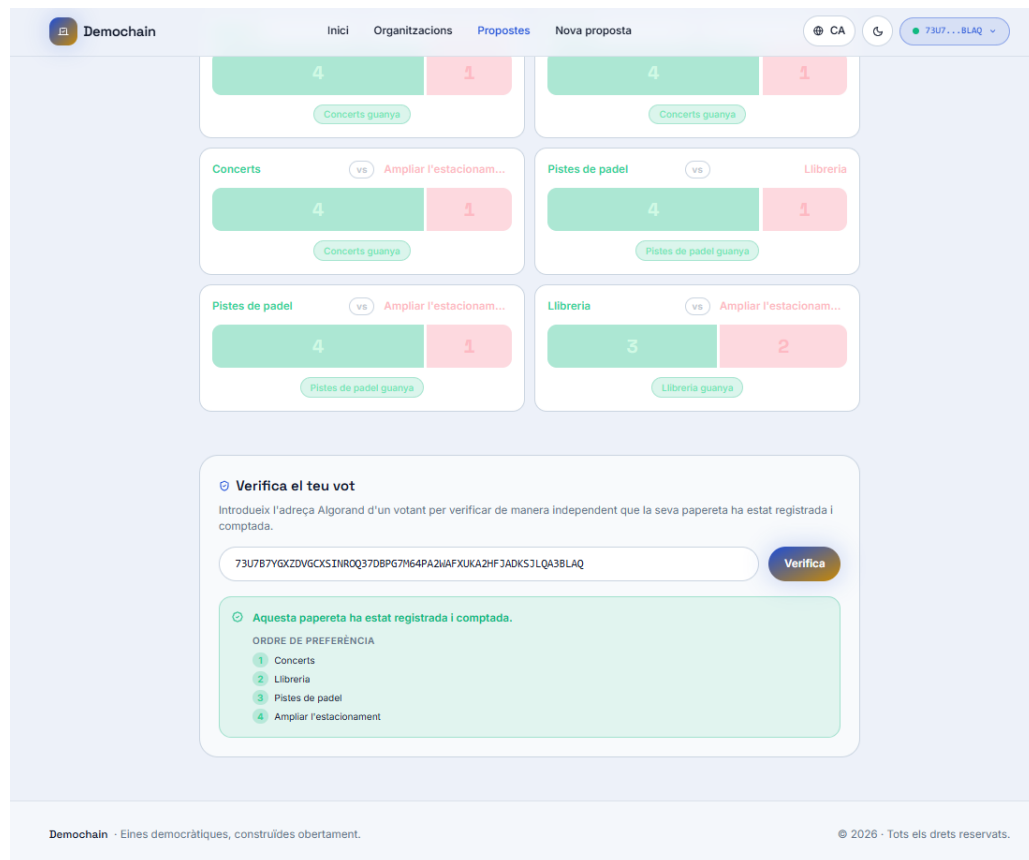


Figura A.17: Pantalla de verificació del vot que s'ha emès una vegada acabada l'elecció.

BIBLIOGRAFIA

- [1] S. Brown, *The C4 model for visualising software architecture*. Leanpub, 2018. [Online]. Available: <https://c4model.com> 1.1, 1.3.2, 3, 3.4, 6.1.1, 6.3.2, 7.1.1
- [2] J. Chen and S. Micali, “Algorand: A secure and efficient distributed ledger,” in *Theoretical Computer Science*, vol. 777, 2019, pp. 155–183. 1.3.1, 6.3.1, 7.3
- [3] Algorand Foundation, “Algorand Python (algopy) i compilador Puya,” <https://algorandfoundation.github.io/puya/>, 2024, consultat el maig de 2026. 1.3.1, 3.1.1, 6.3.2, 6.5.2, 7.1.1
- [4] P. S. L. M. Barreto and M. Naehrig, “Pairing-friendly elliptic curves of prime order,” in *Selected Areas in Cryptography – SAC 2005*, ser. Lecture Notes in Computer Science, vol. 3897, 2006, pp. 319–331. 1.3.1
- [5] V. Buterin, “Ethereum: A next-generation smart contract and decentralized application platform,” <https://ethereum.org/en/whitepaper/>, 2014. 1.3.1, 6.3.1
- [6] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020. [Online]. Available: <https://scrumguides.org> 1.3.2, 1.7, 3.1
- [7] Conventional Commits, “Conventional Commits 1.0.0,” <https://www.conventionalcommits.org/>, 2023. 1.3.2, 3.2.3, 3.7, 3.7.2, 7.1.1, 7.2.1
- [8] M. Schulze, “A new monotonic, clone-independent, reversal symmetric, and condorcet-consistent single-winner election method,” *Social Choice and Welfare*, vol. 36, no. 2, pp. 267–303, 2011. 3, 1.5, 3.1.1, 6.3.2, 6.4.1, 7.1.1
- [9] Vocdoni, “DAVINCI Protocol: A digital voting protocol designed for mass adoption,” <https://davinci.vote/>, 2026, consultat el maig de 2026. Codi font: <https://github.com/vocdoni>, llicència AGPL-3.0. 1.5, 2.3.1, 7.1.2
- [10] T. ElGamal, “A public key cryptosystem and a signature scheme based on discrete logarithms,” *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985. 1.6, 2.2.1, 2.2.2, 2.2.3, 2.3.1, 7.1.3
- [11] B. Adida, “Helios: Web-based open-audit voting,” in *Proceedings of the 17th USENIX Security Symposium*, 2008, pp. 335–348. [Online]. Available: https://www.usenix.org/legacy/event/sec08/tech/full_papers/adida/adida.pdf 1.6, 2.2.1, 6.5.1, 7.1.3

- [12] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989. 1.6, 2.2.1, 2.2.3, 2.3.1, 4.2, 7.1.3
- [13] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, ser. Lecture Notes in Computer Science, vol. 9666, 2016, pp. 305–326. 1.6, 2.3.1, 4.2, 7.1.3
- [14] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, “Secure distributed key generation for discrete-log based cryptosystems,” in *Advances in Cryptology – EUROCRYPT 1999*, ser. Lecture Notes in Computer Science, vol. 1592, 1999, pp. 295–310. 1.6, 4.2, 7.1.3
- [15] D. Chaum and T. P. Pedersen, “Wallet databases with observers,” in *Advances in Cryptology – CRYPTO 1992*, ser. Lecture Notes in Computer Science, vol. 740, 1992, pp. 89–105. 1.6, 7.1.3
- [16] V. Cortier, P. Gaudry, and S. Glondou, “Belenios: A simple private and verifiable electronic voting system,” in *Foundations of Security, Protocols, and Equational Reasoning*, ser. Lecture Notes in Computer Science, vol. 11565, 2019, pp. 214–238. 1.6, 2.2.2, 6.5.1
- [17] J. Vanyó Mestre, D. L. Canning Garcia, A. Contestí Coll, and M. Link Cladera, “Demochain: sistema de votació descentralitzat (codi font),” <https://github.com/jvanyom/demochain>, 2026. 1.6, 1.8, 3.1.1, 3.2, 6.2, 7.1.3, 7.1.3, 4
- [18] e-Estonia, “i-Voting: Internet voting in Estonia,” <https://e-estonia.com/solutions/e-governance/i-voting/>, 2024, consultat el maig de 2026. 2.1.1
- [19] D. Springall, T. Finkenauer, Z. Durumeric, J. Kitcat, H. Hursti, M. MacAlpine, and J. A. Halderman, “Security analysis of the Estonian Internet voting system,” in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2014, pp. 703–715. 2.1.1
- [20] S. J. Lewis, O. Pereira, and V. Teague, “Ceci n’est pas une preuve: The use of trapdoor commitments in Bayer-Groth proofs and the implications for the verifiability of the Scytl-SwissPost Internet voting system,” <https://people.eng.unimelb.edu.au/vjteague/SwissVote.html>, 2019. 2.1.2
- [21] M. A. Specter, J. Koppel, and D. Weitzner, “The ballot is busted before the blockchain: A security analysis of Voatz, the first Internet voting application used in U.S. federal elections,” in *Proceedings of the 29th USENIX Security Symposium*, 2020, pp. 1535–1553. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/specter> 2.1.3
- [22] Microsoft Research, “ElectionGuard: An open source software development kit (sdk) for end-to-end verifiable elections,” <https://www.electionguard.vote/>, 2019, consultat el maig de 2026. 2.2.3

-
- [23] J. Benaloh and D. Tuinstra, “Receipt-free secret-ballot elections,” in *Proceedings of the 26th Annual ACM Symposium on Theory of Computing (STOC)*, 1994, pp. 544–553. 2.3.1, 7.1.3
- [24] R. W. Floyd, *Algorithm 97: Shortest Path*. Communications of the ACM, 1962, vol. 5, no. 6. 3.1.1, 6.3.4, 6.4.1
- [25] Algorand Foundation, “AlgoKit: Developer toolkit for algorand,” <https://developer.algorand.org/algokit/>, 2024, consultat el maig de 2026. 3.1.1, 6.3.1, 6.5.2
- [26] —, “ARC-4: Application Binary Interface (abi) per Algorand,” <https://arc.algorand.foundation/ARCs/arc-0004>, 2023, consultat el maig de 2026. 3.5.1, 6.4.1, 6.5.2
- [27] —, “ARC-56: Application Specification,” <https://arc.algorand.foundation/ARCs/arc-0056>, 2023, consultat el maig de 2026. 3.5.2, 6.4.1
- [28] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 of the European Parliament and of the Council (general data protection regulation),” <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016. 4.5.2
- [29] S. Micali, M. O. Rabin, and S. P. Vadhan, “Verifiable random functions,” in *Proceedings of the 40th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 1999, pp. 120–130. 4.6
- [30] K. J. Arrow, *Social Choice and Individual Values*, 2nd ed. Yale University Press, 1963. 4.7.1
- [31] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. 4.8
- [32] V. Buterin *et al.*, “EIP-1559: Fee market change for ETH 1.0 chain,” <https://eips.ethereum.org/EIPS/eip-1559>, 2019. 6.4.1